THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram-Tamil Nadu

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that Ms.**S.RASHIMA BANU (20524UCSC070)** a student of III B.Sc. Computer Science, has successfully completed the practice-based practical learning component of **Advanced Database Management System – Practical (UCSR513)** during the academic year 2026–2027.

The practical examination will be held on **16.10.2026**

Course In-chaRge: Dr. S Manimekalai

Head of the Department: Ms. K Manohari

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** • CREATE • ALTER • DROP • TRUNCATE		
2	**DML Commands** • INSERT • UPDATE • DELETE		
3	**DQL Commands** • SELECT • WHERE • ORDER BY • GROUP BY • HAVING		
4	**Transactions** • COMMIT • ROLLBACK • SAVEPOINT		
5	**Joins** • Inner Join • Left Join • Right Join • Full Join • Self Join		
6	**SQL Functions** • Aggregate Functions • String Functions • Date Functions		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		
10	**Triggers**		
11	**Cursor Implementation**		
12	**PL/SQL Exception Handling**		
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		
14	**Practical Sql Queries**		
15	**Mini project**		
16	**Annexures:** • Khan Academy • Hacker Rank Code Chef		

EX. NO: 01 Creation of DDL Operations Using Vehicle Table

DATE:09/07/2026

Aim

To create a **Vehicle** table, insert vehicle records, display all records, and retrieve vehicles manufactured from the year 2020 onwards.

Software Used

Oracle / MySQL / SQLite3

Description

DDL (Data Definition Language) is used to create and modify the structure of database objects.

DDL COMMANDS

- CREATE
- ALTER
- DROP
- TRUNCATE

CREATE

CREATE command is used to create a new table and define its columns and constraints.

Syntax:

```
CREATE TABLE table_name (
    column1 datatype constraints,
    column2 datatype constraints,
    column3 datatype constraints
);
```

Table Structure:

```
CREATE TABLE vehicle (
    id INTEGER PRIMARY KEY,
    name TEXT,
    brand TEXT,
    manufacture_year INTEGER
);
```

Table Created

ALTER

ALTER command is used to alter a table or make changes to a table.

It has methods such as **ADD, MODIFY, DROP and RENAME**.

ADD

ADD command is used to add a new column with a data type.

Syntax:

```
ALTER TABLE table_name ADD column_name datatype;
```

Query:

```
ALTER TABLE vehicle ADD price INTEGER;
```

Table Altered

MODIFY

MODIFY command is used to change the datatype or size of a column.

Syntax:

```
ALTER TABLE table_name MODIFY column_name datatype;
```

Query:

```
ALTER TABLE vehicle MODIFY name VARCHAR(150);
```

DROP COLUMN

DROP command is used to delete a column from the table.

Syntax:

```
ALTER TABLE table_name DROP COLUMN column_name;
```

Query:

```
ALTER TABLE vehicle DROP COLUMN price;
```

RENAME

RENAME command is used to change the name of a table.

Query:

```
ALTER TABLE vehicle RENAME TO vehicles;
```

Column Rename:

```
ALTER TABLE vehicle RENAME COLUMN name TO vehicle_name;
```

TRUNCATE

TRUNCATE command is used to delete all records from a table while keeping the table structure.

Syntax:

```
TRUNCATE TABLE table_name;
```

Query:

```
TRUNCATE TABLE vehicle;
```

DROP

DROP command is used to delete the table and all its data permanently.

Syntax:

```
DROP TABLE table_name;
```

Query:

```
DROP TABLE vehicle;
```

Explanation

- Create a table named **vehicle**.
- Insert vehicle details into the table.

- Display all vehicle records.
- Retrieve vehicles manufactured from the year **2020 onwards**.
- Sort the vehicles according to their manufacturing year.

Create Table

```
CREATE TABLE vehicle (
    id INTEGER PRIMARY KEY,
    name TEXT,
    brand TEXT,
    manufacture_year INTEGER
);
```

Sample Data

```
INSERT INTO vehicle VALUES (1, 'Swift', 'Maruti', 2019);
INSERT INTO vehicle VALUES (2, 'Creta', 'Hyundai', 2021);
INSERT INTO vehicle VALUES (3, 'Nexon', 'Tata', 2020);
INSERT INTO vehicle VALUES (4, 'City', 'Honda', 2022);
INSERT INTO vehicle VALUES (5, 'Fortuner', 'Toyota', 2018);
INSERT INTO vehicle VALUES (6, 'Thar', 'Mahindra', 2023);
```

SQL Query

Query 1: Display all vehicle records

```
SELECT * FROM vehicle;
```

Resultant Query

id	name	brand	manufacture_year
1	Swift	Maruti	2019
2	Creta	Hyundai	2021
3	Nexon	Tata	2020
4	City	Honda	2022
5	Fortuner	Toyota	2018
6	Thar	Mahindra	2023

Query 2: Display vehicles manufactured from the year 2020 onwards

```
SELECT *
FROM vehicle
WHERE manufacture_year >= 2020
ORDER BY manufacture_year;
```

Resultant Query

id	name	brand	manufacture_year
3	Nexon	Tata	2020
2	Creta	Hyundai	2021
4	City	Honda	2022
6	Thar	Mahindra	2023

RESULT:

Thus, the **Vehicle** table was created successfully, records were inserted, all vehicle details were displayed, and the vehicles manufactured from the year **2020 onwards** were retrieved successfully.

EX. NO: 02 DML OPERATIONS USING VEHICLE TABLE

DATE:12/07/2026

Aim

To perform **DML operations** such as INSERT, UPDATE and DELETE on a Vehicle table.

Software Used

Oracle / MySQL / SQLite3

Description

DML (Data Manipulation Language) is used to insert, modify and delete the data stored in a database table.

DML COMMANDS

- INSERT
- UPDATE
- DELETE

INSERT

INSERT command is used to add new records into a table.

Syntax:

```
INSERT INTO table_name VALUES (value1, value2, value3, value4);
```

Query:

```
INSERT INTO vehicle VALUES (1, 'Swift', 'Maruti', 2019);
INSERT INTO vehicle VALUES (2, 'Creta', 'Hyundai', 2021);
INSERT INTO vehicle VALUES (3, 'Nexon', 'Tata', 2020);
INSERT INTO vehicle VALUES (4, 'City', 'Honda', 2022);
INSERT INTO vehicle VALUES (5, 'Fortuner', 'Toyota', 2018);
INSERT INTO vehicle VALUES (6, 'Thar', 'Mahindra', 2023);
```

Result:

6 rows inserted successfully.

UPDATE

UPDATE command is used to modify existing records in a table.

Syntax:

```
UPDATE table_name
SET column_name = value
WHERE condition;
```

Query:

```
UPDATE vehicle
SET manufacture_year = 2020
WHERE id = 1;
```

Result:

1 row updated successfully.

DELETE

DELETE command is used to remove records from a table.

Syntax:

```
DELETE FROM table_name
WHERE condition;
```

Query:

```
DELETE FROM vehicle
WHERE id = 5;
```

Result:

1 row deleted successfully.

Explanation

- Create a Vehicle table.
- Insert vehicle records into the table.
- Update the manufacturing year of a vehicle.

- Delete a vehicle record from the table.
- Display the updated vehicle records.

Create Table

```
CREATE TABLE vehicle (
    id INTEGER PRIMARY KEY,
    name TEXT,
    brand TEXT,
    manufacture_year INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO vehicle VALUES (1, 'Swift', 'Maruti', 2019);
INSERT INTO vehicle VALUES (2, 'Creta', 'Hyundai', 2021);
INSERT INTO vehicle VALUES (3, 'Nexon', 'Tata', 2020);
INSERT INTO vehicle VALUES (4, 'City', 'Honda', 2022);
INSERT INTO vehicle VALUES (5, 'Fortuner', 'Toyota', 2018);
INSERT INTO vehicle VALUES (6, 'Thar', 'Mahindra', 2023);
```

SQL Query

Query 1: Display all vehicle records

```
SELECT * FROM vehicle;
```

Resultant Query

id	name	brand	manufacture_year
1	Swift	Maruti	2019
2	Creta	Hyundai	2021
3	Nexon	Tata	2020
4	City	Honda	2022
5	Fortuner	Toyota	2018
6	Thar	Mahindra	2023

Query 2: Update the manufacturing year of Swift

```
UPDATE vehicle
SET manufacture_year = 2020
WHERE id = 1;
```

Resultant Query

id	name	brand	manufacture_year
1	Swift	Maruti	2020
2	Creta	Hyundai	2021
3	Nexon	Tata	2020
4	City	Honda	2022
5	Fortuner	Toyota	2018
6	Thar	Mahindra	2023

Query 3: Delete the Fortuner record

```
DELETE FROM vehicle
      WHERE id = 5;
```

Resultant Query

id	name	brand	manufacture_year
1	Swift	Maruti	2020
2	Creta	Hyundai	2021
3	Nexon	Tata	2020
4	City	Honda	2022
6	Thar	Mahindra	2023

RESULT

Thus, the **DML operations INSERT, UPDATE and DELETE** were performed successfully on the Vehicle table.

EX. NO: 03 DQL OPERATIONS USING VEHICLE TABLE

DATE:16/07/2026

Aim

To perform **DQL operations** such as SELECT, WHERE, ORDER BY, GROUP BY and HAVING on a Vehicle table.

Software Used

Oracle / MySQL / SQLite3

Description

DQL (Data Query Language) is used to retrieve and display data from a database table.

DQL COMMANDS

- SELECT
- WHERE
- ORDER BY
- GROUP BY
- HAVING

SELECT

SELECT command is used to retrieve data from a table.

Syntax:

```
SELECT column1, column2 FROM table_name;
```

Query:

```
SELECT * FROM vehicle;
```

Result:

All vehicle records are displayed.

WHERE

WHERE clause is used to filter records based on a specified condition.

Syntax:

```
SELECT * FROM table_name WHERE condition;
```

Query:

```
SELECT * FROM vehicle WHERE manufacture_year >= 2020;
```

Result:

Vehicles manufactured from 2020 onwards are displayed.

ORDER BY

ORDER BY clause is used to sort the records in ascending or descending order.

Syntax:

```
SELECT * FROM table_name ORDER BY column_name;
```

Query:

```
SELECT * FROM vehicle ORDER BY manufacture_year;
```

Result:

Vehicle records are displayed in ascending order of manufacturing year.

GROUP BY

GROUP BY clause is used to group rows that have the same values in a specified column.

Syntax:

```
SELECT column_name, COUNT(*)
FROM table_name
GROUP BY column_name;
```

Query:

```
SELECT brand, COUNT(*)
FROM vehicle
GROUP BY brand;
```

Result:

The number of vehicles for each brand is displayed.

HAVING

HAVING clause is used to filter grouped records based on a condition.

Syntax:

```
SELECT column_name, COUNT(*)
FROM table_name
GROUP BY column_name
HAVING condition;
```

Query:

```
SELECT brand, COUNT(*)
FROM vehicle
GROUP BY brand
HAVING COUNT(*) >= 2;
```

Result:

Brands having two or more vehicles are displayed.

Explanation

- Create a Vehicle table.
- Insert vehicle records into the table.
- Retrieve all vehicle records using SELECT.
- Retrieve vehicles based on a condition using WHERE.
- Sort vehicle records using ORDER BY.
- Group vehicle records based on brand using GROUP BY.
- Filter grouped records using HAVING.

Create Table

```
CREATE TABLE vehicle (
    id INTEGER PRIMARY KEY,
    name TEXT,
    brand TEXT,
    manufacture_year INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO vehicle VALUES (1, 'Swift', 'Maruti', 2019);
INSERT INTO vehicle VALUES (2, 'Creta', 'Hyundai', 2021);
INSERT INTO vehicle VALUES (3, 'Nexon', 'Tata', 2020);
INSERT INTO vehicle VALUES (4, 'City', 'Honda', 2022);
INSERT INTO vehicle VALUES (5, 'Baleno', 'Maruti', 2020);
INSERT INTO vehicle VALUES (6, 'Thar', 'Mahindra', 2023);
INSERT INTO vehicle VALUES (7, 'i20', 'Hyundai', 2022);
INSERT INTO vehicle VALUES (8, 'Harrier', 'Tata', 2021);
```

SQL Query

Query 1: Display all vehicle records

```
SELECT * FROM vehicle;
```

Resultant Query

id	name	brand	manufacture_year
1	Swift	Maruti	2019
2	Creta	Hyundai	2021
3	Nexon	Tata	2020
4	City	Honda	2022
5	Baleno	Maruti	2020
6	Thar	Mahindra	2023
7	i20	Hyundai	2022
8	Harrier	Tata	2021

Query 2: Display vehicles manufactured from 2020 onwards

```
SELECT * FROM vehicle
WHERE manufacture_year >= 2020;
```

Resultant Query

id	name	brand	manufacture_year
2	Creta	Hyundai	2021
3	Nexon	Tata	2020
4	City	Honda	2022
5	Baleno	Maruti	2020
6	Thar	Mahindra	2023
7	i20	Hyundai	2022
8	Harrier	Tata	2021

Query 3: Display vehicles in ascending order of manufacturing year

```
SELECT * FROM vehicle
ORDER BY manufacture_year;
```

Resultant Query

id	name	brand	manufacture_year
1	Swift	Maruti	2019
3	Nexon	Tata	2020
5	Baleno	Maruti	2020
2	Creta	Hyundai	2021
8	Harrier	Tata	2021
4	City	Honda	2022
7	i20	Hyundai	2022
6	Thar	Mahindra	2023

Query 4: Group vehicles by brand

```
SELECT brand, COUNT(*)
FROM vehicle
GROUP BY brand;
```

Resultant Query

brand	COUNT(*)
Maruti	2
Hyundai	2

Tata	2
Honda	1
Mahindra	1

Query 5: Display brands having two or more vehicles

```
SELECT brand, COUNT(*)
FROM vehicle
GROUP BY brand
HAVING COUNT(*) >= 2;
```

Resultant Query

brand	COUNT(*)
Maruti	2
Hyundai	2
Tata	2

RESULT

Thus, the **DQL operations SELECT, WHERE, ORDER BY, GROUP BY and HAVING** were performed successfully on the Vehicle table.

EX. NO: 04 TRANSACTION CONTROL OPERATIONS USING VEHICLE TABLE

DATE:20/07/2026

Aim

To perform transaction operations such as **COMMIT, ROLLBACK and SAVEPOINT** on a Vehicle table.

Software Used

Oracle / MySQL / SQLite3

Description

Transaction commands are used to manage the changes made to the data in a database.

TRANSACTION COMMANDS

- COMMIT
- ROLLBACK
- SAVEPOINT

COMMIT

COMMIT command is used to permanently save the changes made during a transaction.

Syntax:

```
COMMIT;
```

Query:

```
INSERT INTO vehicle VALUES (9, 'Verna', 'Hyundai', 2024);
COMMIT;
```

Result:

The changes are permanently saved.

ROLLBACK

ROLLBACK command is used to undo the changes made after the last COMMIT.

Syntax:

```
ROLLBACK;
```

Query:

```
DELETE FROM vehicle WHERE id = 9;
ROLLBACK;
```

Result:

The deleted record is restored.

SAVEPOINT

SAVEPOINT command is used to create a temporary point within a transaction. The changes can be rolled back to that particular point.

Syntax:

```
SAVEPOINT savepoint_name;
```

Query:

```
INSERT INTO vehicle VALUES (10, 'Kia Seltos', 'Kia', 2023);
SAVEPOINT vehicle_savepoint;
DELETE FROM vehicle WHERE id = 10;
ROLLBACK TO vehicle_savepoint;
COMMIT;
```

Result:

The transaction is rolled back to the specified savepoint and the changes before the savepoint are retained.

Explanation

- Create a Vehicle table.
- Insert vehicle records into the table.
- Use COMMIT to permanently save changes.
- Use ROLLBACK to undo changes.
- Use SAVEPOINT to create a point within a transaction.
- Roll back the transaction to the specified savepoint.

Create Table

```
CREATE TABLE vehicle (
    id INTEGER PRIMARY KEY,
    name TEXT,
    brand TEXT,
    manufacture_year INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO vehicle VALUES (1, 'Swift', 'Maruti', 2019);
INSERT INTO vehicle VALUES (2, 'Creta', 'Hyundai', 2021);
INSERT INTO vehicle VALUES (3, 'Nexon', 'Tata', 2020);
INSERT INTO vehicle VALUES (4, 'City', 'Honda', 2022);
INSERT INTO vehicle VALUES (5, 'Thar', 'Mahindra', 2023);
COMMIT;
```

SQL Query

Query 1: Insert a new vehicle and commit the transaction

```
INSERT INTO vehicle VALUES (6, 'Venue', 'Hyundai', 2024);
COMMIT;
```

Resultant Query

```
SELECT * FROM vehicle;
```

id	name	brand	manufacture_year
1	Swift	Maruti	2019
2	Creta	Hyundai	2021

3	Nexon	Tata	2020
4	City	Honda	2022
5	Thar	Mahindra	2023
6	Venue	Hyundai	2024

Query 2: Delete a record and rollback

```
DELETE FROM vehicle WHERE id = 6;
ROLLBACK;
```

Resultant Query

```
SELECT * FROM vehicle;
```

id	name	brand	manufacture_year
1	Swift	Maruti	2019
2	Creta	Hyundai	2021
3	Nexon	Tata	2020
4	City	Honda	2022
5	Thar	Mahindra	2023
6	Venue	Hyundai	2024

Query 3: Using SAVEPOINT

```
INSERT INTO vehicle VALUES (7, 'Seltos', 'Kia', 2023);
SAVEPOINT vehicle_savepoint;
DELETE FROM vehicle WHERE id = 7;
ROLLBACK TO vehicle_savepoint;
COMMIT;
```

Resultant Query

```
SELECT * FROM vehicle;
```

id	name	brand	manufacture_year
1	Swift	Maruti	2019
2	Creta	Hyundai	2021
3	Nexon	Tata	2020
4	City	Honda	2022
5	Thar	Mahindra	2023
6	Venue	Hyundai	2024
7	Seltos	Kia	2023

RESULT

Thus, the transaction operations **COMMIT, ROLLBACK and SAVEPOINT** were performed successfully on the Vehicle table.

EX. NO: 05 IMPLEMENTATION OF JOINS USING VEHICLE AND OWNER TABLES

DATE:23/07/2026

Aim

To perform different types of SQL Joins such as **INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN and SELF JOIN** using Vehicle and Owner tables.

Software Used

Oracle / MySQL / SQLite3

Description

A **JOIN** is used to combine data from two or more tables based on a related column between them.

JOIN COMMANDS

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL JOIN
- SELF JOIN

INNER JOIN

INNER JOIN returns only the records that have matching values in both tables.

Syntax:

```
SELECT columns
FROM table1
```

```
INNER JOIN table2
ON table1.column = table2.column;
```

Query:

```
SELECT vehicle.name, owner.owner_name
FROM vehicle
INNER JOIN owner
ON vehicle.owner_id = owner.owner_id;
```

LEFT JOIN

LEFT JOIN returns all records from the left table and the matching records from the right table.

Syntax:

```
SELECT columns
FROM table1
LEFT JOIN table2
ON table1.column = table2.column;
```

Query:

```
SELECT vehicle.name, owner.owner_name
FROM vehicle
LEFT JOIN owner
ON vehicle.owner_id = owner.owner_id;
```

RIGHT JOIN

RIGHT JOIN returns all records from the right table and the matching records from the left table.

Syntax:

```
SELECT columns
FROM table1
RIGHT JOIN table2
ON table1.column = table2.column;
```

Query:

```
SELECT vehicle.name, owner.owner_name
FROM vehicle
RIGHT JOIN owner
ON vehicle.owner_id = owner.owner_id;
```

FULL JOIN

FULL JOIN returns all records from both tables. If there is no matching record, NULL values are displayed.

Syntax:

```
SELECT columns
FROM table1
FULL OUTER JOIN table2
ON table1.column = table2.column;
```

Query:

```
SELECT vehicle.name, owner.owner_name
FROM vehicle
FULL OUTER JOIN owner
ON vehicle.owner_id = owner.owner_id;
```

SELF JOIN

SELF JOIN is a join in which a table is joined with itself.

Syntax:

```
SELECT a.column, b.column
FROM table_name a, table_name b
WHERE condition;
```

Query:

```
SELECT a.name AS vehicle1, b.name AS vehicle2
FROM vehicle a
JOIN vehicle b
ON a.brand = b.brand
WHERE a.id < b.id;
```

Explanation

- Create Vehicle and Owner tables.
- Insert records into both tables.
- Use INNER JOIN to display matching records.
- Use LEFT JOIN to display all vehicles and matching owners.
- Use RIGHT JOIN to display all owners and matching vehicles.
- Use FULL JOIN to display all records from both tables.
- Use SELF JOIN to compare vehicles from the same brand.

Create Vehicle Table

```
CREATE TABLE vehicle (
    id INTEGER PRIMARY KEY,
    name TEXT,
    brand TEXT,
    owner_id INTEGER
);
```

Create Owner Table

```
CREATE TABLE owner (
    owner_id INTEGER PRIMARY KEY,
    owner_name TEXT
);
```

Sample Data

```
INSERT INTO owner VALUES (101, 'Arun');
INSERT INTO owner VALUES (102, 'Priya');
INSERT INTO owner VALUES (103, 'Rahul');
INSERT INTO owner VALUES (104, 'Divya');
INSERT INTO owner VALUES (105, 'Karthik');INSERT INTO vehicle VALUES
(1, 'Swift', 'Maruti', 101);
INSERT INTO vehicle VALUES (2, 'Creta', 'Hyundai', 102);
INSERT INTO vehicle VALUES (3, 'Nexon', 'Tata', 103);
INSERT INTO vehicle VALUES (4, 'City', 'Honda', 104);
INSERT INTO vehicle VALUES (5, 'Baleno', 'Maruti', 105);
INSERT INTO vehicle VALUES (6, 'i20', 'Hyundai', 102);
```

SQL Query

Query 1: INNER JOIN

```
SELECT vehicle.name, owner.owner_name
FROM vehicle
INNER JOIN owner
ON vehicle.owner_id = owner.owner_id;
```

Resultant Query

name	owner_name
Swift	Arun
Creta	Priya
Nexon	Rahul
City	Divya
Baleno	Karthik
i20	Priya

Query 2: LEFT JOIN

```
SELECT vehicle.name, owner.owner_name
FROM vehicle
LEFT JOIN owner
ON vehicle.owner_id = owner.owner_id;
```

Resultant Query

name	owner_name
Swift	Arun
Creta	Priya
Nexon	Rahul
City	Divya
Baleno	Karthik
i20	Priya

Query 3: RIGHT JOIN

```
SELECT vehicle.name, owner.owner_name
FROM vehicle
RIGHT JOIN owner
ON vehicle.owner_id = owner.owner_id;
```

Resultant Query

name	owner_name
Swift	Arun
Creta	Priya
Nexon	Rahul
City	Divya
Baleno	Karthik
i20	Priya

Query 4: FULL JOIN

```
SELECT vehicle.name, owner.owner_name
FROM vehicle
FULL OUTER JOIN owner
ON vehicle.owner_id = owner.owner_id;
```

Resultant Query

name	owner_name
Swift	Arun
Creta	Priya
Nexon	Rahul
City	Divya
Baleno	Karthik
i20	Priya

Query 5: SELF JOIN

```
SELECT a.name AS vehicle1, b.name AS vehicle2
FROM vehicle a
JOIN vehicle b
ON a.brand = b.brand
WHERE a.id < b.id;
```

Resultant Query

vehicle1	vehicle2
Swift	Baleno
Creta	i20

RESULT

Thus, the different types of SQL Joins **INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN and SELF JOIN** were performed successfully using the Vehicle and Owner tables

EX. NO: 06 IMPLEMENTATION OF SQL FUNCTIONS USING VEHICLE TABLE

DATE:27/072026

Aim

To perform SQL functions such as **Aggregate Functions, String Functions and Date Functions** on a Vehicle table.

Software Used

Oracle / MySQL / SQLite3

Description

SQL Functions are built-in functions used to perform calculations, manipulate strings and work with date values in a database.

SQL FUNCTIONS

- Aggregate Functions
- String Functions
- Date Functions

AGGREGATE FUNCTIONS

Aggregate functions perform calculations on a group of rows and return a single result.

Common aggregate functions are:

- COUNT()
- SUM()

- AVG()
- MAX()
- MIN()

Query 1: COUNT()

COUNT() is used to count the number of records.

```
SELECT COUNT(*) AS total_vehicles FROM vehicle;
```

Query 2: MAX()

MAX() is used to find the maximum value.

```
SELECT MAX(price) AS maximum_price FROM vehicle;
```

Query 3: MIN()

MIN() is used to find the minimum value.

```
SELECT MIN(price) AS minimum_price FROM vehicle;
```

Query 4: AVG()

AVG() is used to find the average value.

```
SELECT AVG(price) AS average_price FROM vehicle;
```

Query 5: SUM()

SUM() is used to calculate the total value.

```
SELECT SUM(price) AS total_price FROM vehicle;
```

STRING FUNCTIONS

String functions are used to manipulate text values.

Common string functions are:

- UPPER()
- LOWER()
- LENGTH()

- CONCAT()

Query 1: UPPER()

UPPER() converts text into uppercase letters.

```
SELECT UPPER(name) AS vehicle_name FROM vehicle;
```

Query 2: LOWER()

LOWER() converts text into lowercase letters.

```
SELECT LOWER(brand) AS brand_name FROM vehicle;
```

Query 3: LENGTH()

LENGTH() returns the number of characters in a string.

```
SELECT name, LENGTH(name) AS name_length FROM vehicle;
```

Query 4: CONCAT()

CONCAT() combines two strings.

```
SELECT CONCAT(name, ' - ', brand) AS vehicle_details FROM vehicle;
```

DATE FUNCTIONS

Date functions are used to work with date and time values.

Common date functions are:

- CURRENT_DATE
- CURRENT_TIMESTAMP
- EXTRACT()

Query 1: CURRENT_DATE

CURRENT_DATE returns the current date.

```
SELECT CURRENT_DATE AS today FROM vehicle;
```

Query 2: CURRENT_TIMESTAMP

CURRENT_TIMESTAMP returns the current date and time.

```
SELECT CURRENT_TIMESTAMP AS current_time FROM vehicle;
```

Query 3: EXTRACT()

EXTRACT() is used to extract a specific part of a date.

```
SELECT EXTRACT(YEAR FROM purchase_date) AS purchase_year FROM vehicle;
```

Explanation

- Create a Vehicle table with vehicle details.
- Insert vehicle records into the table.
- Use aggregate functions to calculate count, sum, average, maximum and minimum price.
- Use string functions to manipulate vehicle names and brands.
- Use date functions to display and extract date values.

Create Table

```
CREATE TABLE vehicle (
    id INTEGER PRIMARY KEY,
    name TEXT,
    brand TEXT,
    price INTEGER,
    purchase_date DATE
);
```

Table Created

Sample Data

```
INSERT INTO vehicle VALUES (1, 'Swift', 'Maruti', 650000, '2022-05-
10');
INSERT INTO vehicle VALUES (2, 'Creta', 'Hyundai', 1200000, '2023-06-
15');
INSERT INTO vehicle VALUES (3, 'Nexon', 'Tata', 900000, '2021-08-20');
INSERT INTO vehicle VALUES (4, 'City', 'Honda', 1100000, '2024-01-
12');
INSERT INTO vehicle VALUES (5, 'Thar', 'Mahindra', 1500000, '2023-11-
25');
```

SQL Query

Query 1: Find the total number of vehicles

```
SELECT COUNT(*) AS total_vehicles FROM vehicle;
```

Resultant Query

total_vehicles
5

Query 2: Find the maximum vehicle price

```
SELECT MAX(price) AS maximum_price FROM vehicle;
```

Resultant Query

maximum_price
1500000

Query 3: Find the minimum vehicle price

```
SELECT MIN(price) AS minimum_price FROM vehicle;
```

Resultant Query

minimum_price
650000

Query 4: Find the average vehicle price

```
SELECT AVG(price) AS average_price FROM vehicle;
```

Resultant Query

average_price
1070000

Query 5: Convert vehicle names to uppercase

```
SELECT UPPER(name) AS vehicle_name FROM vehicle;
```

Resultant Query

vehicle_name
SWIFT
CRETA
NEXON
CITY
THAR

Query 6: Convert brand names to lowercase

```
SELECT LOWER(brand) AS brand_name FROM vehicle;
```

Resultant Query

brand_name
maruti
hyundai
tata
honda
mahindra

Query 7: Find the length of vehicle names

```
SELECT name, LENGTH(name) AS name_length FROM vehicle;
```

Resultant Query

name	name_length
Swift	5
Creta	5
Nexon	5
City	4
Thar	4

Query 8: Display the current date

```
SELECT CURRENT_DATE AS today;
```

Resultant Query

today

Current Date

Query 9: Extract the year from purchase date

```
SELECT name, EXTRACT(YEAR FROM purchase_date) AS purchase_year FROM
vehicle;
```

Resultant Query

name	purchase_year
Swift	2022
Creta	2023
Nexon	2021
City	2024
Thar	2023

RESULT

Thus, the **Aggregate Functions, String Functions and Date Functions** were performed successfully on the Vehicle table.

EX. NO: 07 SUBQUERIES AND NESTED QUERIES USING VEHICLE TABLE

DATE:31/07/2026

Aim

To perform **Subqueries and Nested Queries** using SQL on a Vehicle table.

Software Used

Oracle / MySQL / SQLite3

Description

A **Subquery** is a query written inside another SQL query. The inner query is executed first and its result is used by the outer query.

A **Nested Query** is another name for a query containing a subquery.

TYPES OF SUBQUERIES

- Single Row Subquery
- Multiple Row Subquery
- Nested Subquery

SINGLE ROW SUBQUERY

A single row subquery returns only one value or one row.

Syntax:

```
SELECT column_name
FROM table_name
WHERE column_name > (SELECT column_name FROM table_name WHERE
condition);
```

Query:

```
SELECT name, price
FROM vehicle
WHERE price > (SELECT AVG(price) FROM vehicle);
```

Result:

Vehicles having a price greater than the average vehicle price are displayed.

MULTIPLE ROW SUBQUERY

A multiple row subquery returns more than one value. Operators such as **IN, ANY and ALL** can be used.

Syntax:

```
SELECT column_name
FROM table_name
WHERE column_name IN (SELECT column_name FROM table_name WHERE
condition);
```

Query:

```
SELECT name, brand
FROM vehicle
WHERE brand IN (SELECT brand FROM vehicle WHERE price > 1000000);
```

Result:

Vehicles belonging to brands whose vehicle price is greater than 1000000 are displayed.

NESTED QUERY

A nested query contains a query inside another query.

Query:

```
SELECT name, price
FROM vehicle
WHERE price = (SELECT MAX(price) FROM vehicle);
```

Result:

The vehicle with the highest price is displayed.

Explanation

- Create a Vehicle table.
- Insert vehicle records into the table.
- Use a single row subquery to find vehicles above the average price.
- Use a multiple row subquery to find vehicles based on matching brands.
- Use a nested query to find the vehicle with the highest price.

Create Table

```
CREATE TABLE vehicle (
    id INTEGER PRIMARY KEY,
    name TEXT,
    brand TEXT,
    price INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO vehicle VALUES (1, 'Swift', 'Maruti', 650000);
INSERT INTO vehicle VALUES (2, 'Creta', 'Hyundai', 1200000);
INSERT INTO vehicle VALUES (3, 'Nexon', 'Tata', 900000);
INSERT INTO vehicle VALUES (4, 'City', 'Honda', 1100000);
INSERT INTO vehicle VALUES (5, 'Thar', 'Mahindra', 1500000);
INSERT INTO vehicle VALUES (6, 'Baleno', 'Maruti', 750000);
INSERT INTO vehicle VALUES (7, 'Venue', 'Hyundai', 1000000);
INSERT INTO vehicle VALUES (8, 'Harrier', 'Tata', 1400000);
```

SQL Query

Query 1: Display vehicles having price greater than the average price

```
SELECT name, price
FROM vehicle
WHERE price > (SELECT AVG(price) FROM vehicle);
```

Resultant Query

Average price = **1068750**

name	price
Creta	1200000
City	1100000
Thar	1500000
Harrier	1400000

Query 2: Display vehicles belonging to brands having a vehicle above 1000000

```
SELECT name, brand
FROM vehicle
WHERE brand IN (SELECT brand FROM vehicle WHERE price > 1000000);
```

Resultant Query

name	brand
Creta	Hyundai
City	Honda
Thar	Mahindra
Venue	Hyundai
Harrier	Tata

Query 3: Display the vehicle having the highest price

```
SELECT name, price
FROM vehicle
WHERE price = (SELECT MAX(price) FROM vehicle);
```

Resultant Query

name	price
Thar	1500000

Query 4: Display vehicles having price less than the average price

```
SELECT name, price
FROM vehicle
WHERE price < (SELECT AVG(price) FROM vehicle);
```

Resultant Query

name	price
Swift	650000
Nexon	900000
Baleno	750000
Venue	1000000

RESULT

Thus, the **Subqueries and Nested Queries** were performed successfully using the Vehicle table.

EX. NO: 08 VIEWS, INDEXES AND SEQUENCES USING VEHICLE TABLE

DATE: 03/08/2026

Aim

To create and use **Views, Indexes and Sequences** on a Vehicle table.

Software Used

Oracle / MySQL / SQLite3

Description

View is a virtual table created from the result of a SQL query.

Index is used to improve the speed of data retrieval from a table.

Sequence is a database object used to generate unique numeric values automatically.

VIEWS

A View is a virtual table based on the result of a SELECT query.

Syntax:

```
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition;
```

Query:

```
CREATE VIEW vehicle_view AS
SELECT id, name, brand, price
FROM vehicle
WHERE price >= 1000000;
```

Display the View:

```
SELECT * FROM vehicle_view;
```

INDEXES

An Index is used to speed up the retrieval of records from a table.

Syntax:

```
CREATE INDEX index_name
ON table_name(column_name);
```

Query:

```
CREATE INDEX vehicle_brand_index
ON vehicle(brand);
```

Display data using the indexed column:

```
SELECT * FROM vehicle
WHERE brand = 'Hyundai';
```

SEQUENCES

A Sequence is used to generate unique numbers automatically.

Syntax:

```
CREATE SEQUENCE sequence_name
START WITH value
INCREMENT BY value;
```

Query:

```
CREATE SEQUENCE vehicle_seq
START WITH 9
INCREMENT BY 1;
```

Using Sequence:

```
INSERT INTO vehicle VALUES
(vehicle_seq.NEXTVAL, 'Verna', 'Hyundai', 1300000);
```

Display Sequence Value:

```
SELECT vehicle_seq.NEXTVAL FROM dual;
```

Explanation

- Create a Vehicle table.
- Insert vehicle records into the table.
- Create a View to display vehicles with price greater than or equal to 1000000.
- Create an Index on the brand column for faster searching.
- Create a Sequence to generate unique vehicle IDs.
- Insert a new vehicle using the Sequence.

Create Table

```
CREATE TABLE vehicle (
    id INTEGER PRIMARY KEY,
    name VARCHAR(50),
    brand VARCHAR(50),
    price INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO vehicle VALUES (1, 'Swift', 'Maruti', 650000);
INSERT INTO vehicle VALUES (2, 'Creta', 'Hyundai', 1200000);
INSERT INTO vehicle VALUES (3, 'Nexon', 'Tata', 900000);
INSERT INTO vehicle VALUES (4, 'City', 'Honda', 1100000);
INSERT INTO vehicle VALUES (5, 'Thar', 'Mahindra', 1500000);
INSERT INTO vehicle VALUES (6, 'Baleno', 'Maruti', 750000);
INSERT INTO vehicle VALUES (7, 'Venue', 'Hyundai', 1000000);
INSERT INTO vehicle VALUES (8, 'Harrier', 'Tata', 1400000);
```

SQL Query

Query 1: Create a View for vehicles with price 1000000 or above

```
CREATE VIEW vehicle_view AS
SELECT id, name, brand, price
FROM vehicle
WHERE price >= 1000000;
```

Resultant Query

```
SELECT * FROM vehicle_view;
```

id	name	brand	price
2	Creta	Hyundai	1200000
4	City	Honda	1100000
5	Thar	Mahindra	1500000
7	Venue	Hyundai	1000000
8	Harrier	Tata	1400000

Query 2: Create an Index on brand

```
CREATE INDEX vehicle_brand_index
ON vehicle(brand);
```

Result:

Index created successfully.

Query 3: Retrieve vehicles using the indexed column

```
SELECT * FROM vehicle
WHERE brand = 'Hyundai';
```

Resultant Query

id	name	brand	price
2	Creta	Hyundai	1200000
7	Venue	Hyundai	1000000

Query 4: Create a Sequence

```
CREATE SEQUENCE vehicle_seq
START WITH 9
INCREMENT BY 1;
```

Result:

Sequence created successfully.

Query 5: Insert a new vehicle using Sequence

```
INSERT INTO vehicle VALUES
(vehicle_seq.NEXTVAL, 'Verna', 'Hyundai', 1300000);
```

Result:

1 row inserted successfully.

Query 6: Display the newly inserted vehicle

```
SELECT * FROM vehicle
WHERE id = 9;
```

Resultant Query

id	name	brand	price
9	Verna	Hyundai	1300000

RESULT

Thus, the **View, Index and Sequence** operations were created and executed successfully on the Vehicle table.

EX.NO: 09 Stored Procedures And Functions

DATE: 07/08/2026

Aim

To implement and execute **Stored Procedures and Functions** using SQL with a Vehicle table.

Software Used

- Oracle / MySQL / SQL-supported DBMS

Description

Stored Procedure:

A stored procedure is a set of SQL statements stored in the database and executed whenever required. It can perform operations such as INSERT, UPDATE, DELETE, and SELECT.

Function:

A function is a stored program that performs a specific operation and **returns a value**.

1. STORED PROCEDURE

SYNTAX

```
CREATE OR REPLACE PROCEDURE procedure_name
AS
BEGIN
    SQL statements;
END;
/
```

CREATE TABLE

```
CREATE TABLE vehicle (
    id INTEGER PRIMARY KEY,
    name VARCHAR(50),
    brand VARCHAR(50),
    price INTEGER
);
```

INSERT SAMPLE DATA

```
INSERT INTO vehicle VALUES (1, 'Swift', 'Maruti', 650000);
INSERT INTO vehicle VALUES (2, 'Creta', 'Hyundai', 1200000);
INSERT INTO vehicle VALUES (3, 'Nexon', 'Tata', 900000);
INSERT INTO vehicle VALUES (4, 'City', 'Honda', 1100000);
INSERT INTO vehicle VALUES (5, 'Thar', 'Mahindra', 1500000);
```

PROCEDURE TO DISPLAY VEHICLES

```
CREATE OR REPLACE PROCEDURE display_vehicles
AS
BEGIN
    SELECT * FROM vehicle;
END;
/
```

EXECUTE PROCEDURE

```
EXEC display_vehicles;
```

OUTPUT

ID	NAME	BRAND	PRICE
1	Swift	Maruti	650000
2	Creta	Hyundai	1200000
3	Nexon	Tata	900000
4	City	Honda	1100000
5	Thar	Mahindra	1500000

2. PROCEDURE WITH PARAMETER

SYNTAX

```
CREATE OR REPLACE PROCEDURE procedure_name(parameter datatype)
AS
BEGIN
    SQL statements;
END;
/
```

CREATE PROCEDURE

```
CREATE OR REPLACE PROCEDURE update_price(
    p_id IN INTEGER,
    p_price IN INTEGER
)
AS
BEGIN
    UPDATE vehicle
    SET price = p_price
    WHERE id = p_id;
END;
/
```

EXECUTE PROCEDURE

```
EXEC update_price(1, 700000);
```

DISPLAY UPDATED RECORD

```
SELECT * FROM vehicle WHERE id = 1;
```

OUTPUT

ID	NAME	BRAND	PRICE
1	Swift	Maruti	700000

3. STORED FUNCTION

SYNTAX

```
CREATE OR REPLACE FUNCTION function_name
RETURN datatype
AS
BEGIN
    RETURN value;
END;
/
```

FUNCTION TO RETURN VEHICLE COUNT

```
CREATE OR REPLACE FUNCTION vehicle_count
RETURN INTEGER
AS
    total INTEGER;
BEGIN
    SELECT COUNT(*) INTO total
    FROM vehicle;
    RETURN total;
END;
/
```

EXECUTE FUNCTION

```
SELECT vehicle_count() AS total_vehicles FROM dual;
```

OUTPUT

TOTAL_VEHICLES
5

4. FUNCTION WITH PARAMETER

CREATE FUNCTION

```
CREATE OR REPLACE FUNCTION get_price(
    p_id IN INTEGER
)
RETURN INTEGER
AS
    v_price INTEGER;
BEGIN
    SELECT price INTO v_price
    FROM vehicle
    WHERE id = p_id;
    RETURN v_price;
END;
/
```

EXECUTE FUNCTION

```
SELECT get_price(2) AS vehicle_price FROM dual;
```

OUTPUT

VEHICLE_PRICE
1200000

RESULT

Thus, the **Stored Procedures and Functions** were successfully created and executed using the Vehicle table.

EX. NO: 10 TRIGERS USING VEHICLE TABLE

DATE:13/08/2026

Aim

To implement and execute **Triggers** using SQL with a Vehicle table.

Software Used

- Oracle / MySQL / SQL-supported DBMS

Description

A **Trigger** is a stored program that automatically executes when a specific event such as **INSERT, UPDATE, or DELETE** occurs on a table.

1. BEFORE INSERT TRIGGER

SYNTAX

```
CREATE OR REPLACE TRIGGER trigger_name
BEFORE INSERT ON table_name
FOR EACH ROW
BEGIN
    SQL statements;
END;
/
```

CREATE TABLE

```
CREATE TABLE vehicle (
    id INTEGER PRIMARY KEY,
    name VARCHAR(50),
    brand VARCHAR(50),
    price INTEGER
);
```

INSERT SAMPLE DATA

```
INSERT INTO vehicle VALUES (1, 'Swift', 'Maruti', 650000);
INSERT INTO vehicle VALUES (2, 'Creta', 'Hyundai', 1200000);
INSERT INTO vehicle VALUES (3, 'Nexon', 'Tata', 900000);
INSERT INTO vehicle VALUES (4, 'City', 'Honda', 1100000);
INSERT INTO vehicle VALUES (5, 'Thar', 'Mahindra', 1500000);
```

CREATE TRIGGER

```
CREATE OR REPLACE TRIGGER vehicle_insert_trigger
BEFORE INSERT ON vehicle
FOR EACH ROW
BEGIN
    IF :NEW.price < 0 THEN
        :NEW.price := 0;
    END IF;
END;
/
```

INSERT RECORD

```
INSERT INTO vehicle VALUES (6, 'Baleno', 'Maruti', -500000);
```

DISPLAY RECORD

```
SELECT * FROM vehicle WHERE id = 6;
```

OUTPUT

ID	NAME	BRAND	PRICE
6	Baleno	Maruti	0

The trigger automatically changes a negative price to `0`.

2. BEFORE UPDATE TRIGGER

CREATE TRIGGER

```
CREATE OR REPLACE TRIGGER vehicle_update_trigger
BEFORE UPDATE ON vehicle
FOR EACH ROW
BEGIN
    IF :NEW.price < 0 THEN
        :NEW.price := 0;
    END IF;
END;
/
```

UPDATE RECORD

```
UPDATE vehicle
SET price = -100000
WHERE id = 1;
```

DISPLAY RECORD

```
SELECT * FROM vehicle WHERE id = 1;
```

OUTPUT

ID	NAME	BRAND	PRICE
1	Swift	Maruti	0

3. AFTER DELETE TRIGGER

CREATE AUDIT TABLE

```
CREATE TABLE vehicle_delete_log (
    vehicle_id INTEGER,
    vehicle_name VARCHAR(50)
);
```

CREATE TRIGGER

```
CREATE OR REPLACE TRIGGER vehicle_delete_trigger
AFTER DELETE ON vehicle
FOR EACH ROW
BEGIN
    INSERT INTO vehicle_delete_log
    VALUES (:OLD.id, :OLD.name);
END;
/
```

DELETE RECORD

```
DELETE FROM vehicle
WHERE id = 5;
```

DISPLAY DELETE LOG

```
SELECT * FROM vehicle_delete_log;
```

OUTPUT

VEHICLE_ID	VEHICLE_NAME
5	Thar

The trigger automatically stores the deleted vehicle details in the log table.

4. DISPLAY VEHICLE TABLE

```
SELECT * FROM vehicle;
```

OUTPUT

ID	NAME	BRAND	PRICE
2	Creta	Hyundai	1200000
3	Nexon	Tata	900000
4	City	Honda	1100000
6	Baleno	Maruti	0

RESULT

Thus, the **Triggers** were successfully created and executed for **INSERT, UPDATE, and DELETE** operations using the Vehicle table.

EX.NO: 11 CURSOR IMPLEMENTATION

DATE: 17/08/2026

Aim

To implement and execute **Cursors** using SQL with a Vehicle table.

Software Used

- Oracle SQL / SQL Developer

Description

A **Cursor** is a pointer used to process the rows returned by a SQL query one by one. It is useful when we need to perform operations on each record individually.

There are two types of cursors:

1. **Implicit Cursor**
2. **Explicit Cursor**

1. IMPLICIT CURSOR

An implicit cursor is automatically created by the database whenever an SQL statement such as INSERT, UPDATE, DELETE, or SELECT INTO is executed.

CREATE TABLE

```
CREATE TABLE vehicle (
    id INTEGER PRIMARY KEY,
    name VARCHAR(50),
    brand VARCHAR(50),
    price INTEGER
);
```

INSERT SAMPLE DATA

```
INSERT INTO vehicle VALUES (1, 'Swift', 'Maruti', 650000);
INSERT INTO vehicle VALUES (2, 'Creta', 'Hyundai', 1200000);
INSERT INTO vehicle VALUES (3, 'Nexon', 'Tata', 900000);
INSERT INTO vehicle VALUES (4, 'City', 'Honda', 1100000);
INSERT INTO vehicle VALUES (5, 'Thar', 'Mahindra', 1500000);
```

IMPLEMENTATION

```
BEGIN
    UPDATE vehicle
    SET price = price + 50000
    WHERE brand = 'Maruti';

    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' row updated');
END;
/
```

OUTPUT

```
1 row updated
```

DISPLAY UPDATED TABLE

```
SELECT * FROM vehicle;
```

OUTPUT

ID	NAME	BRAND	PRICE
1	Swift	Maruti	700000
2	Creta	Hyundai	1200000
3	Nexon	Tata	900000
4	City	Honda	1100000
5	Thar	Mahindra	1500000

2. EXPLICIT CURSOR

An explicit cursor is created by the programmer to process multiple rows returned by a query.

SYNTAX

```
DECLARE
    CURSOR cursor_name IS
        SELECT statement;
BEGIN
    OPEN cursor_name;
    FETCH cursor_name INTO variables;
    CLOSE cursor_name;
END;
/
```

IMPLEMENTATION

```
DECLARE
    CURSOR vehicle_cursor IS
        SELECT id, name, price FROM vehicle;

    v_id vehicle.id%TYPE;
    v_name vehicle.name%TYPE;
    v_price vehicle.price%TYPE;
BEGIN
    OPEN vehicle_cursor;

    LOOP
        FETCH vehicle_cursor INTO v_id, v_name, v_price;
        EXIT WHEN vehicle_cursor%NOTFOUND;
```

```
        DBMS_OUTPUT.PUT_LINE(
            v_id || ' ' || v_name || ' ' || v_price
        );
    END LOOP;

    CLOSE vehicle_cursor;
END;
/
```

OUTPUT

```
1 Swift 700000
2 Creta 1200000
3 Nexon 900000
4 City 1100000
5 Thar 1500000
```

3. CURSOR WITH CONDITION

The cursor can also be used to display only vehicles whose price is greater than a specified amount.

IMPLEMENTATION

```
DECLARE
    CURSOR vehicle_cursor IS
        SELECT name, price
        FROM vehicle
        WHERE price > 1000000;

    v_name vehicle.name%TYPE;
    v_price vehicle.price%TYPE;
BEGIN
    OPEN vehicle_cursor;

    LOOP
        FETCH vehicle_cursor INTO v_name, v_price;
        EXIT WHEN vehicle_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            v_name || ' - ' || v_price
```

```
        );
    END LOOP;

    CLOSE vehicle_cursor;
END;
/
```

OUTPUT

```
Creta - 1200000
City - 1100000
Thar - 1500000
```

RESULT

Thus, the **Cursor Implementation** was successfully performed using **Implicit and Explicit Cursors** on the Vehicle table.

EX.NO: 12 PL/SQL EXCEPTION HANDLING

DATE:21/08/2026

Aim

To implement and execute **Exception Handling** using PL/SQL with a Vehicle table.

Software Used

Oracle SQL / SQL Developer

Description

An **Exception** is an error that occurs during the execution of a PL/SQL program. Exception handling is used to handle these errors and prevent the program from terminating abnormally.

There are three types of exceptions:

1. Predefined Exception
2. User-Defined Exception

3. Others Exception

1. PREDEFINED EXCEPTION

A predefined exception is automatically raised by Oracle when a predefined error occurs.

CREATE TABLE

```
CREATE TABLE vehicle (
    id INTEGER PRIMARY KEY,
    name VARCHAR(50),
    brand VARCHAR(50),
    price INTEGER
);
```

INSERT SAMPLE DATA

```
INSERT INTO vehicle VALUES (1, 'Swift', 'Maruti', 650000);
INSERT INTO vehicle VALUES (2, 'Creta', 'Hyundai', 1200000);
INSERT INTO vehicle VALUES (3, 'Nexon', 'Tata', 900000);
INSERT INTO vehicle VALUES (4, 'City', 'Honda', 1100000);
INSERT INTO vehicle VALUES (5, 'Thar', 'Mahindra', 1500000);
```

IMPLEMENTATION

```
DECLARE
    v_name vehicle.name%TYPE;
BEGIN
    SELECT name INTO v_name
    FROM vehicle
    WHERE id = 10;

    DBMS_OUTPUT.PUT_LINE('Vehicle Name: ' || v_name);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No vehicle found for the given ID');
END;
/
```

OUTPUT

```
No vehicle found for the given ID
```

2. ZERO_DIVIDE EXCEPTION

The ZERO_DIVIDE exception occurs when a number is divided by zero.

IMPLEMENTATION

```
DECLARE
    v_price INTEGER := 100000;
    v_value INTEGER;
BEGIN
    v_value := v_price / 0;

    DBMS_OUTPUT.PUT_LINE('Result: ' || v_value);

EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('Error: Cannot divide by zero');
END;
/
```

OUTPUT

```
Error: Cannot divide by zero
```

3. TOO_MANY_ROWS EXCEPTION

The TOO_MANY_ROWS exception occurs when a SELECT INTO statement returns more than one row.

IMPLEMENTATION

```
DECLARE
    v_name vehicle.name%TYPE;
BEGIN
    SELECT name INTO v_name
    FROM vehicle
    WHERE brand = 'Maruti';
```

```
    DBMS_OUTPUT.PUT_LINE('Vehicle Name: ' || v_name);

EXCEPTION
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Error: More than one vehicle found');
END;
/
```

OUTPUT

```
Error: More than one vehicle found
```

4. USER-DEFINED EXCEPTION

A user-defined exception is created by the programmer to handle a specific condition.

IMPLEMENTATION

```
DECLARE
    v_price vehicle.price%TYPE;
    low_price EXCEPTION;
BEGIN
    SELECT price INTO v_price
    FROM vehicle
    WHERE id = 1;

    IF v_price < 700000 THEN
        RAISE low_price;
    END IF;

    DBMS_OUTPUT.PUT_LINE('Vehicle price is acceptable');

EXCEPTION
    WHEN low_price THEN
        DBMS_OUTPUT.PUT_LINE('Error: Vehicle price is below the minimum
limit');
END;
/
```

OUTPUT

```
Error: Vehicle price is below the minimum limit
```

5. OTHERS EXCEPTION

The OTHERS exception is used to handle any exception that is not specifically handled by the previous exception handlers.

IMPLEMENTATION

```
DECLARE
    v_name vehicle.name%TYPE;
BEGIN
    SELECT name INTO v_name
    FROM vehicle
    WHERE id = 20;

    DBMS_OUTPUT.PUT_LINE('Vehicle Name: ' || v_name);

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An error occurred');
END;
/
```

OUTPUT

```
An error occurred
```

RESULT

Thus, the **PL/SQL Exception Handling** was successfully implemented using **Predefined, User-Defined, and Others Exceptions** on the Vehicle table.

EX.NO: 13 DATABASE NORMALIZATION (1NF, 2NF, 3NF, BCNF)

DATE: 25/08/2026

Aim

To implement and understand **Database Normalization** using **1NF, 2NF, 3NF, and BCNF** to reduce data redundancy and improve data consistency.

Software Used

Oracle SQL / SQL Developer

Description

Normalization is the process of organizing data in a database to reduce redundancy and avoid insertion, deletion, and update anomalies.

The main normal forms are:

1. First Normal Form (1NF)
2. Second Normal Form (2NF)
3. Third Normal Form (3NF)
4. Boyce-Codd Normal Form (BCNF)

1. FIRST NORMAL FORM (1NF)

A table is in **1NF** when all attributes contain **atomic values** and there are no repeating groups.

UNNORMALIZED TABLE

STUDENT_ID	STUDENT_NAME	COURSES
1	Arun	DBMS, Java
2	Priya	Python, SQL

The COURSES column contains multiple values, so the table is not in 1NF.

1NF TABLE

STUDENT_ID	STUDENT_NAME	COURSE
1	Arun	DBMS
1	Arun	Java
2	Priya	Python
2	Priya	SQL

Now each column contains a single atomic value, so the table is in **1NF**.

CREATE TABLE

```
CREATE TABLE student_course (
    student_id INTEGER,
    student_name VARCHAR(50),
    course VARCHAR(50),
    PRIMARY KEY (student_id, course)
);
```

INSERT SAMPLE DATA

```
INSERT INTO student_course VALUES (1, 'Arun', 'DBMS');
INSERT INTO student_course VALUES (1, 'Arun', 'Java');
INSERT INTO student_course VALUES (2, 'Priya', 'Python');
INSERT INTO student_course VALUES (2, 'Priya', 'SQL');
```

DISPLAY TABLE

```
SELECT * FROM student_course;
```

OUTPUT

STUDENT_ID	STUDENT_NAME	COURSE
1	Arun	DBMS
1	Arun	Java
2	Priya	Python
2	Priya	SQL

2. SECOND NORMAL FORM (2NF)

A table is in **2NF** when:

- It is already in 1NF.
- There is no partial dependency on a part of a composite primary key.

In the above table, the primary key is `(student_id, course)`.

`student_name` depends only on `student_id`, not on the complete primary key. Therefore, it has a partial dependency.

STUDENT TABLE

```
CREATE TABLE student (
    student_id INTEGER PRIMARY KEY,
    student_name VARCHAR(50)
);
```

COURSE TABLE

```
CREATE TABLE course (
    course_id INTEGER PRIMARY KEY,
    course_name VARCHAR(50)
);
```

ENROLLMENT TABLE

```
CREATE TABLE enrollment (
    student_id INTEGER,
    course_id INTEGER,
    PRIMARY KEY (student_id, course_id)
);
```

INSERT SAMPLE DATA

```
INSERT INTO student VALUES (1, 'Arun');
INSERT INTO student VALUES (2, 'Priya');INSERT INTO course VALUES (101,
'DBMS');
INSERT INTO course VALUES (102, 'Java');
INSERT INTO course VALUES (103, 'Python');
INSERT INTO course VALUES (104, 'SQL');INSERT INTO enrollment VALUES (1,
101);
INSERT INTO enrollment VALUES (1, 102);
INSERT INTO enrollment VALUES (2, 103);
INSERT INTO enrollment VALUES (2, 104);
```

OUTPUT

STUDENT

STUDENT_ID	STUDENT_NAME
1	Arun
2	Priya

COURSE

COURSE_ID	COURSE_NAME
101	DBMS
102	Java
103	Python
104	SQL

ENROLLMENT

STUDENT_ID	COURSE_ID
1	101
1	102
2	103
2	104

The partial dependency is removed, so the database is in **2NF**.

3. THIRD NORMAL FORM (3NF)

A table is in **3NF** when:

- It is already in 2NF.
- There is no transitive dependency.

Consider the following table:

STUDENT_ID	STUDENT_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
1	Arun	10	Computer Science
2	Priya	20	Information Technology

Here:

STUDENT_ID → DEPARTMENT_ID

and

DEPARTMENT_ID → DEPARTMENT_NAME

Therefore:

STUDENT_ID → DEPARTMENT_NAME

This is a **transitive dependency**.

STUDENT TABLE

```
CREATE TABLE student_3nf (
    student_id INTEGER PRIMARY KEY,
    student_name VARCHAR(50),
    department_id INTEGER
);
```

DEPARTMENT TABLE

```
CREATE TABLE department (
    department_id INTEGER PRIMARY KEY,
    department_name VARCHAR(100)
);
```

INSERT SAMPLE DATA

```
INSERT INTO department VALUES (10, 'Computer Science');
INSERT INTO department VALUES (20, 'Information Technology');INSERT INTO
student_3nf VALUES (1, 'Arun', 10);
INSERT INTO student_3nf VALUES (2, 'Priya', 20);
```

DISPLAY TABLE

```
SELECT * FROM student_3nf;
```

OUTPUT

STUDENT_ID	STUDENT_NAME	DEPARTMENT_ID
1	Arun	10
2	Priya	20

```
SELECT * FROM department;
```

OUTPUT

DEPARTMENT_ID	DEPARTMENT_NAME
10	Computer Science
20	Information Technology

The transitive dependency is removed, so the database is in **3NF**.

4. BOYCE-CODD NORMAL FORM (BCNF)

A relation is in **BCNF** when every determinant is a **candidate key**.

Consider the following relation:

STUDENT_COURSE

STUDENT	COURSE	INSTRUCTOR
Arun	DBMS	Ravi
Priya	DBMS	Ravi
Kumar	Java	Meena

Assume the following dependencies:

```
Student, Course → Instructor

Instructor → Course
```

Here, `Instructor` determines `Course`, but `Instructor` is not a candidate key of the relation. Therefore, the relation violates **BCNF**.

DECOMPOSITION FOR BCNF

INSTRUCTOR_COURSE

```
CREATE TABLE instructor_course (
    instructor VARCHAR(50) PRIMARY KEY,
    course VARCHAR(50)
);
```

STUDENT_INSTRUCTOR

```
CREATE TABLE student_instructor (
    student VARCHAR(50),
    instructor VARCHAR(50),
    PRIMARY KEY (student, instructor)
);
```

INSERT SAMPLE DATA

```
INSERT INTO instructor_course VALUES ('Ravi', 'DBMS');
INSERT INTO instructor_course VALUES ('Meena', 'Java');INSERT INTO
student_instructor VALUES ('Arun', 'Ravi');
INSERT INTO student_instructor VALUES ('Priya', 'Ravi');
INSERT INTO student_instructor VALUES ('Kumar', 'Meena');
```

DISPLAY INSTRUCTOR COURSE

```
SELECT * FROM instructor_course;
```

OUTPUT

INSTRUCTOR	COURSE
Ravi	DBMS
Meena	Java

DISPLAY STUDENT INSTRUCTOR

```
SELECT * FROM student_instructor;
```

OUTPUT

STUDENT	INSTRUCTOR
Arun	Ravi

Priya	Ravi
Kumar	Meena

The violating dependency is removed and the relations satisfy **BCNF**.

RESULT

Thus, **Database Normalization** was successfully implemented and studied using **1NF, 2NF, 3NF, and BCNF**, reducing data redundancy and improving data consistency.

EX.NO: 14 PRACTICAL QUESTIONS : 19

DATE:03/09/2026

1. Consider the following tables CABHUB and CUSTOMER and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:
(i) To display the names of all the white colored vehicles.
(ii) To display name of vehicle, make the capacity of vehicles in ascending order of their sitting Capacity.
(iii) To display the highest charges at which a vehicle can be hired from CABHUB.
(iv) To display the customer and the corresponding name of the vehicle hired by them.
(b) (i) SELECT COUNT (DISTINCT Make) FROM CABHUB;
(ii) SELECT MAX (CHARGES), MIN (Charges) FROM CABHUB;
(iii) SELECT COUNT(*), Make FROM CABHUB;
(iv) SELECT VehicleName FROM CABHUB WHERE Capacity = 4;

CABHUB

Vcode	VehicleName	Make	Color	Capacity	Charges
100	Innova	Toyota	WHITE	7	15
102	SX4	Suzuki	BLUE	4	14
104	C-Class	Mercedes	RED	4	35
105	A-Star	Suzuki	WHITE	3	14
108	Indigo	Tata	SILVER	3	12

CUSTOMER

Code	CName	VCode
1	HemantSahu	101
2	Raj Lal	108
3	Feroza Shah	105
4	Ketan Dhal	104

Syntax:

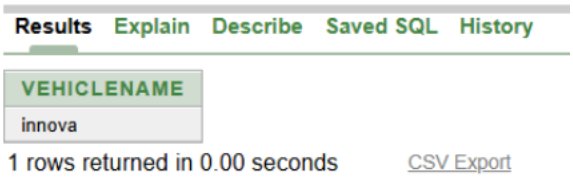
```
create table cabhub (vcode number primary key, vehiclename varchar2(20), make
varchar2(20), color varchar2(10), capacity number, charges number);
insert into cabhub values (100,'innova','toyota','white',7,15);
insert into cabhub values (102,'sx4','suzuki','blue',4,14);
insert into cabhub values (104,'c-class','mercedes','red',4,35);
insert into cabhub values (105,'a-star','suzuki','white',3,14);
insert into cabhub values (108,'indigo','tata','silver',3,12);
create table customer (code number primary key, cname varchar2(20), vcode number);
insert into customer values (1,'hemantsahu',101);
insert into customer values (2,'raj lal',108);
insert into customer values (3,'feroza shah',105);
insert into customer values (4,'ketan dhall',104);
```

Question: (i) To display the names of all the white colored vehicles.

Input:

```
select vehiclename from cabhub where color = 'white';
```

Output:



Question: (ii) To display name of vehicle, make the capacity of vehicles in ascending order of their sitting Capacity.

Input:

```
select vehiclename, capacity from cabhub order by capacity asc;
```

Output:

Results Explain Describe Saved SQL

VEHICLENAME	CAPACITY
City	4
Swift Dzire	4
i20	5
innova	7
Innova	7
Ertiga	7

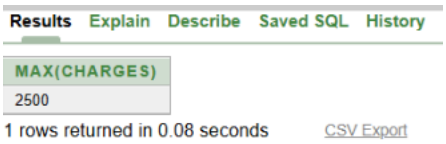
6 rows returned in 0.11 seconds CSV I

Question:(iii) To display the highest charges at which a vehicle can be hired from CABHUB.

Input:

select max(charges) from cabhub;

Output:



Question:(iv) To display the customer and the corresponding name of the vehicle hired by them.

Input:

select cname,vehiclename from customer,cabhub where customer.vcode = cabhub.vcode;

Output:

Results Explain Describe Saved SQL History

CNAME	VEHICLENAME
raj lal	indigo
feroza shah	a-star
ketan dhall	c-class

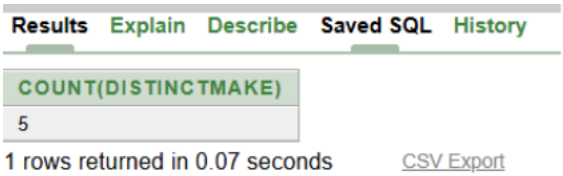
3 rows returned in 0.08 seconds CSV Export

Question:(b) (i) SELECT COUNT (DISTINCT Make) FROM CABHUB;

Input:

select count(distinct make) from cabhub;

Output:

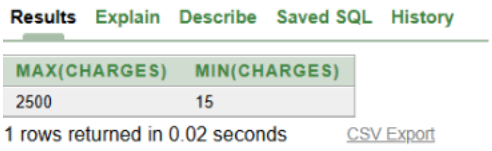


Question: (ii) SELECT MAX (CHARGES), MIN (Charges) FROM CABHUB;

Input:

select max(charges), min(charges) from cabhub;

Output:

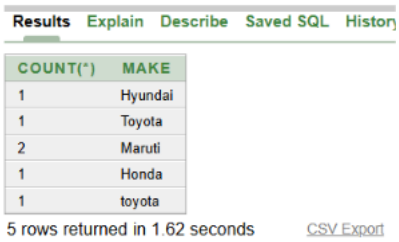


Question: (iii) SELECT COUNT(*), Make FROM CABHUB;

Input:

select count(*), make from cabhub group by make;

Output:

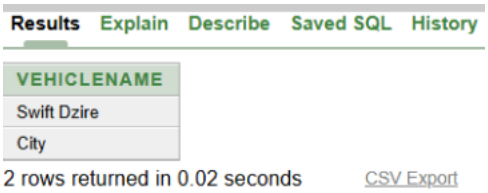


Question: (iv) SELECT VehicleName FROM CABHUB WHERE Capacity = 4;

Input:

select vehiclename from cabhub where capacity = 4;

Output:



2. Given the following CO2_EMISSION relation, write SQL commands for questions (a) to (f).

(a) Display all the details of vehicles that use **Petrol** as the fuel type.
(b) Display the **Company** and **Model** of vehicles whose **EngineSize** is greater than **1.5 litres**.
(c) Display the **Company**, **Model**, and **CO2Emission** in **ascending order of CO2Emission**.
(d) Display the details of vehicles whose **CO2Emission** is greater than **150 g/km**.0
(e) Count the number of vehicles that use **Diesel** as the fuel type.
(f) Insert the following record into the **CO2_EMISSION** table.
108, 'MG', 'Hector', 'Petrol', 1.5, 16
(g) Write the output of the following SQL commands.
(i) SELECT MAX(CO2Emission)FROM CO2_EMISSION;
(ii) SELECT MIN(EngineSize)FROM CO2_EMISSION;
(iii) SELECT AVG(CO2Emission)FROM CO2_EMISSIONWHERE FuelType='Petrol';
(iv) SELECT COUNT(*)FROM CO2_EMISSIONWHERE EngineSize>=1.5;

VehicleID	Company	Model	FuelType	EngineSize	CO2Emission
101	Toyota	Corolla	Petrol	1.8	165
102	Hyundai	Creta	Diesel	1.5	145
103	Honda	City	Petrol	1.5	152
104	Tata	Nexon	Diesel	1.5	138
105	Maruti	Swift	Petrol	1.2	125
106	Kia	Seltos	Diesel	1.5	150
107	Mahindra	XUV700	Diesel	2.2	185

Syntax:

```
create table co2_emission (vehicleid number primary key, company varchar2(20),
model varchar2(20), fueltype varchar2(10), enginesize number(3,1), co2emission
number);
insert into co2_emission values (101,'toyota','corolla','petrol',1.8,165);
insert into co2_emission values (102,'hyundai','creta','diesel',1.5,145);
insert into co2_emission values (103,'honda','city','petrol',1.5,152);
insert into co2_emission values (104,'tata','nexon','diesel',1.5,138);
insert into co2_emission values (105,'maruti','swift','petrol',1.2,125);
insert into co2_emission values (106,'kia','seltos','diesel',1.5,150);
```

insert into co2_emission values (107,'mahindra','xuv700','diesel',2.2,185);

Question: **(a)** Display all the details of vehicles that use **Petrol** as the fuel type.

Input:

```
select * from co2_emission where fueltype = 'petrol';
```

Output:

Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	toyota	corolla	petrol	1.8	165
103	honda	city	petrol	1.5	152
105	maruti	swift	petrol	1.2	125
108	mg	hector	petrol	1.5	160

4 rows returned in 0.00 seconds CSV Export

Question: **(b)** Display the **Company** and **Model** of vehicles whose **EngineSize** is greater than **1.5 litres**.

Input:

```
select company, model from co2_emission where enginesize > 1.5;
```

Output:

Results Explain Describe Saved SQL Histor

COMPANY	MODEL
toyota	corolla
mahindra	xuv700

2 rows returned in 0.07 seconds CSV Export

Question: **(c)** Display the **Company**, **Model**, and **CO2Emission** in **ascending order of CO2Emission**.

Input:

```
select company, model, co2emission from co2_emission order by co2emission asc;
```

Output:

Results Explain Describe Saved SQL Histo

COMPANY	MODEL	CO2EMISSION
maruti	swift	125
tata	nexon	138
hyundai	creta	145
kia	seltos	150
honda	city	152
mg	hector	160
toyota	corolla	165
mahindra	xuv700	185

8 rows returned in 0.00 seconds CSV Export

Question: **(d)** Display the details of vehicles whose **CO2Emission** is greater than **150 g/km**.

Input:

```
select * from co2_emission where co2emission > 150;
```

Output:

Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	toyota	corolla	petrol	1.8	165
103	honda	city	petrol	1.5	152
107	mahindra	xuv700	diesel	2.2	185
108	mg	hector	petrol	1.5	160

4 rows returned in 0.00 seconds CSV Export

Question:

(e) Count the number of vehicles that use **Diesel** as the fuel type.

Input:

```
select count(*) from co2_emission where fueltype = 'diesel';
```

Output:

Results Explain Describe Saved SQL H

COUNT(*)
4

1 rows returned in 0.02 seconds CSV Ex

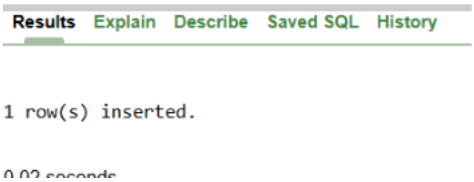
Question: **(f)** Insert the following record into the **CO2_EMISSION** table.

108, 'MG', 'Hector', 'Petrol', 1.5, 16

Input:

insert into co2_emission values (108,'mg','hector','petrol',1.5,160);

Output:

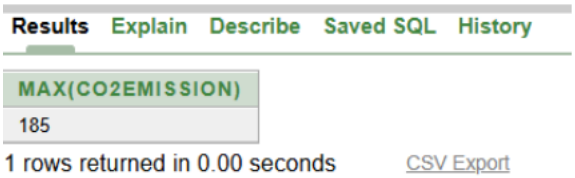


Question:**(i)** SELECT MAX(CO2Emission)FROM CO2_EMISSION;

Input:

select max(co2emission) from co2_emission;

Output:

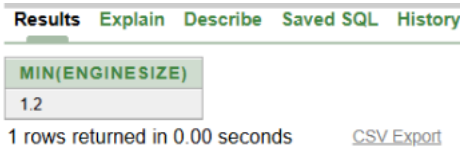


Question:**(ii)** SELECT MIN(EngineSize)FROM CO2_EMISSION;

Input:

select min(enginesize) from co2_emission;

Output:

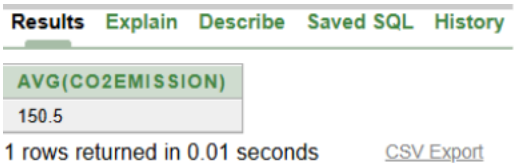


Question:**(iii)** SELECT AVG(CO2Emission)FROM CO2_EMISSIONWHERE FuelType='Petrol';

Input:

select avg(co2emission) from co2_emission where fueltype = 'petrol';

Output:

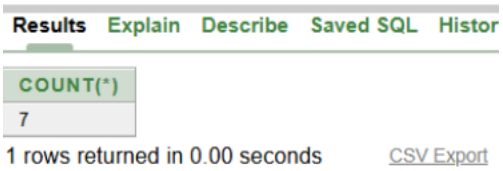


Question: **(iv)** SELECT COUNT(*)FROM CO2_EMISSIONWHERE EngineSize>=1.5;

Input:

select count(*) from co2_emission where enginesize >= 1.5;

Output:



3.Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables APPLICANTS and COURSES.

(a) To display name, fee, gender, joinyear about the applicants, who have joined before 2010.
(b) To display the names of applicants, who are paying fee more than 30000.
(c) To display name of all applicants in ascending order of their joinyear.
(d) To display the year and the total number of applicants joined in each YEAR from the table APPLICANTS.
(e) To display the C_ID (i.e. Course ID) and the number of applicants registered in the course from the APPLICANTS table.
(f) To display the applicant's name with their respective course's name from the tables APPLICANTS and COURSES.
(g) Give the output of following SQL statements:
(g1) SELECT NAME, JOIN YEAR FROM APPLICANTS WHERE GENDER= 'F' AND C_ID= '02';
(g2) SELECT MIN(JOINYEAR) FROM APPLICANTS WHERE Gender= 'M';
(g3) SELE CT AVG(FEE) FROM APPLICANTS WHERE C_ID= 'A01' OR C_ID= 'A05';
(g4) SELECT SUM (FEE), C_ID FROM APPLICATIONS GROUP BY C_ID HAVING COUNT (*)=2;

APPLICANTS

NO	NAME	FEE	GENDER	C_ID	JOINYEAR
1012	Amandeep	30000	M	A01	2012
1102	Avisha	25000	F	A02	2009
1103	Ekant	30000	M	A02	2011
1049	Arun	30000	M	A03	2009
1025	Amber	40000	M	A02	2011
1106	Ela	40000	F	A05	2010
1017	Nikita	35000	F	A03	2012
1108	Arleena	30000	F	A03	2012
2109	Shakti	35000	M	A04	2011
1101	Kirat	25000	M	A01	2012

COURSES

C_ID	COURSE
A01	Fashion Design
A02	Networking
A03	Hotel Management
A04	Event Management
A05	Office Management

Syntax:

```
create table applicants (no number primary key, name varchar2(20), fee number,
gender varchar2(5), c_id varchar2(5), joinyear number);
insert into applicants values (1012,'amandeep',30000,'m','a01',2012);
insert into applicants values (1102,'avisha',25000,'f','a02',2009);
insert into applicants values (1103,'ekant',30000,'m','a02',2011);
insert into applicants values (1049,'arun',30000,'m','a03',2009);
insert into applicants values (1025,'amber',40000,'m','a02',2011);
insert into applicants values (1106,'ela',40000,'f','a05',2010);
insert into applicants values (1017,'nikita',35000,'f','a03',2012);
insert into applicants values (1108,'arleena',30000,'f','a03',2012);
insert into applicants values (2109,'shakti',35000,'m','a04',2011);
insert into applicants values (1101,'kirat',25000,'m','a01',2012);
create table courses (c_id varchar2(5) primary key, course varchar2(30));
insert into courses values ('a01','fashion design');
insert into courses values ('a02','networking');
insert into courses values ('a03','hotel management');
insert into courses values ('a04','event management');
insert into courses values ('a05','office management');
```

Question:

(a) To display name, fee, gender, joinyear about the applicants, who have joined before 2010.

Input:

```
select name, fee, gender, joinyear from applicants where joinyear < 2010;
```

Output:

Results Explain Describe Saved SQL History

NAME	FEE	GENDER	JOINYEAR
avisha	25000	f	2009
arun	30000	m	2009

2 rows returned in 0.04 seconds CSV Export

Question: (b) To display the names of applicants, who are paying fee more than 30000.

Input:

select name from applicants where fee > 30000;

Output:

Results Explain Describe Saved SQ

NAME
amber
ela
nikita
shakti

4 rows returned in 0.00 seconds C

Question: (c) To display name of all applicants in ascending order of their joinyear.

Input:

select name from applicants order by joinyear asc;

Output:

Results Explain Describe Saved SQ

NAME
arun
avisha
ela
ekant
amber
shakti
nikita
arleena
amandeep
kirat

10 rows returned in 0.00 seconds

Question: (d) To display the year and the total number of applicants joined in each YEAR from the table APPLICANTS.

Input:

select joinyear, count(*) from applicants group by joinyear;

Output:

Results Explain Describe Saved SQL

JOINYEAR	COUNT(*)
2009	2
2010	1
2011	3
2012	4

4 rows returned in 0.01 seconds CSV

Question: (e) To display the C_ID (i.e. Course ID) and the number of applicants registered in the course from the APPLICANTS table.

Input:

select c_id, count(*) from applicants group by c_id;

Output:

Results Explain Describe Saved

C_ID	COUNT(*)
a01	2
a02	3
a05	1
a03	3
a04	1

5 rows returned in 0.00 seconds

Question: (f) To display the applicant's name with their respective course's name from the tables APPLICANTS and COURSES.

Input:

select applicants.name, courses.course from applicants, courses where applicants.c_id = courses.c_id;

Output:

Results Explain Describe Saved

NAME	COURSE
amandeep	fashion design
avisha	networking
ekant	networking
arun	hotel management
amber	networking
ela	office management
nikita	hotel management
arleena	hotel management
shakti	event management
kirat	fashion design

10 rows returned in 0.04 seconds

Question:(g) Give the output of following SQL statements:

(g1) SELECT NAME, JOIN YEAR FROM APPLICANTS WHERE GENDER= ‘F’ AND C_ID= ‘02’;

Input:

select name, joinyear from applicants where gender = 'f' and c_id = 'a02';

Output:

Results Explain Describe Saved SQL

NAME	JOINYEAR
avisha	2009

1 rows returned in 0.00 seconds

Question:(g2) SELECT MIN(JOINYEAR) FROM APPLICANTS WHERE Gender= ‘M’;

Input:

select min(joinyear) from applicants where gender = 'm';

Output:

Results Explain Describe Sa

MIN(JOINYEAR)
2009

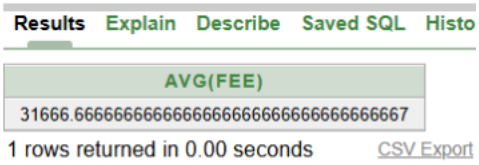
1 rows returned in 0.00 seconds

Question:(g3) SELE CT AVG(FEE) FROM APPLICANTS WHERE C_ID= ‘A01’ OR C_ID= ‘A05’;

Input:

select avg(fee) from applicants where c_id = 'a01' or c_id = 'a05';

Output:

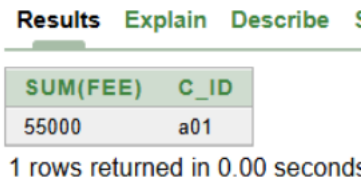


Question:(g4) SELECT SUM (FEE), C_ID FROM APPLICATIONS GROUP BY C_ID HAVING COUNT (*)=2;

Input:

select sum(fee), c_id from applicants group by c_id having count(*) = 2;

Output:



4. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables **PRODUCTS AND SUPPLIERS**

(a) To display the details of all the products in ascending order of product names (i.e. PNAME).
(b) To display product name and price of all those products, whose price is in the range of 10000 and 15000 (both values inclusive).
(c) To display the number of products which are supplied by each supplier i.e. the expected output should be S01 2 , S02 2 , S03 1
(d) To display the price, product name (i.e. PName) and quantity (i.e. QTY) of those which have quantity more than 100.
(e) To display the names of those suppliers, who are either from DELHI or from CHENNAI.
(f) To display the name of the companies and the name of the products in descending order of company names.
(g) Obtain the outputs of the following SQL queries based on the data given in tables PRODUCTS and SUPPLIERS.
(g1) SELECT DISTINCT SUPCODE FROM PRODUCTS;

(g2) SELECT MAX(PRICE), MIN (PRICE) FROM PRODUCTS;
(g3) SELECT PRICE * QTY AMOUNT FROM PRODUCTS WHERE PID = 104;
(g4) SELECT PNAME, SNAME FROM PRODUCTS P, SUPPLIERS S WHERE P. SUPCODE = S. SUPCODE AND QTY>100;

PRODUCTS

PID	SNAME	QTY	PRICE	COMPANY	SUPCODE
101	DIGITAL CAMERA14X	120	12000	RENIX	S01
102	DIGITAL PAD 1li	100	22000	DIGI POP	S02
104	PEN DRIVE 16 GB	500	1100	STOREKING	S01
106	LED SCREEN	70	28000	DISEXPERTS	S02
105	CAR GPS SYSSTEM	60	12000	MOVEON	S03

SUPPLIERS

SUPCODE	SNAME	CITY
S01	GET ALL INC	KOLKATA
S03	EASY MARKET CORP	DELHI
S02	DIGI BUSY GROUP	CHENNAI

Syntax:

```
create table products (pid number primary key, sname varchar2(30), qty number, price number, company varchar2(20), supcode varchar2(5));
insert into products values (101,'digital camera14x',120,12000,'renix','s01');
insert into products values (102,'digital pad 11i',100,22000,'digi pop','s02');
insert into products values (104,'pen drive 16 gb',500,1100,'storeking','s01');
insert into products values (106,'led screen',70,28000,'disexperts','s02');
insert into products values (105,'car gps syssystem',60,12000,'moveon','s03');
create table suppliers (supcode varchar2(5) primary key, sname varchar2(30), city varchar2(20));
insert into suppliers values ('s01','get all inc','kolkata');
insert into suppliers values ('s03','easy market corp','delhi');
insert into suppliers values ('s02','digi busy group','chennai');
```

Question: (a) To display the details of all the products in ascending order of product names (i.e. PNAME).

Input:

```
select * from products order by sname asc;
```

Output:

Results Explain Describe Saved SQL History

PID	SNAME	QTY	PRICE	COMPANY	SUPCODE
105	car gps syssystem	60	12000	moveon	s03
101	digital camera14x	120	12000	renix	s01
102	digital pad 11i	100	22000	digi pop	s02
106	led screen	70	28000	disexperts	s02
104	pen drive 16 gb	500	1100	storeking	s01

5 rows returned in 0.02 seconds CSV Export

Question:(b) To display product name and price of all those products, whose price is in the range of 10000 and 15000 (both values inclusive).

Input:

select sname, price from products where price between 10000 and 15000;

Output:

Results Explain Describe Saved SQL History

SNAME	PRICE
digital camera14x	12000
car gps syssystem	12000

2 rows returned in 0.00 seconds CSV Export

Question:(c) To display the number of products which are supplied by each supplier i.e. the expected output should be S01 2 , S02 2 , S03 1

Input:

select supcode, count(*) from products group by supcode;

Output:

Results Explain Describe Saved S

SUPCODE	COUNT(*)
s02	2
s01	2
s03	1

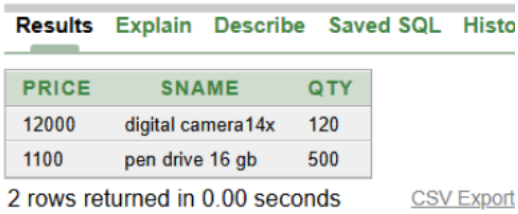
3 rows returned in 0.01 seconds

Question:(d) To display the price, product name (i.e. PName) and quantity (i.e. QTY) of those which have quantity more than 100.

Input:

select price, sname, qty from products where qty > 100;

Output:



Results Explain Describe Saved SQL Histo

PRICE	SNAME	QTY
12000	digital camera14x	120
1100	pen drive 16 gb	500

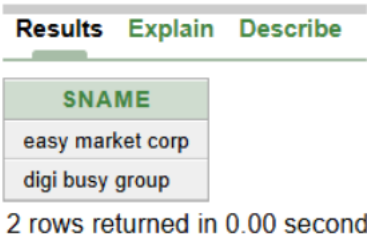
2 rows returned in 0.00 seconds CSV Export

Question: (e) To display the names of those suppliers, who are either from DELHI or from CHENNAI.

Input:

select sname from suppliers where city = 'delhi' or city = 'chennai';

Output:



Results Explain Describe

SNAME
easy market corp
digi busy group

2 rows returned in 0.00 second

Question: (f) To display the name of the companies and the name of the products in descending order of company names.

Input:

select suppliers.sname, products.sname from products, suppliers where products.supcode = suppliers.supcode order by suppliers.sname desc;

Output:

Results Explain Describe Saved S(

SNAME	SNAME
get all inc	digital camera14x
get all inc	pen drive 16 gb
easy market corp	car gps syssystem
digi busy group	led screen
digi busy group	digital pad 11i

5 rows returned in 0.00 seconds

Question: (g) Obtain the outputs of the following SQL queries based on the data given in tables PRODUCTS and SUPPLIERS.

(g1) SELECT DISTINCT SUPCODE FROM PRODUCTS;

Input:

select distinct supcode from products;

Output:

Results Explain I

SUPCODE
s02
s01
s03

3 rows returned in 0.

Question: (g2) SELECT MAX(PRICE), MIN (PRICE) FROM PRODUCTS;

Input:

select max(price), min(price) from products;

Output:

Results Explain Describe Saved S

MAX(PRICE)	MIN(PRICE)
28000	1100

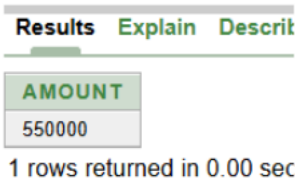
1 rows returned in 0.00 seconds

Question: (g3) SELECT PRICE * QTY AMOUNT FROM PRODUCTS WHERE PID = 104;

Input:

select price * qty as amount from products where pid = 104;

Output:

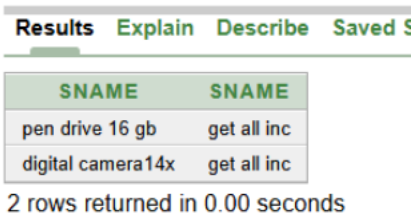


Question: (g4) SELECT PNAME, SNAME FROM PRODUCTS P, SUPPLIERS S WHERE P. SUPCODE = S. SUPCODE AND QTY>100;

Input:

select products.sname, suppliers.sname from products, suppliers where products.supcode = suppliers.supcode and qty > 100;

Output:



5. Answer the questions (a) and (b) on the basis of the following tables SHOPPE and ACCESSORIES

(a) Write the SQL queries:
(i) To display Name and Price of all the accessories in ascending order of their Price.
(ii) To display Id and SName of all Shoppe in Nehru Place.
(iii) To display Minimum and Maximum Price of each Name of accessories.
(iv) To display Name, Price of all accessories and their respective SName where they are available.
(b) (i) SELECT DISTINCT Name FROM ACCESSORIES WHERE Price>=500;
(ii) SELECT Area, COUNT (*) FROM GROUP BY Area;
(iii) SELECT COUNT (DISTINCT Area) FROM SHOPPE;

(iv) SELECT Name, Price*0.05 DISCOUNT FROM ACCESSORIES WHERE SNo IN (‘S02, ‘S03’);

SHOPPE

ID	SName	Area
S0001	ABC Computeronics	CP
S0002	All Infotech Media	GK II
S0003	Tech Shoppe	CP
S0004	Greeks Techno Soft	Nehru Place
S0005	Hitech Tech Store	Nehru Place

ACCESSORIES

No	Name	Price	ID
A01	Mother Board	12000	S01
A02	Hard Disk	5000	S01
A03	Keyboard	500	S02
A04	Mouse	300	S01
A05	Mother Board	13000	S02
A06	Keyboard	400	S03
A07	LCD	6000	S04
T08	LCD	5500	S05
T09	Mouse	350	S05
T10	Hard Disk	4500	S03

Syntax:

```
create table shoppe(id varchar2(10) primary key,sname varchar2(30),area varchar2(20));
insert into shoppe values('S0001','ABC Computeronics','CP');
insert into shoppe values('S0002','All Infotech Media','GK II');
insert into shoppe values('S0003','Tech Shoppe','CP');
insert into shoppe values('S0004','Greeks Techno Soft','Nehru Place');
insert into shoppe values('S0005','Hitech Tech Store','Nehru Place');
create table accessories(no varchar2(10) primary key,name varchar2(30),price number,id
varchar2(10));
insert into accessories values('A01','Mother Board',12000,'S01');
insert into accessories values('A02','Hard Disk',5000,'S01');
insert into accessories values('A03','Keyboard',500,'S02');
insert into accessories values('A04','Mouse',300,'S01');
insert into accessories values('A05','Mother Board',13000,'S02');
insert into accessories values('A06','Keyboard',400,'S03');
insert into accessories values('A07','LCD',6000,'S04');
insert into accessories values('T08','LCD',5500,'S05');
insert into accessories values('T09','Mouse',350,'S05');
insert into accessories values('T10','Hard Disk',4500,'S03');
```

Question:

(i) To display Name and Price of all the accessories in ascending order of their Price.

Input:

```
select name,price from accessories order by price asc;
```

Output:

Results Explain Describe Saved

NAME	PRICE
Mouse	300
Mouse	350
Keyboard	400
Keyboard	500
Hard Disk	4500
Hard Disk	5000
LCD	5500
LCD	6000
Mother Board	12000
Mother Board	13000

10 rows returned in 0.10 seconds

Question: (ii) To display Id and SName of all Shoppe in Nehru Place.

Input:

select id,sname from shoppe where area='Nehru Place';

Output:

Results Explain Describe Sav

ID	SNAME
S0004	Greeks Techno Soft
S0005	Hitech Tech Store

2 rows returned in 0.10 seconds

Question: (iii) To display Minimum and Maximum Price of each Name of accessories.

Input:

select name,min(price),max(price) from accessories group by name;

Output:

Results Explain Describe Saved SQL His

NAME	MIN(PRICE)	MAX(PRICE)
Mother Board	12000	13000
LCD	5500	6000
Mouse	300	350
Keyboard	400	500
Hard Disk	4500	5000

Question: (iv) To display Name, Price of all accessories and their respective SName where they are available.

Input:

select a.name,a.price,s.sname from accessories a,shoppe s where substr(a.id,-2)=substr(s.id,-2);

Output:

Results Explain Describe Saved SQL

NAME	PRICE	SNAME
Mother Board	12000	ABC Computeronics
Hard Disk	5000	ABC Computeronics
Keyboard	500	All Infotech Media
Mouse	300	ABC Computeronics
Mother Board	13000	All Infotech Media
Keyboard	400	Tech Shoppe
LCD	6000	Greeks Techno Soft
LCD	5500	Hitech Tech Store
Mouse	350	Hitech Tech Store
Hard Disk	4500	Tech Shoppe

10 rows returned in 0.03 seconds CSV

Question: (b) (i) SELECT DISTINCT Name FROM ACCESSORIES WHERE Price>=500;

Input:

select distinct name from accessories where price>=500;

Output:

Results Explain Describe Save

NAME
Mother Board
LCD
Keyboard
Hard Disk

4 rows returned in 0.02 seconds

Question: (ii) SELECT Area, COUNT (*) FROM GROUP BY Area;

Input:

select area,count(*) from shoppe group by area;

Output:

Results Explain Describe Save

AREA	COUNT(*)
GK II	1
CP	2
Nehru Place	2

3 rows returned in 0.00 seconds

Question: (iii) SELECT COUNT (DISTINCT Area) FROM SHOPPE;

Input:

select count(distinct area) from shoppe;

Output:

Results Explain Describe Save

COUNT(DISTINCTAREA)
3

1 rows returned in 0.00 seconds

Question: (iv) SELECT Name, Price*0.05 DISCOUNT FROM ACCESSORIES WHERE SNo IN ('S02, 'S03');

Input:

select name,price*0.05 discount from accessories where id in('S02','S03');

Output:

Results Explain Describe Saved S

NAME	DISCOUNT
Keyboard	25
Mother Board	650
Keyboard	20
Hard Disk	225

4 rows returned in 0.01 seconds

6. Write SQL commands for (a) to (f) and Write the outputs for (g) on the basis of table HOSPITAL

(a) To select all the information of patients of cardiology department.
(b) To list the names of female patients who are in ENT department.
(c) To list name of all patients with their date of admission in ascending order.
(d) To display Patient's Name, Charges, Age for only female patients.
(e) To count the number of patients with Age<30.
(f) To inset in a new row in the HOSPITAL table with the following data: 11, "Aftab", 24, "Surgery", {25/02/98}, 300, "M"
(g) Give the output of following SQL statements:
(i) Select COUNT (DISTINCT charges) from HOSPITAL;
(ii) Select MIN (Age) from HOSPITAL where Gender = "F"
(iii) Select SUM (Charges) from HOSPITAL where Department = "ENT"
(iv) Select AVG (Charges) from HOSPITAL where Datofadm< {12/08/98}

Syntax:

```
create table hospital(no number primary key,name varchar2(20),age
number,department varchar2(30),dateofadm date,charges number,gender char(1));
insert into hospital values(1,'Arpit',62,'Surgery','21/01/98',300,'M');
insert into hospital values(2,'Zarina',22,'ENT','12/12/97',250,'F');
insert into hospital values(3,'Kareem',32,'Orthopedic','19/02/98',200,'M');
insert into hospital values(4,'Arun',12,'Surgery','11/01/98',300,'M');
insert into hospital values(5,'Zubin',30,'ENT','24/02/98',250,'M');
insert into hospital values(6,'Ketaki',16,'ENT','12/01/98',250,'M');
insert into hospital values(7,'Ankita',29,'Cardiology','20/02/98',800,'F');
insert into hospital values(8,'Zareen',45,'Gynecology','22/02/98',300,'F');
insert into hospital values(9,'Kush',19,'Cardiology','13/01/98',800,'M');
insert into hospital values(10,'Shilpa',23,'Nuclear Medicine','21/02/98',400,'F');
```

Question: (a) To select all the information of patients of cardiology department.

Input:

```
select * from hospital where department='cardiology';
```

Output:

Results Explain Describe Saved SQL History

NO	NAME	AGE	DEPARTMENT	DATEOFADM	CHARGES	GENDER
7	ankita	29	cardiology	20-FEB-98	800	f
9	kush	19	cardiology	13-JAN-98	800	f

2 rows returned in 0.02 seconds CSV Export

Question: (b) To list the names of female patients who are in ENT department.

Input:

select name from hospital where gender='f' and department='ent';

Output:

Results Explain Describe Sa

NAME
zarina

1 rows returned in 0.12 seconds

Question: (c) To list name of all patients with their date of admission in ascending order.

Input:

select name,dateofadm from hospital order by dateofadm asc;

Output:

Results Explain Describe Saved SQL History

NAME	DATEOFADM
zarina	12-DEC-97
arun	11-JAN-98
ketaki	12-JAN-98
kush	13-JAN-98
Arpit	21-JAN-98
kareem	19-FEB-98
ankita	20-FEB-98
shilpa	21-FEB-98
zareen	22-FEB-98
zubin	24-FEB-98
More than 10 rows available. Increase rows selector to view more rows.	

10 rows returned in 0.01 seconds CSV Export

Question: (d) To display Patient's Name, Charges, Age for only female patients.

Input:

select name,charges,age from hospital where gender='f';

Output:

Results Explain Describe Saved

NAME	CHARGES	AGE
zarina	250	22
ankita	800	29
zareen	300	45
kush	800	19
shilpa	400	23

5 rows returned in 0.00 seconds

Question: (e) To count the number of patients with Age<30

Input:

select count(*) from hospital where age<30;

Output:

Results Explain Describe Sa

COUNT(*)
7

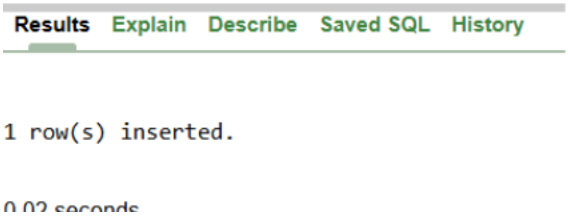
1 rows returned in 0.00 seconds

Question: (f) To inset in a new row in the HOSPITAL table with the following data: 11, "Aftab", 24, "Surgery", {25/02/98}, 300, "M"

Input:

insert into hospital values(11,'aftab',24,'surgery',to_date('25/02/1998','dd/mm/yyyy'),300,'m');

Output:



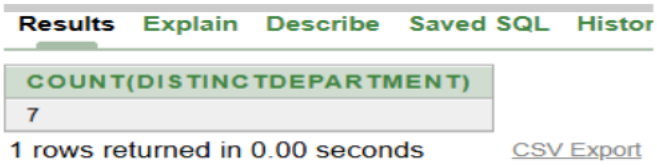
Question: (g) Give the output of following SQL statements:

(i) Select COUNT (DISTINCT charges) from HOSPITAL;

Input:

select count(distinct department) from hospital;

Output:

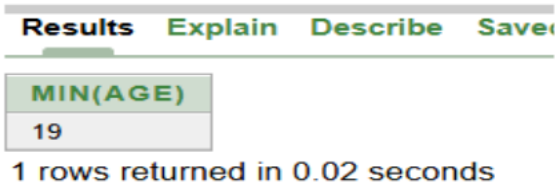


Question: (ii) Select MIN (Age) from HOSPITAL where Gender = "F"

Input:

select min(age) from hospital where gender='f';

Output:



Question: (iii) Select SUM (Charges) from HOSPITAL where Department = "ENT"

Input:

select sum(charges) from hospital where department='ent';

Output:

Results Explain Describe Sav

SUM(CHARGES)
750

1 rows returned in 0.03 seconds

Question: (iv) Select AVG (Charges) from HOSPITAL where Datofadm< {12/08/98}

Input:

select avg(charges) from hospital where dateofadm<to_date('12/08/1998','dd/mm/yyyy');

Output:

Results Explain Describe Saved SQL Histor

AVG(CHARGES)
377.27272727272727272727272727272727272727

1 rows returned in 0.03 seconds CSV Export

7.Answer the questions (a) and (b) on the basis of the following tables **STORE** and **ITEM**

(a) Write the SQL queries (i) to (iv):
(i) To display IName and Price of all the items in ascending order of their Price.
(ii) To display SNo and SName of all store location in CP.
(iii) To display Minimum and maximum Price of each IName from the table ITEM.
(iv) To display IName, Price of all items and their respective SName where they are available.
(b) Write the output of the following SQL commands (i) to (iv):
(i) SELECT DISTINCT IName FROM ITEM WHERE Price >=5000;
(ii) SELECT Area, COUNT (*) FROM STORE GROUP BY Area;
(iii) SELECT COUNT (DISTINCT Area) FROM STORE:
(iv) SELECT IName, Price * 0.05 DISCOUNT FROM ITEM WHERE SNo IN ('S02', 'S03');

STORE

SNo	SName	Area
S01	ABC Computronics	GK II
S02	All InfoTech Media	CP
S03	Tech Shoppe	Nehru Place
S04	Geeks Techno Soft	Nehru Place
S05	Hitech Tech Store	CP

ITEM

INo	IName	Price	SNo
T01	Mother Board	12000	S01
T02	Hard Disk	5000	S01
T03	Keyboard	500	S02
T04	Mouse	300	S01
T05	Mother Board	13000	S02
T06	Keyboard	400	S03
T07	LCD	6000	S04
T08	LCD	5500	S05
T09	Mouse	350	S05
T10	Hard Disk	4500	S03

Syntax:

```
create table stores(sno varchar2(3) primary key,sname varchar2(30),area varchar2(20));
create table item(ino varchar2(3) primary key,iname varchar2(20),price number,sno
varchar2(3));
insert into stores values('s01','abc computronics','gk ii');
insert into stores values('s02','all infotech media','cp');
insert into stores values('s03','tech shoppe','nehru place');
insert into stores values('s04','geeks techno soft','nehru place');
insert into stores values('s05','hitech tech store','cp');
insert into item values('t01','mother board',12000,'s01');
insert into item values('t02','hard disk',5000,'s01');
insert into item values('t03','keyboard',500,'s02');
insert into item values('t04','mouse',300,'s01');
insert into item values('t05','mother board',13000,'s02');
insert into item values('t06','keyboard',400,'s03');
insert into item values('t07','lcd',6000,'s04');
insert into item values('t08','lcd',5500,'s05');
insert into item values('t09','mouse',350,'s05');
insert into item values('t10','hard disk',4500,'s03');
```

Question: (i) To display IName and Price of all the items in ascending order of their Price.

Input:

```
select iname,price from item order by price asc;
```

Output:

Results Explain Describe Save

INAME	PRICE
mouse	300
mouse	350
keyboard	400
keyboard	500
hard disk	4500
hard disk	5000
lcd	5500
lcd	6000
mother board	12000
mother board	13000

10 rows returned in 0.05 seconds

Question: (ii) To display SNo and SName of all store location in CP.

Input:

```
select sno,sname from stores where area='cp';
```

Output:

Results Explain Describe Sav

SNO	SNAME
s02	all infotech media
s05	hitech tech store

2 rows returned in 0.04 seconds

Question: (iii) To display Minimum and maximum Price of each IName from the table ITEM.

Input:

select iname,min(price),max(price) from item group by iname;

Output:

Results Explain Describe Saved SQL Histo

INAME	MIN(PRICE)	MAX(PRICE)
lcd	5500	6000
hard disk	4500	5000
keyboard	400	500
mother board	12000	13000
mouse	300	350

5 rows returned in 0.01 seconds CSV Export

Question: (iv) To display IName, Price of all items and their respective SName where they are available.

Input:

select iname,price,sname from item,stores where item.sno=stores.sno;

Output:

Results Explain Describe Saved SQL

INAME	PRICE	SNAME
mother board	12000	abc computronics
hard disk	5000	abc computronics
keyboard	500	all infotech media
mouse	300	abc computronics
mother board	13000	all infotech media
keyboard	400	tech shoppe
lcd	6000	geeks techno soft
lcd	5500	hitech tech store
mouse	350	hitech tech store
hard disk	4500	tech shoppe

10 rows returned in 0.03 seconds CSV

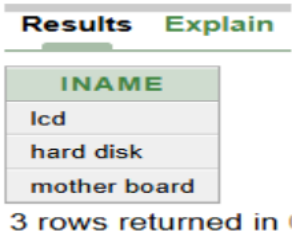
Question: (b) Write the output of the following SQL commands (i) to (iv):

(i) SELECT DISTINCT IName FROM ITEM WHERE Price >=5000;

Input:

select distinct iname from item where price>=5000;

Output:



Results Explain

INAME
lcd
hard disk
mother board

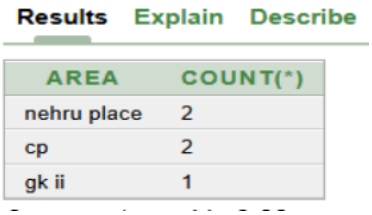
3 rows returned in

Question: (ii) SELECT Area, COUNT (*) FROM STORE GROUP BY Area;

Input:

select area,count(*) from stores group by area;

Output:



Results Explain Describe

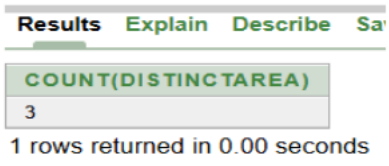
AREA	COUNT(*)
nehru place	2
cp	2
gk ii	1

Question: (iii) SELECT COUNT (DISTINCT Area) FROM STORE:

Input:

select count(distinct area) from stores;

Output:



Results Explain Describe Sa

COUNT(DISTINCTAREA)
3

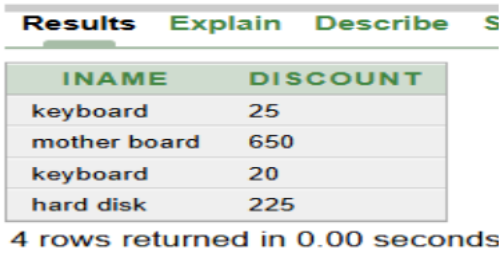
1 rows returned in 0.00 seconds

Question: (iv) SELECT IName, Price * 0.05 DISCOUNT FROM ITEM WHERE SNo IN ('S02', 'S03');

Input:

select iname,price*0.05 discount from item where sno in('s02','s03');

Output:



Results Explain Describe S

INAME	DISCOUNT
keyboard	25
mother board	650
keyboard	20
hard disk	225

4 rows returned in 0.00 seconds

8. Write SQL commands for (a) to (f) and write output for (g) on the basis of Teacher relation given below:

(a) To show all information about the teacher of history department
(b) To list the names of female teacher who are in Hindi department
(c) To list names of all teachers with their date of joining in ascending order.
(d) To display teacher's Name, Salary, Age for male teacher only
(e) To count the number of teachers with Age>23.
(f) To inset a new row in the TEACHER table with the following data: 9, "Raja", 26, "Computer", {13/05/95}, 2300, "M"
(g) Give the output of following SQL statements:
(i) Select COUNT (distinct department) from TEACHER;
(II) Select MAX (Age) from TEACHER where Gender= "F"
(iii) Select AVG (Salary) from TEACHER where Date of join < {12/07/96};
(iv) Select SUM (Salary) from TEACHER where Date of join < {12/07/96};

No.	Name	Age	Department	Date of Join	Salary	Gender
1	Jugal	34	Computer	10/01/97	12000	M
2	Sharmila	31	History	24/03/98	20000	F
3	Sandeep	32	Maths	12/12/96	30000	M
4	Sangeeta	35	History	01/07/99	40000	F
5	Rakesh	42	Maths	05/09/97	25000	M
6	Shyam	50	History	27/06/98	30000	M
7	Shiv Om	44	Computer	25/02/97	21000	M
8	Shalakha	33	Maths	31/07/97	20000	F

Syntax:

create table teacher(no number primary key,name varchar2(20),age number,department varchar2(20),dateofjoin date,salary number,gender varchar2(1));
insert into teacher values(1,'jugal',34,'computer',to_date('10/01/97','dd/mm/yy'),12000,'m');
insert into teacher values(2,'sharmila',31,'history',to_date('24/03/98','dd/mm/yy'),20000,'f');
insert into teacher values(3,'sandeep',32,'maths',to_date('12/12/96','dd/mm/yy'),30000,'m');
insert into teacher values(4,'sangeeta',35,'history',to_date('01/07/99','dd/mm/yy'),40000,'f');
insert into teacher values(5,'rakesh',42,'maths',to_date('05/09/97','dd/mm/yy'),25000,'m');
insert into teacher values(6,'shyam',50,'history',to_date('27/06/98','dd/mm/yy'),30000,'m');

insert into teacher values(7,'shiv om',44,'computer',to_date('25/02/97','dd/mm/yy'),21000,'m');
insert into teacher values(8,'shalakha',33,'maths',to_date('31/07/97','dd/mm/yy'),20000,'f');

Question: (a) To show all information about the teacher of history department

Input:

select * from teacher where department='history';

Output:

Results Explain Describe Saved SQL History

NO	NAME	AGE	DEPARTMENT	DATEOFJOIN	SALARY	GENDER
2	sharmila	31	history	24-MAR-98	20000	f
4	sangeeta	35	history	01-JUL-99	40000	f
6	shyam	50	history	27-JUN-98	30000	m

3 rows returned in 0.12 seconds CSV Export

Question: (b) To list the names of female teacher who are in Hindi department

Input:

select name from teacher where gender='f' and department='maths';

Output:

Results Explain Desc

NAME
shalakha

1 rows returned in 0.02 s

Question: (c) To list names of all teachers with their date of joining in ascending order.

Input:

select name,dateofjoin from teacher order by dateofjoin asc;

Output:

Results Explain Describe Sa

NAME	DATEOFJOIN
sandeep	12-DEC-96
jugal	10-JAN-97
shiv om	25-FEB-97
shalakha	31-JUL-97
rakesh	05-SEP-97
sharmila	24-MAR-98
shyam	27-JUN-98
sangeeta	01-JUL-99

8 rows returned in 0.01 seconds

Question: (d) To display teacher's Name, Salary, Age for male teacher only

Input:

select name,salary,age from teacher where gender='m';

Output:

Results Explain Describe S

NAME	SALARY	AGE
jugal	12000	34
sandeep	30000	32
rakesh	25000	42
shyam	30000	50
shiv om	21000	44

5 rows returned in 0.01 seconds

Question: (e) To count the number of teachers with Age>23.

Input:

select count(*) from teacher where age>23;

Output:

Results Explain Desc

COUNT(*)
8

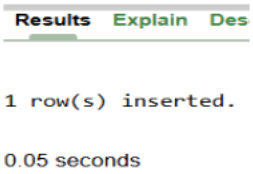
1 rows returned in 0.02 s

Question: (f) To inset a new row in the TEACHER table with the following data: 9, "Raja", 26, "Computer", {13/05/95}, 2300, "M"

Input:

insert into teacher values(9,'raja',26,'computer',to_date('13/05/95','dd/mm/yy'),2300,'m');

Output:

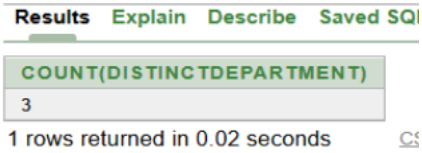


Question: (g) Give the output of following SQL statements: (i) Select COUNT (distinct department) from TEACHER;

Input:

select count(distinct department) from teacher;

Output:

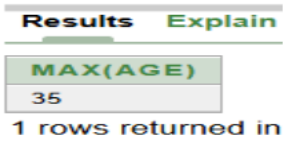


Question:(II) Select MAX (Age) from TEACHER where Gender= "F"

Input:

select max(age) from teacher where gender='f';

Output:



Question: (iii) Select AVG (Salary) from TEACHER where Date of join < {12/07/96};

Input:

select avg(salary) from teacher where dateofjoin<to_date('12/07/96','dd/mm/yy');

Output:

Results Explain Describe Sa

AVG(SALARY)
2300

1 rows returned in 0.04 seconds

Question: (iv) Select SUM (Salary) from TEACHER where Date of join < {12/07/96};

Input:

select sum(salary) from teacher where dateofjoin<to_date('12/07/96','dd/mm/yy');

Output:

Results Explain Describe Sav

SUM(SALARY)
2300

1 rows returned in 0.00 seconds

9. Given the following tables for a database LIBRARY:

Write SQL queries for (a) to (f):
(a) To show Book name, Author name and Price of books of EPB publishers.
(b) To list the names of the books of Fiction Type.
(c) To display the names and price of the books in descending order of their price.
(d) To increase the price of all books of first publisher by 50.
(e) To display the Book_Id, Book_Name and Quantity issued for all books which have been issued. (The query will require contents from both the tables).
(f) To insert a new row in the table Issued following the data: "F0002",4
(g) Give the output of the following queries based on the above tables:
(i) SELECT COUNT(DISTINCT Publishers) FROM Books;
(ii) SELECT SUM(Price) FROM Books WHERE Quantity > 5;
(iii) SELECT BOOK_NAME,AUTHOR_NAME FROM Books WHERE Price < 500;
(iv) SELECT COUNT (*) FROM Books;

Books

Book Id	Book Name	Author Name	Publishers	Price	Type	Qty.
F0001	The Tears	William Hopkins	First Publ.	750	Fiction	10
F0002	Thunderbolts	Anna Roberts	First Publ.	700	Fiction	5
T0001	My First C++	Briian & Brooke	EPB	250	Text	10
T0002	C++ Brainworks	A. W. Rossaine	TDH	325	Text	5
C0001	Fast Cook	LataKapoor	EPB	350	Cookery	8

Issued

Book Id	Quantity Issued
F0001	3
T0001	1
C0001	5

Syntax:

```
create table books(book_id varchar2(10) primary key,book_name varchar2(30),author_name
varchar2(30),publishers varchar2(20),price number,type varchar2(20),qty number);
insert into books values('f0001','the tears','william hopkins','first publ.',750,'fiction',10);
insert into books values('f0002','thunderbolts','anna roberts','first publ.',700,'fiction',5);
insert into books values('t0001','my first c++','brian & brooke','epb',250,'text',10);
insert into books values('t0002','c++ brainworks','a w rossaine','tdh',325,'text',5);
insert into books values('c0001','fast cook','latakapoop','epb',350,'cookery',8);
create table issued(book_id varchar2(10),quantity_issued number);
insert into issued values('f0001',3);
insert into issued values('t0001',1);
insert into issued values('c0001',5);
```

Question: (a) To show Book name, Author name and Price of books of EPB publishers.

Input:

```
select book_name,author_name,price from books where publishers='epb';
```

Output:



Results | Explain | Describe | Saved SQL | Hist

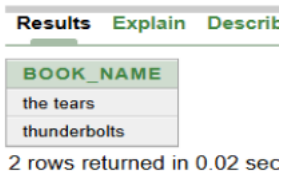
BOOK_NAME	AUTHOR_NAME	PRICE
my first c++	brian & brooke	250
fast cook	latakapoop	350

Question: (b) To list the names of the books of Fiction Type.

Input:

```
select book_name from books where type='fiction';
```

Output:



Results | Explain | Describ

BOOK_NAME
the tears
thunderbolts

2 rows returned in 0.02 sec

Question: (c) To display the names and price of the books in descending order of their price.

Input:

```
select book_name,price from books order by price desc;
```

Output:

Results Explain Describe Sa

BOOK_NAME	PRICE
the tears	750
thunderbolts	700
fast cook	350
c++ brainworks	325
my first c++	250

5 rows returned in 0.00 seconds

Question: (d) To increase the price of all books of first publisher by 50.

Input:

update books set price=price+50 where publishers='first publ.';

Output:

Results Explain Describe

```
2 row(s) updated.

0.10 seconds
```

Question: (e) To display the Book_Id, Book_Name and Quantity issued for all books which have been issued. (The query will require content from both the tables).

Input:

select books.book_id,book_name,quantity_issued from books,issued where books.book_id=issued.book_id;

Output:

Results Explain Describe Saved SQL History

BOOK_ID	BOOK_NAME	QUANTITY_ISSUED
f0001	the tears	3
t0001	my first c++	1
c0001	fast cook	5

3 rows returned in 0.11 seconds CSV Export

Question: (f) To insert a new row in the table Issued following the data: “F0002”,4

Input:

insert into issued values('f0002',4);

Output:

Results Explain Desc

```
1 row(s) inserted.

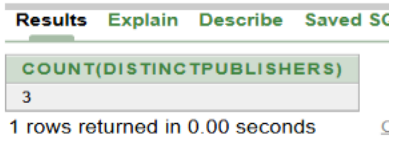
0.03 seconds
```

Question: (g) Give the output of the following queries based on the above tables:
(i) SELECT COUNT (DISTINCT Publishers) FROM Books;

Input:

SELECT COUNT (DISTINCT Publishers) FROM Books;

Output:

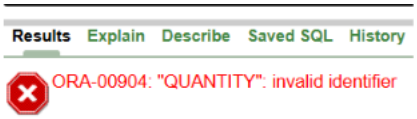


Question: (ii) SELECT SUM(Price) FROM Books WHERE Quantity > 5;

Input:

SELECT SUM(Price) FROM Books WHERE Quantity > 5;

Output:

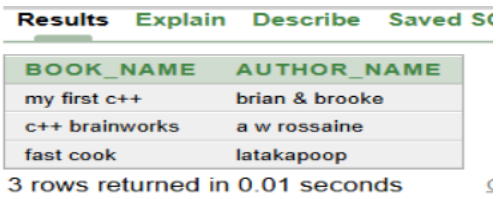


Question: (iii) SELECT BOOK_NAME, AUTHOR_NAME FROM Books WHERE Price < 500;

Input:

SELECT BOOK_NAME, AUTHOR_NAME FROM Books WHERE Price < 500;

Output:

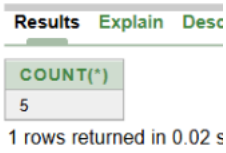


Question: (iv) SELECT COUNT (*) FROM Books;

Input:

SELECT COUNT (*) FROM Books;

Output:



10. Consider the following tables CARDEN and CUSTOMER and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

(i) To display the name of all the SILVER colored cars.

(ii) To display name of car, make and capacity of cars in descending order of their sitting capacity.

(iii) To display the highest Charges at which a vehicle can be hired from CARDEN.

(iv) To display the customer name and the corresponding name of the cards hired by them.

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (DISTINCT Make) FROM CARDEN;

(ii) SELECT MAX (Charges), MIN (Charges) FROM CARDEN;

(iii) SELECT COUNT (*), Make FROM CARDEN;

(iv) SELECT CarName FROM CARDEN WHERE Capacity = 4;

CARDEN

Ccode	CarName	Make	Color	Capacity	Charges
501	A-star	Suzuki	RED	3	14
503	Indigo	Tata	SILVER	3	12
502	Innova	Toyota	WHITE	7	15
509	SX4	Suzuki	SILVER	4	14
510	C-Class	Mercedes	RED	4	35

CUSTOMER

CCode	Cname	Ccode (Car Code)
1001	HamantSahu	501
1002	Raj Lal	509
1003	Feroja Shah	503
1004	Ketan Dhal	502

Syntax:

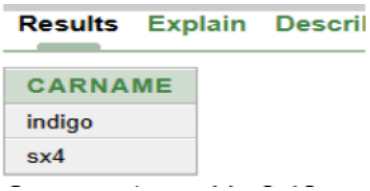
```
create table carden(ccode number,carname varchar2(20),make varchar2(20),color
varchar2(10),capacity number,charges number);
insert into carden values(501,'a-star','suzuki','red',3,14);
insert into carden values(503,'indigo','tata','silver',3,12);
insert into carden values(502,'innova','toyota','white',7,15);
insert into carden values(509,'sx4','suzuki','silver',4,14);
insert into carden values(510,'c-class','mercedes','red',4,35);
create table customers(ccode number,cname varchar2(20),code number);
insert into customers values(1001,'hamant sahu',501);
insert into customers values(1002,'raj lal',509);
insert into customers values(1003,'feroja shah',503);
insert into customers values(1004,'ketan dhal',502);
```

Question: (i) To display the name of all the SILVER colored cars.

Input:

select carname from carden where color='silver';

Output:



Results Explain Descril

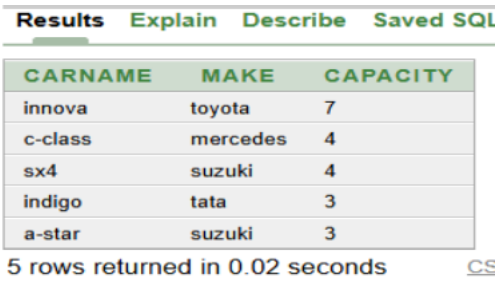
CARNAME
indigo
sx4

Question: (ii) To display name of car, make and capacity of cars in descending order of their sitting capacity.

Input:

select carname,make,capacity from carden order by capacity desc;

Output:



Results Explain Describe Saved SQL

CARNAME	MAKE	CAPACITY
innova	toyota	7
c-class	mercedes	4
sx4	suzuki	4
indigo	tata	3
a-star	suzuki	3

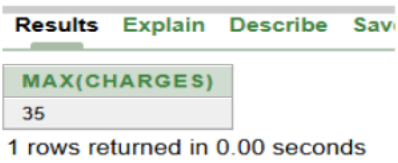
5 rows returned in 0.02 seconds

Question: (iii) To display the highest Charges at which a vehicle can be hired from CARDEN.

Input:

select max(charges) from carden;

Output:



Results Explain Describe Sav

MAX(CHARGES)
35

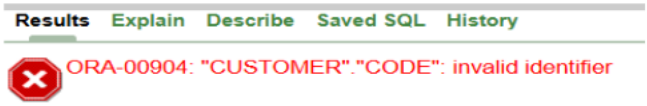
1 rows returned in 0.00 seconds

Question: (iv) To display the customer name and the corresponding name of the cards hired by them.

Input:

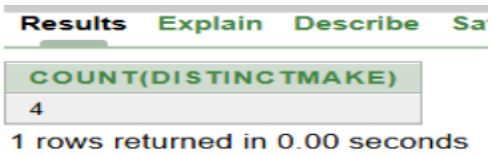
select customers.cname,carden.carname from customer,carden where customers.code=carden.ccode;

Output:



Question: (b) Give the output of the following SQL queries:
(i) SELECT COUNT (DISTINCT Make) FROM CARDEN;

Output:



Question: (ii) SELECT MAX (Charges), MIN (Charges) FROM CARDEN;

Output:



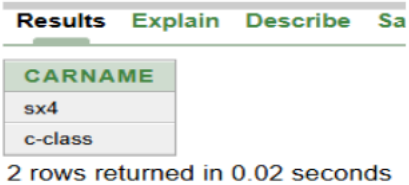
Question: (iii) SELECT COUNT (*), Make FROM CARDEN;

Output:



Question: (iv) SELECT CarName FROM CARDEN WHERE Capacity = 4;

Output:



11. Consider the following tables EMPLOYEE and SALGRADE and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

(i) To display the detail of all the EMPLOYEE in descending order of DOJ.

(ii) To display name and design of those EMPLOYEE, whose sgrade is either S02 or S03.

(iii) To display the content of all the EMPLOYEE table, whose DOJ is in between '09-FEB-2006' and '08-AUG-2009'.

(iv) TO add a new row in the EMPLOYEE table with the following data:
109, 'Harish Roy', 'HEAD-IT', 'S02', '09-SEP-2007', '21-APR-1983'.

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (SGRADE), SGRADE FROM EMPLOYEE GROUP BY SGRADE;

(ii) SELECT MIN (DOB), MAX (DOJ) FROM EMPLOYEE;

(iii) SELECT NAME, SALARY FROM EMPLOYEE E, SALGRADE S
WHERE E. SGRADE = S. SGRADE AND E. ECODE<103;

(iv) SELECT SGRADE, SALARY+HRA FROM SALGRADE WHERE SGRADE = 'S02';

EMPLOYEE

ECODE	NAME	DESIG	SGRADE	DOJ	DOB
101	Abdul Ahmad	EXECUTIVE	S03	23-MAR-2003	13-JAN-1980
102	Ravi Chander	HEAD-IT	S02	12-FEB-2010	22-JUL-1987
103	John Ken	Receptionist	S03	24-JUN-2009	24-FEB-1983
105	NazarAmeen	GM	S02	11-AUG-2006	03-MAR-1984
108	PriyamSen	CEO	S01	29-DEC-2004	19-JAN-1982

SALGRADE

SGRADE	SALARY	HRA
S01	56000	18000
S02	32000	12000
S03	24000	8000

Syntax:

```
create table employee(ecode number,name varchar2(20),desig varchar2(20),sgrade varchar2(5),doj date,dob date);
insert into employee values(101,'abdul ahmad','executive','s03','23-mar-2003','13-jan-1980');
insert into employee values(102,'ravi chander','head-it','s02','12-feb-2010','22-jul-1987');
insert into employee values(103,'john ken','receptionist','s03','24-jun-2009','24-feb-1983');
insert into employee values(105,'nazar meen','gm','s02','11-aug-2006','03-mar-1984');
insert into employee values(108,'priyam sen','ceo','s01','29-dec-2004','19-jan-1982');
create table salgrade(sgrade varchar2(5),salary number,hra number);
insert into salgrade values('s01',56000,18000);
insert into salgrade values('s02',32000,12000);
insert into salgrade values('s03',24000,8000);
```

Question: (i) To display the detail of all the EMPLOYEE in descending order of DOJ.

Input:

select * from employee order by doj desc;

Output:

Results Explain Describe Saved SQL History

ECODE	NAME	DESIG	SGRADE	DOJ	DOB
102	ravi chander	head-it	s02	12-FEB-10	22-JUL-87
103	john ken	receptionist	s03	24-JUN-09	24-FEB-83
105	nazar meen	gm	s02	11-AUG-06	03-MAR-84
108	priyam sen	ceo	s01	29-DEC-04	19-JAN-82
101	abdul ahmad	executive	s03	23-MAR-03	13-JAN-80

5 rows returned in 0.07 seconds CSV Export

Question: (ii) To display name and design of those EMPLOYEE, whose sgrade is either S02 or S03.

Input:

select name,desig from employee where sgrade in('s02','s03');

Output:

Results Explain Describe Sa

NAME	DESIG
abdul ahmad	executive
ravi chander	head-it
john ken	receptionist
nazar meen	gm

4 rows returned in 0.03 seconds

Question: (iii) To display the content of all the EMPLOYEE table, whose DOJ is in between ’09-FEB-2006’ and ’08-AUG-2009’.

Input:

select * from employee where doj between '09-feb-2006' and '08-aug-2009';

Output:

Results Explain Describe Saved SQL History

ECODE	NAME	DESIG	SGRADE	DOJ	DOB
103	john ken	receptionist	s03	24-JUN-09	24-FEB-83
105	nazar meen	gm	s02	11-AUG-06	03-MAR-84

2 rows returned in 0.11 seconds CSV Export

Question: (iv) TO add a new row in the EMPLOYEE table with the following data: 109, 'Harish Roy', 'HEAD-IT', 'S02', '09-SEP-2007', '21-APR-1983'.

Input:

```
insert into employee values(109,'harish roy','head-it','s02','09-sep-2007','21-apr-1983');
```

Output:

Results Explain De:

```
1 row(s) inserted.

0.00 seconds
```

Question: (b) Give the output of the following SQL queries:
(i) SELECT COUNT (SGRADE), SGRADE FROM EMPLOYEE GROUP BY SGRADE;

Output:

Results Explain Describe Saved

COUNT(SGRADE)	SGRADE
3	s02
1	s01
2	s03

3 rows returned in 0.03 seconds

Question: (ii) SELECT MIN (DOB), MAX (DOJ) FROM EMPLOYEE;

Output:

Results Explain Describe Sav

MIN(DOB)	MAX(DOJ)
13-JAN-80	12-FEB-10

1 rows returned in 0.00 seconds

Question: (iii) SELECT NAME, SALARY FROM EMPLOYEE E, SALGRADE S WHERE E. SGRADE = S. SGRADE AND E. ECODE<103;

Output:

Results Explain Describe Sa

NAME	SALARY
ravi chander	32000
abdul ahmad	24000

2 rows returned in 0.02 seconds

Question: (iv) SELECT SGRADE, SALARY+HRA FROM SALGRADE WHERE SGRADE = 'S02';

Output:

Results Explain Describe Save

SGRADE	SALARY+HRA
s02	44000

1 rows returned in 0.00 seconds

12. Consider the following tables WORKER and PAYLAVEL and answer (a) and (b) parts of this question

(a) Write SQL commands for the following statements:

(i) To display the detail of all WORKER in descending order of DOB.

(ii) To display name and design of those WORKER, whoseplevel is either P001 to P002.

(iii) To display the content of all the WORKER table, whose DOB is in between '19-JAN-1984' and '18-JAN-1987'.

(iv) To add a new row with the following:

19, 'Daya Kishore', 'Operator', 'P003', '19-JUN-2008', '11-JUL-1984'.

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (PLEVEL), PLEVEL FROM WORKER GROUP BY PLEVEL;

(ii) SELECT MAX (DOB), MIN (DOJ) FROM WORKER;

(iii) SELECT NAME, PAY FROM WORKER W, PAYLEVEL P WHERE W. PLEVEL= P.LEVEL AND W. ECODE<13;

(iv) SELECT PLEVEL, PAY+ALLOWANCE FROM PLEVEL WHERE PLEVEL = 'P003';

WORKER

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
11	RadheShyam	Supervisor	P001	13-SEP-2004	23-AUG-1981
12	ChanderNath	Operator	P003	22-FEB-2010	12-JUL-1987
13	Fizza	Operator	P003	14-JUN-2009	14-OCT-1983
15	Ahmeen Ahmad	Mechanic	P002	21-AUG-2006	13-MAR-1984
18	Sanya	Clerk	P002	19-DEC-2005	09-JUN-1983

PAYLAVEL

PLEVEL	PAY	ALLOWANCE
P001	26000	12000
P002	22000	10000
P003	12000	6000

Syntax:

```
create table worker(ecode number,name varchar2(20),design varchar2(20),plevel varchar2(5),doj date,dob date);
insert into worker values(11,'radheshyam','supervisor','p001','13-sep-2004','23-aug-1981');
```

```
insert into worker values(12,'chandernath','operator','p003','22-feb-2010','12-jul-1987');
insert into worker values(13,'fizza','operator','p003','14-jun-2009','14-oct-1983');
insert into worker values(15,'ahmeen ahmad','mechanic','p002','21-aug-2006','13-mar-1984');
insert into worker values(18,'sanya','clerk','p002','19-dec-2005','09-jun-1983');
create table paylevel(plevel varchar2(5),pay number,allowance number);
insert into paylevel values('p001',26000,12000);
insert into paylevel values('p002',22000,10000);
insert into paylevel values('p003',12000,6000);
```

Question: (i) To display the detail of all WORKER in descending order of DOB.

Input:

```
select * from worker order by dob desc;
```

Output:

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
12	chandernath	operator	p003	22-FEB-10	12-JUL-87
15	ahmeen ahmad	mechanic	p002	21-AUG-06	13-MAR-84
13	fizza	operator	p003	14-JUN-09	14-OCT-83
18	sanya	clerk	p002	19-DEC-05	09-JUN-83
11	radheshyam	supervisor	p001	13-SEP-04	23-AUG-81

5 rows returned in 0.08 seconds CSV Export

Question: (ii) To display name and design of those WORKER, whoseplevel is either P001 to P002.

Input:

```
select name,design from worker where plevel in('p001','p002');
```

Output:

Results Explain Describe Sav

NAME	DESIGN
radheshyam	supervisor
ahmeen ahmad	mechanic
sanya	clerk

3 rows returned in 0.00 seconds

Question: (iii) To display the content of all the WORKER table, whose DOB is in between '19-JAN-1984' and '18-JAN-1987'.

Input:

```
select * from worker where dob between '19-jan-1984' and '18-jan-1987';
```

Output:

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
15	ahmeen ahmad	mechanic	p002	21-AUG-06	13-MAR-84

1 rows returned in 0.00 seconds CSV Export

Question: (iv) To add a new row with the following:
19, 'Daya Kishore', 'Operator', 'P003', '19-JUN-2008', '11-JUL-1984'.

Input:

```
insert into worker values(19,'deva kishore','operator','p003','19-jun-2008','11-jul-1984');
```

Output:

Results Explain De:

```
1 row(s) inserted.

0.00 seconds
```

Question: (b) Give the output of the following SQL queries:
(i) SELECT COUNT (PLEVEL), PLEVEL FROM WORKER GROUP BY PLEVEL;

Output:

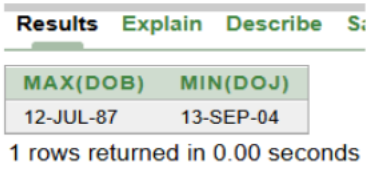
Results Explain Describe Sa

COUNT(PLEVEL)	PLEVEL
1	p001
2	p002
3	p003

3 rows returned in 0.02 seconds

Question: (ii) SELECT MAX (DOB), MIN (DOJ) FROM WORKER;

Output:



Results Explain Describe S:

MAX(DOB)	MIN(DOJ)
12-JUL-87	13-SEP-04

1 rows returned in 0.00 seconds

Question: (iii) SELECT NAME, PAY FROM WORKER W, PAYLEVEL P WHERE W. PLEVEL= P.LEVEL AND W. ECODE<13;

Output:



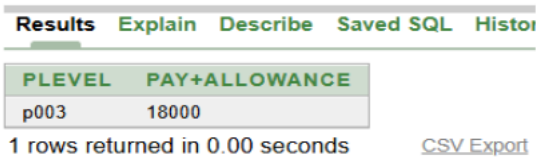
Results Explain Describe

NAME	PAY
fizza	12000

1 rows returned in 0.03 seconc

Question: (iv) SELECT PLEVEL, PAY+ALLOWANCE FROM PLEVEL WHERE PLEVEL = 'P003';

Output:



Results Explain Describe Saved SQL Histor

PLEVEL	PAY+ALLOWANCE
p003	18000

1 rows returned in 0.00 seconds CSV Export

13. Consider the following tables STORE and SUPPLIERS and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

(i) To display details of all the items in the Store table in ascending order of LastBuy.

(ii) To display ItemNo and Item name of those items from STORE table whose Rate is more than15 Rupees.

(iii)To display the details of those items whose Supplier code (Scode) is 22 or Quantity in Store (Qty) is more than 110 from the table STORE.

(iv)To display minimum Rate of items for each supplier individually as per Scode from the table STORE.

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (DISTINCT Scode) FROM STORE;

(ii)SELECT Rate * Qty FROM STORE WHERE ItemNo=2004;

(iii)SELECT Item,Sname FROM Store S, SUPPLIERS P WHERE S.Scode=P.ScodeANDItemNo=2006;

(iv) SELECT MAX(LastBuy) FROM STORE;

STORE

ItemNo	Item	Scode	Qty	Rate	LastBuy
2005	Sharpener Classic	23	60	8	31-JUN-09
2003	Ball Pen 0.25	22	50	25	01-FEB-09
2002	Gel Pen Premium	21	150	12	24-FEB-10
2006	Gel Pen Classic	21	250	20	11-MAY-09
2001	Eraser Small	22	220	6	19-JAN-09
2004	Eraser Big	22	110	8	02-DEC-09
2009	Ball Pen 0.5	21	180	18	03-NOV-09

SUPPLIERS

Scode	Sname
21	Premium Stationers
23	Soft Plastics
22	Tetra Supply

Syntax:

```
create table store(itemno number,item varchar2(30),scode number,qty number,rate number,lastbuy date);
insert into store values(2005,'sharpener classic',23,60,8,'30-jun-2009');
insert into store values(2003,'ball pen 0.25',22,50,25,'01-feb-2009');
insert into store values(2002,'gel pen premium',21,150,12,'24-feb-2010');
insert into store values(2006,'gel pen classic',21,250,20,'11-may-2009');
insert into store values(2001,'eraser small',22,220,6,'19-jan-2009');
insert into store values(2004,'eraser big',22,110,8,'02-dec-2009');
insert into store values(2009,'ball pen 0.5',21,180,18,'03-nov-2009');
create table suppliers(scode number,sname varchar2(30));
insert into suppliers values(21,'premium stationers');
insert into suppliers values(23,'soft plastics');
insert into suppliers values(22,'tetra supply');
```

Question: (i) To display details of all the items in the Store table in ascending order of LastBuy.

Input:

```
select * from store order by lastbuy asc;
```

Output:

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2001	eraser small	22	220	6	19-JAN-09
2003	ball pen 0.25	22	50	25	01-FEB-09
2006	gel pen classic	21	250	20	11-MAY-09
2005	sharpener classic	23	60	8	30-JUN-09
2009	ball pen 0.5	21	180	18	03-NOV-09
2004	eraser big	22	110	8	02-DEC-09
2002	gel pen premium	21	150	12	24-FEB-10

7 rows returned in 0.08 seconds CSV Export

Question: (ii) To display ItemNo and Item name of those items from STORE table whose Rate is more than15 Rupees.

Input:

select itemno,item from store where rate>15;

Output:

Results Explain Describe S

ITEMNO	ITEM
2003	ball pen 0.25
2006	gel pen classic
2009	ball pen 0.5

3 rows returned in 0.02 seconds

Question: (iii)To display the details of those items whose Supplier code (Scode) is 22 or Quantity in Store (Qty) is more than 110 from the table STORE.

Input:

select * from store where scode=22 or qty>110;

Output:

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2003	ball pen 0.25	22	50	25	01-FEB-09
2002	gel pen premium	21	150	12	24-FEB-10
2006	gel pen classic	21	250	20	11-MAY-09
2001	eraser small	22	220	6	19-JAN-09
2004	eraser big	22	110	8	02-DEC-09
2009	ball pen 0.5	21	180	18	03-NOV-09

6 rows returned in 0.02 seconds CSV Export

Question: (iv)To display minimum Rate of items for each supplier individually as per Scode from the table STORE.

Input:

select scode,min(rate) from store group by scode;

Output:

Results Explain Describ

SCODE	MIN(RATE)
22	6
21	12
23	8

3 rows returned in 0.01 sec

Question: (b) Give the output of the following SQL queries:

(i) SELECT COUNT (DISTINCT Scode) FROM STORE;

Output:

Results Explain Describe S

RATE*QTY
880

1 rows returned in 0.02 seconds

Question: (ii)SELECT Rate * Qty FROM STORE WHERE ItemNo=2004;

Output:

Results Explain Describe

ITEM
gel pen classic

1 rows returned in 0.03 secor

Question: (iii)SELECT Item,Sname FROM Store S, SUPPLIERS P WHERE S.Scode=P.ScodeANDItemNo=2006;

Output:

Results Explain Describe Sa

MAX(LASTBUY)
24-FEB-10

1 rows returned in 0.00 seconds

Question: (iv) SELECT MAX(LastBuy) FROM STORE;

Output:

Results Explain Describe S

ITEMNO	ITEM
2003	ball pen 0.25
2006	gel pen classic
2009	ball pen 0.5

3 rows returned in 0.02 seconds

14. Consider the following table GARMENT and FABRIC,

Write SQL commands for the statements (i) to (iv) and give outputs for the SQL queries (v) to (viii).

(i) To display GCODE and DESCRIPTION of each GARMENT in descending order of GCODE.
(ii) To display the details of all the GARMENT, which have READYDATE in between 08-DEC-07 and 16-JUN-08 (inclusive if both the dates).
(iii) To display the average PRICE of all the GARMENT, which are made up of fabric with FCODE as F03.
(iv) To display fabric wise highest and lowest price of GARMENT from GARMENT table. (Display FCODE of each GARMENT along with highest and lowest Price).
(v) SELECT SUM (PRICE) FROM GARMENT WHERE FCODE = 'F01';
(vi) SELECT DESCRIPTION, TYPE FROM GARMENT, FABRIC WHERE GARMENT, FCODE = FABRIC.FCODE AND GARMENT.PRICE >=1260;
(vii) SELECT MAX (FCODE) FROM FABRIC;
(viii) SELECT COUNT (DISTINCT PRICE) FROM GARMENT;

GARMENT

GCODE	DESCRIPTION	PRICE	FCODE	READYDATE
10023	Pencil Skirt	1150	F03	19-DEC-08
10001	Formal Shirt	1250	F01	12-JAN-08
10012	Informal Shirt	1550	F02	06-JUN-08
10024	Baby Top	750	F03	07-APR-07
10090	Tulip Skirt	850	F02	31-MAR-07
10019	Evening Gown	850	F03	06-JUN-08
10009	Informal Pant	1500	F02	20-OCT-08
10007	Formal Pant	1350	F01	09-MAR-08
10020	Frock	850	F04	09-SEP-07
10089	Slacks	750	F03	20-OCT-08

FABRIC

FCODE	TYPE
F04	Polyester
F02	Cotton
F03	Silk
F01	Terelene

Syntax:

```
create table fabric(fcode varchar2(3) primary key,type varchar2(20));
insert into fabric values('F04','Polyester');
insert into fabric values('F02','Cotton');
insert into fabric values('F03','Silk');
insert into fabric values('F01','Terelene');
create table garment(gcode number(5) primary key,description varchar2(30),price
number(5),fcode varchar2(3),readydate date);
insert into garment values(10023,'Pencil Skirt',1150,'F03',to_date('19-DEC-08','DD-MON-RR'));
insert into garment values(10001,'Formal Shirt',1250,'F01',to_date('12-JAN-08','DD-MON-
RR'));
insert into garment values(10012,'Informal Shirt',1550,'F02',to_date('06-JUN-08','DD-MON-
RR'));
insert into garment values(10024,'Baby Top',750,'F03',to_date('07-APR-07','DD-MON-RR'));
insert into garment values(10090,'Tulip Skirt',850,'F02',to_date('31-MAR-07','DD-MON-RR'));
insert into garment values(10019,'Evening Gown',850,'F03',to_date('06-JUN-08','DD-MON-
RR'));
insert into garment values(10009,'Informal Pant',1500,'F02',to_date('20-OCT-08','DD-MON-
RR'));
insert into garment values(10007,'Formal Pant',1350,'F01',to_date('09-MAR-08','DD-MON-
RR'));
insert into garment values(10020,'Frock',850,'F04',to_date('09-SEP-07','DD-MON-RR'));
insert into garment values(10089,'Slacks',750,'F03',to_date('20-OCT-08','DD-MON-RR'));
```

Question: (i) To display GCODE and DESCRIPTION of each GARMENT in descending order of GCODE.

Input:

select gcode,description from garment order by gcode desc;

Output:

Results Explain Describe Saved SC

GCODE	DESCRIPTION
10090	Tulip Skirt
10089	Slacks
10024	Baby Top
10023	Pencil Skirt
10020	Frock
10019	Evening Gown
10012	Informal Shirt
10009	Informal Pant
10007	Formal Pant
10001	Formal Shirt

10 rows returned in 0.04 seconds

Question: (ii) To display the details of all the GARMENT, which have READYDATE in between 08-DEC-07 and 16-JUN-08 (inclusive if both the dates).

Input:

select * from garment where readydate between to_date('08-dec-07','dd-mon-rr') and to_date('16-jun-08','dd-mon-rr');

Output:

Results Explain Describe Saved SQL History

GCODE	DESCRIPTION	PRICE	FCODE	READYDATE
10001	Formal Shirt	1250	F01	12-JAN-08
10012	Informal Shirt	1550	F02	06-JUN-08
10019	Evening Gown	850	F03	06-JUN-08
10007	Formal Pant	1350	F01	09-MAR-08

4 rows returned in 0.03 seconds CSV Export

Question: (iii) To display the average PRICE of all the GARMENT, which are made up of fabric with FCODE as F03.

Input:

select avg(price) from garment,fabric where garment.fcode=fabric.fcode and fabric.fcode='f03';

Output:

Results Explain Describe

AVG(PRICE)
875

1 rows returned in 0.00 seconds

Question: (iv) To display fabric wise highest and lowest price of GARMENT from GARMENT table. (Display FCODE of each GARMENT along with highest and lowest Price).

Input:

select fabric.fcode,max(price),min(price) from garment,fabric where garment.fcode=fabric.fcode group by fabric.fcode;

Output:

Results Explain Describe Saved SQL Hi

FCODE	MAX(PRICE)	MIN(PRICE)
F02	1550	850
F04	850	850
F03	1150	750
F01	1350	1250

4 rows returned in 0.03 seconds CSV Exp

Question: (v) SELECT SUM (PRICE) FROM GARMENT WHERE FCODE = 'F01';

Output:

Results Explain Describe Sav

SUM(PRICE)
2600

1 rows returned in 0.00 seconds

Question: (vi) SELECT DESCRIPTION, TYPE FROM GARMENT, FABRIC WHERE GARMENT, FCODE = FABRIC.FCODE AND GARMENT.PRICE >=1260;

Output:

Results Explain Describe Saved SQL H

DESCRIPTION	TYPE
Informal Pant	Cotton
Informal Shirt	Cotton
Formal Pant	Terelene

3 rows returned in 0.02 seconds CSV Ex

Question: (vii) SELECT MAX (FCODE) FROM FABRIC;

Output:

Results Explain Describe Sav

MAX(FABRIC.FCODE)
F04

1 rows returned in 0.01 seconds

Question: (viii) SELECT COUNT (DISTINCT PRICE) FROM GARMENT;

Output:

Results Explain Describe Saved SQL History

COUNT(DISTINCTPRICE)
7

1 rows returned in 0.00 seconds CSV Export

15. Given the following tables for a database FURNITURE : Write SQL command for (a) to (f) and write the outputs for (g) on the bases of tables **FURNITURE** AND **ARRIVALS.**

(a) To show all information about the baby cots from the FURNITURE table.
(b) To list the ITEMNAME which are priced at more than 15000 from the FURNITURE table.
(c) To list ITEMNAME AND TYPE of those items, in which DATEOFSTOCK is before 22/01/02 from the FURNITURE table in descending order of ITEMNAME.
(d) To display ITEMNAME and DATEOFSTOCK of those items, in which the DISCOUNT percentage is more than 25 from FURNITURE table.
(e) To count the number of items, whose TYPE is "Sofa" from FURNITURE table.
(f) To insert a new row in the ARRIVALS table with the following data: 14, "Velvet touch", Double bed", {25/03/03}, 25000, 30
(g) Give the output of following SQL statement:
Note: outputs of the below mentioned queries should be based on original data given in both the tables i.e., without considering the insertion done in (g) part of this question.
(i) Select COUNT(distinct TYPE) from FURNITURE;
(ii) Select MAX(DISCOUNT) from FURNITURE, ARRIVALS;

(iii) Select AVG(DISCOUNT) from FURNITURE where TYPE = “Baby cot”;
(iv) Select SUM(PRICE) from FURNITURE where DATEOFSTOCK < {12/02/02};

FURNITURE

No.	Item Name	Type	Date of Stock	Price	Discount
1	White lotus	Double Bed	23/02/02	30000	25
2	Pink feather	Baby cot	20/01/02	7000	20
3	Dolphin	Baby cot	19/02/02	9500	20
4	Decent	Office Table	01/01/02	25000	30
5	Comfort zone	Double Bed	12/01/02	25000	25
6	Donald	Baby cot	24/02/02	6500	15
7	Royal Finish	Office Table	20/02/02	18000	30
8	Royal tiger	Sofa	22/02/02	31000	30
9	Econo sitting	Sofa	13/12/01	9500	25
10	Eating paradise	Dining Table	19/02/02	11500	25

ARRIVALS

No.	Item Name	Type	Date of Stock	Price	Discount
11	Wood Comfort	Double Bed	23/03/03	25000	25
12	Old Fox	Sofa	20/02/03	17000	20
13	Micky	Baby cot	21/02/02	7500	15

Syntax:

```
create table furniture(no number(2),itemname varchar2(30),type
varchar2(20),dateofstock date,price number(6),discount number(2));
insert into furniture values(1,'white lotus','double
bed',to_date('23/02/02','dd/mm/yy'),30000,25);
insert into furniture values(2,'pink feather','baby
cot',to_date('20/01/02','dd/mm/yy'),7000,20);
insert into furniture values(3,'dolphin','baby cot',to_date('19/02/02','dd/mm/yy'),9500,20);
insert into furniture values(4,'decent','office
table',to_date('01/01/02','dd/mm/yy'),25000,30);
insert into furniture values(5,'comfort zone','double
bed',to_date('12/01/02','dd/mm/yy'),25000,25);
insert into furniture values(6,'donald','baby cot',to_date('24/02/02','dd/mm/yy'),6500,15);
insert into furniture values(7,'royal finish','office
table',to_date('20/02/02','dd/mm/yy'),18000,30);
insert into furniture values(8,'royal tiger','sofa',to_date('22/02/02','dd/mm/yy'),31000,30);
insert into furniture values(9,'econo
sitting','sofa',to_date('13/12/01','dd/mm/yy'),9500,25);
insert into furniture values(10,'eating paradise','dining
table',to_date('19/02/02','dd/mm/yy'),11500,25);
create table arrivals(no number(2),itemname varchar2(30),type
varchar2(20),dateofstock date,price number(6),discount number(2));
insert into arrivals values(11,'wood comfort','double
bed',to_date('23/03/03','dd/mm/yy'),25000,25);
insert into arrivals values(12,'old fox','sofa',to_date('20/02/03','dd/mm/yy'),17000,20);
insert into arrivals values(13,'micky','baby cot',to_date('21/02/02','dd/mm/yy'),7500,15);
```

Question: (a) To show all information about the baby cots from the FURNITURE table.

Input:select * from furniture where type='baby cot';

Output:

Results Explain Describe Saved SQL History

NO	ITEMNAME	TYPE	DATEOFSTOCK	PRICE	DISCOUNT
2	pink feather	baby cot	20-JAN-02	7000	20
3	dolphin	baby cot	19-FEB-02	9500	20
6	donald	baby cot	24-FEB-02	6500	15

3 rows returned in 0.00 seconds CSV Export

Question: (b) To list the ITEMNAME which are priced at more than 15000 from the FURNITURE table.

Input:select itemname from furniture where price>15000;

Output:

Results Explain D

ITEMNAME
white lotus
decent
comfort zone
royal finish
royal tiger

5 rows returned in 0.0

Question: (c) To list ITEMNAME AND TYPE of those items, in which DATEOFSTOCK is before 22/01/02 from the FURNITURE table in descending order of ITEMNAME.

Input:

select itemname,type from furniture where dateofstock<to_date('22/01/02','dd/mm/yy') order by itemname desc;

Output:

Results Explain Describe Sa

ITEMNAME	TYPE
pink feather	baby cot
econo sitting	sofa
decent	office table
comfort zone	double bed

4 rows returned in 0.01 seconds

Question: (d) To display ITEMNAME and DATEOFSTOCK of those items, in which the DISCOUNT percentage is more than 25 from FURNITURE table.

Input:select itemname,dateofstock from furniture where discount>25;

Output:

Results Explain Describe Saved SQL

ITEMNAME	DATEOFSTOCK
decent	01-JAN-02
royal finish	20-FEB-02
royal tiger	22-FEB-02

3 rows returned in 0.00 seconds CSV

Question: (e) To count the number of items, whose TYPE is “Sofa” from FURNITURE table.

Input: select count(*) from furniture where type='sofa';

Output:

Results Explain Describe Sa

COUNT(*)
2

1 rows returned in 0.00 seconds

Question: (f) To insert a new row in the ARRIVALS table with the following data: 14, “Velvet touch”, Double bed”, {25/03/03}, 25000, 30

Input: insert into arrivals values(14,'velvet touch','double bed',to_date('25/03/03','dd/mm/yy'),25000,30);

Output:



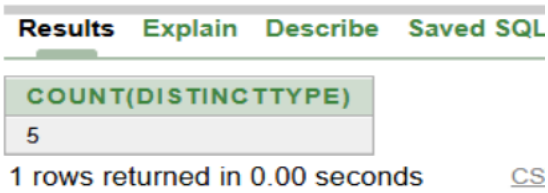

```
1 row(s) inserted.
```

Question: (g) Give the output of following SQL statement:

Note: outputs of the below mentioned queries should be based on original data given in both the tables i.e., without considering the insertion done in (g) part of this question.

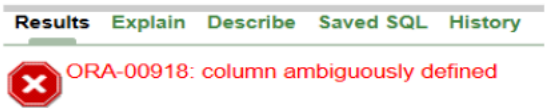
(i) Select COUNT(distinct TYPE) from FURNITURE;

Output:



Question: (ii) Select MAX(DISCOUNT) from FURNITURE, ARRIVALS;

Output:



Question: (iii) Select AVG(DISCOUNT) from FURNITURE where TYPE = "Baby cot";

Output:



Question: (iv) Select SUM(PRICE) from FURNITURE where DATEOFSTOCK < {12/02/02};

Output:

Results Explain Describe Saved SQL History

SUM(PRICE)
66500

1 rows returned in 0.00 seconds CSV Export

16. Given the following COVID19 relation, write SQL commands for questions (a) to (f).

a) Display all details of patients from **Tamil Nadu**.
b) Display the names of patients whose **Status** is **Recovered**.
c) Display **PatientName** and **Age** in ascending order of **Age**.
d) Display the names of patients who are **Vaccinated = 'Yes'**.
e) Count the number of patients whose status is **Active**.
f) Insert the following record into the **COVID19** table:
108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes'
(g) Write the output of the following SQL commands.
a. SELECT COUNT(*) FROM COVID19;
b. SELECT MAX(Age) FROM COVID19;
c. SELECT COUNT(*) FROM COVID19 WHERE Vaccinated='Yes';
d. SELECT MIN(Age)FROM COVID19 WHERE Status='Recovered';

PatientID	PatientName	State	Age	Status	Vaccinated
101	Arun	Tamil Nadu	35	Recovered	Yes
102	Priya	Kerala	42	Active	Yes
103	Karthik	Karnataka	28	Recovered	No
104	Divya	Tamil Nadu	31	Active	Yes
105	Rahul	Andhra Pradesh	55	Deceased	No
106	Meena	Kerala	24	Recovered	Yes
107	Suresh	Tamil Nadu	48	Active	No

Syntax:

```
create table covid19(patientid number(3),patientname varchar2(20),state varchar2(20),age
number(3),status varchar2(15),vaccinated varchar2(3));
insert into covid19 values(101,'arun','tamil nadu',35,'recovered','yes');
insert into covid19 values(102,'priya','kerala',42,'active','yes');
insert into covid19 values(103,'karthik','karnataka',28,'recovered','no');
insert into covid19 values(104,'divya','tamil nadu',31,'active','yes');
insert into covid19 values(105,'rahul','andhra pradesh',55,'deceased','no');
insert into covid19 values(106,'meena','kerala',24,'recovered','yes');
insert into covid19 values(107,'suresh','tamil nadu',48,'active','no');
```

Question: a) Display all details of patients from **Tamil Nadu**.

Input:

select * from covid19 where state='tamil nadu';

Output:

Results Explain Describe Saved SQL History

PATIENTID	PATIENTNAME	STATE	AGE	STATUS	VACCINATED
101	arun	tamil nadu	35	recovered	yes
104	divya	tamil nadu	31	active	yes
107	suresh	tamil nadu	48	active	no
101	arun	tamil nadu	35	recovered	yes

4 rows returned in 0.00 seconds CSV Export

Question: **b)** Display the names of patients whose **Status** is **Recovered**.

Input:

select patientname from covid19 where status='recovered';

Output:

Results Explain Describ

PATIENTNAME
arun
priya
divya
meena
priya
arun

6 rows returned in 0.03 sec

Question: **c)** Display **PatientName** and **Age** in ascending order of **Age**.

Input:

select patientname,age from covid19 order by age asc;

Output:

Results Explain Describe

PATIENTNAME	AGE
meena	24
karthik	28
divya	31
arun	35
arun	35
anitha	39
priya	42
priya	42
suresh	48
rahul	55

10 rows returned in 0.01 seco

Question: d) Display the names of patients who are **Vaccinated = 'Yes'**.

Input:

select patientname from covid19 where vaccinated='yes';

Output:

Results Explain De

PATIENTNAME
arun
priya
divya
meena
priya
arun
anitha

7 rows returned in 0.0

Question: e) Count the number of patients whose status is **Active**.

Input:

select count(*) from covid19 where status='active';
insert into covid19 values(108,'anitha','puducherry',39,'recovered','yes');

Output:

Results Explain De

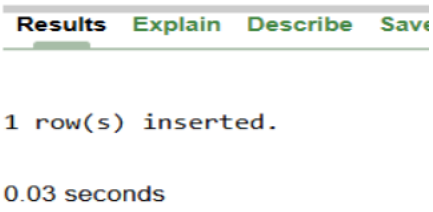
COUNT(*)
4

1 rows returned in 0.02

Question: f) Insert the following record into the **COVID19** table:

108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes'

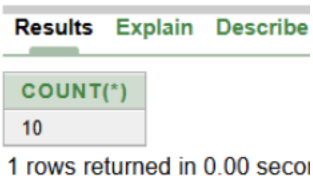
Output:



Question: (g) Write the output of the following SQL commands.

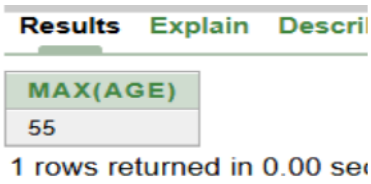
a. SELECT COUNT(*) FROM COVID19;

Output:



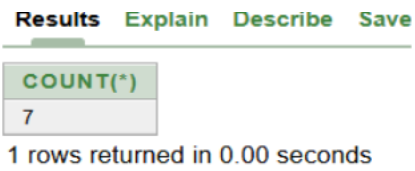
Question: b. SELECT MAX(Age) FROM COVID19;

Output:



Question: c. SELECT COUNT(*) FROM COVID19 WHERE Vaccinated='Yes';

Output:



Question: d. SELECT MIN(Age)FROM COVID19 WHERE Status='Recovered';

Output:

Results Explain Descril

MIN(AGE)
24

1 rows returned in 0.00 sec

17. Given the following HOUSEPRICE relation, write SQL commands for questions (a) to (f).

a) Display all houses located in **Chennai**.
b) Display the **Location** and **Price** of houses having **3 Bedrooms**.
c) Display **Location**, **AreaSqft**, and **Price** in descending order of **Price**.
d) Display the details of houses costing more than **₹70,00,000**.
e) Count the number of **Apartments**.
f) Insert the following record:
208, 'Vellore', 3, 1700, 7300000, 'Villa'
(g) Write the output of the following SQL commands.
(i) SELECT MAX(Price)FROM HOUSEPRICE;
(i) SELECT MIN(AreaSqft)FROM HOUSEPRICE;
(ii) SELECT AVG(Price)FROM HOUSEPRICE WHERE HouseType='Apartment';
(iii) SELECT COUNT(*)FROM HOUSEPRICE WHERE Bedrooms=3;

HouseID	Location	Bedrooms	AreaSqft	Price	HouseType
201	Chennai	2	1100	5200000	Apartment
202	Coimbatore	3	1450	6800000	Villa
203	Madurai	2	980	4100000	Apartment
204	Chennai	4	2200	9800000	Villa
205	Trichy	3	1650	7200000	Independent
206	Salem	2	1200	5000000	Apartment
207	Erode	3	1800	7600000	Independent

Syntax:

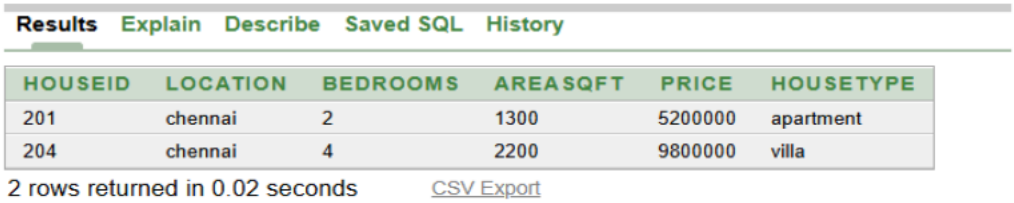
```
create table houseprice (houseid int, location varchar(50), bedrooms int, areasqft int,
price int, housetype varchar(50));
insert into houseprice values (201, 'chennai', 2, 1300, 5200000, 'apartment');
insert into houseprice values (202, 'coimbatore', 3, 1450, 6800000, 'villa');
insert into houseprice values (203, 'madurai', 2, 980, 4100000, 'apartment');
insert into houseprice values (204, 'chennai', 4, 2200, 9800000, 'villa');
insert into houseprice values (205, 'trichy', 3, 1650, 7200000, 'independent');
insert into houseprice values (206, 'salem', 2, 1200, 5000000, 'apartment');
insert into houseprice values (207, 'erode', 3, 1800, 7600000, 'independent');
```

Question: **a)** Display all houses located in **Chennai**.

Input:

```
select * from houseprice where location = 'chennai';
```

Output:



Results | Explain | Describe | Saved SQL | History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
201	chennai	2	1300	5200000	apartment
204	chennai	4	2200	9800000	villa

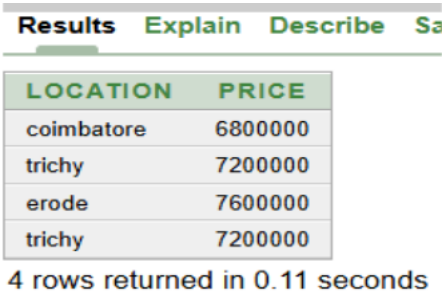
2 rows returned in 0.02 seconds CSV Export

Question: **b)** Display the **Location** and **Price** of houses having **3 Bedrooms**.

Input:

```
select location, price from houseprice where bedrooms = 3;
```

Output:



Results | Explain | Describe | Sa

LOCATION	PRICE
coimbatore	6800000
trichy	7200000
erode	7600000
trichy	7200000

4 rows returned in 0.11 seconds

Question: **c)** Display **Location**, **AreaSqft**, and **Price** in descending order of **Price**.

Input:

```
select location, areasqft, price from houseprice order by price desc;
```

Output:

Results Explain Describe Saved SQL

LOCATION	AREASQFT	PRICE
chennai	2200	9800000
erode	1800	7600000
trichy	1650	7200000
trichy	1650	7200000
coimbatore	1450	6800000
chennai	1300	5200000
salem	1200	5000000
madurai	980	4100000

8 rows returned in 0.01 seconds CSV

Question: d) Display the details of houses costing more than **₹70,00,000**.

Input:

```
select * from houseprice where price > 7000000;
```

Output:

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
204	chennai	4	2200	9800000	villa
205	trichy	3	1650	7200000	independent
207	erode	3	1800	7600000	independent
205	trichy	3	1650	7200000	independent

Question: e) Count the number of **Apartments**.

Input:

```
select count() from houseprice where housetype = 'apartment';
```

Output:

Results Explain Desc

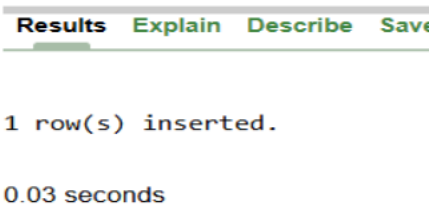
COUNT(*)
3

1 rows returned in 0.00 s

Question: f) Insert the following record:

208, 'Vellore', 3, 1700, 7300000, 'Villa'

Output:



Question: (g) Write the output of the following SQL commands.

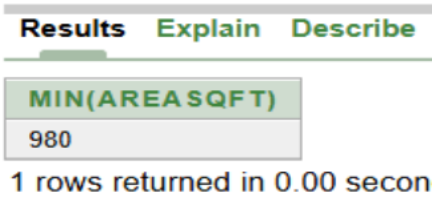
(i) SELECT MAX(Price)FROM HOUSEPRICE;

Output:



Question: (ii) SELECT MIN(AreaSqft)FROM HOUSEPRICE;

Output:



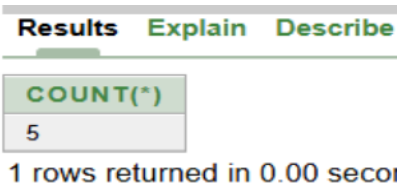
Question: (iii) SELECT AVG(Price)FROM HOUSEPRICE WHERE HouseType='Apartment';

Output:



Question: (iv) SELECT COUNT(*)FROM HOUSEPRICE WHERE Bedrooms=3;

Output:



18. Given the following CANCERDATA relation, write SQL commands for questions (a) to (f).

a) Display all patients diagnosed with **Breast Cancer**.
b) Display the names of patients whose **Stage** is **III**.
c) Display **PatientName** and **Age** in ascending order of **Age**.
d) Display the names of patients having **SurvivalYears greater than 5**.
e) Count the number of patients diagnosed with **Lung Cancer**.
f) Insert the following record:
308, 'Anand', 'Leukemia', 50, 'II', 7
(g) Write the output of the following SQL commands.
(i) SELECT COUNT(DISTINCT CancerType) FROM CANCERDATA;
(ii) SELECT MAX(SurvivalYears) FROM CANCERDATA;
(iii) SELECT AVG(Age) FROM CANCERDATA WHERE CancerType='Breast';
(iv) SELECT MIN(Age) FROM CANCERDATA WHERE Stage='II';

PatientID	PatientName	CancerType	Age	Stage	SurvivalYears
301	Kavya	Breast	45	II	8
302	Arun	Lung	60	III	4
303	Divya	Breast	38	I	10
304	Hari	Colon	52	II	6
305	Meena	Lung	57	IV	2
306	Ravi	Prostate	64	III	5
307	Priya	Breast	41	II	9

Syntax:

```
create table cancerdata (patientid int, patientname varchar(50), cancertype varchar(50), age int, stage varchar(10), survivalyears int);
insert into cancerdata values (301, 'kavya', 'breast', 45, 'II', 8);
insert into cancerdata values (302, 'arun', 'lung', 60, 'III', 4);
insert into cancerdata values (303, 'divya', 'breast', 38, 'I', 10);
insert into cancerdata values (304, 'hari', 'colon', 52, 'II', 6);
insert into cancerdata values (305, 'meena', 'lung', 57, 'IV', 2);
insert into cancerdata values (306, 'ravi', 'prostate', 64, 'III', 5);
insert into cancerdata values (307, 'priya', 'breast', 41, 'II', 9);
```

Question: **a)** Display all patients diagnosed with **Breast Cancer**.

Input:

```
select * from cancerdata where cancertype = 'breast';
```

Output:

Results Explain Describe Saved SQL History

PATIENTID	PATIENTNAME	CANCERTYPE	AGE	STAGE	SURVIVALYEARS
301	kavya	breast	45	II	8
303	divya	breast	38	I	10
307	priya	breast	41	II	9
307	priya	breast	41	II	9

4 rows returned in 0.00 seconds CSV Export

Question: b) Display the names of patients whose **Stage** is **III**.

Input:

select patientname from cancerdata where stage = 'III';

Output:

Results Explain Desc

PATIENTNAME
arun
ravi

2 rows returned in 0.08 s

Question: c) Display **PatientName** and **Age** in ascending order of **Age**.

Input:

select patientname, age from cancerdata order by age asc;

Output:

Results Explain Describe

PATIENTNAME	AGE
divya	38
priya	41
priya	41
kavya	45
hari	52
meena	57
arun	60
ravi	64

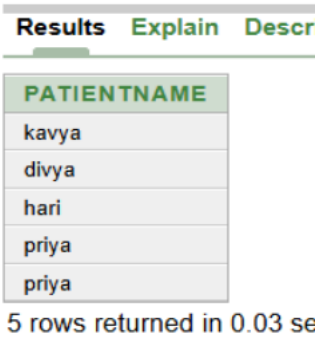
8 rows returned in 0.07 second

Question: d) Display the names of patients having **SurvivalYears greater than 5**.

Input:

select patientname from cancerdata where survivalyears > 5;

Output:



Results Explain Descr

PATIENTNAME
kavya
divya
hari
priya
priya

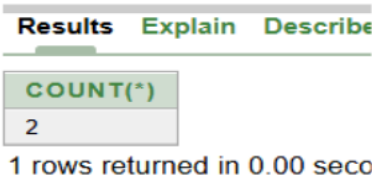
5 rows returned in 0.03 se

Question: e) Count the number of patients diagnosed with **Lung Cancer**.

Input:

select count(*) from cancerdata where cancertype = 'lung';

Output:



Results Explain Describe

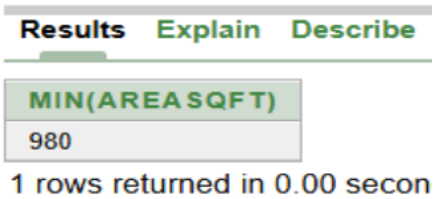
COUNT(*)
2

1 rows returned in 0.00 seco

Question: f) Insert the following record:

308, 'Anand', 'Leukemia', 50, 'II', 7

Output:



Results Explain Describe

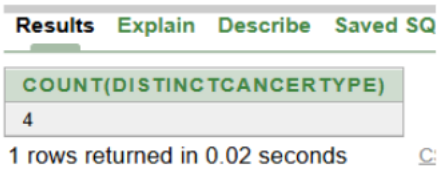
MIN(AREASQFT)
980

1 rows returned in 0.00 secon

Question: (g) Write the output of the following SQL commands.

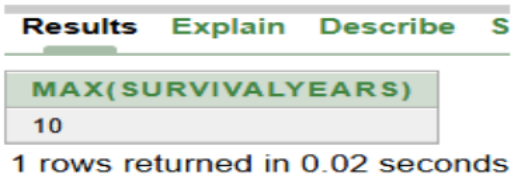
(i) SELECT COUNT(DISTINCT CancerType) FROM CANCERDATA;

Output:



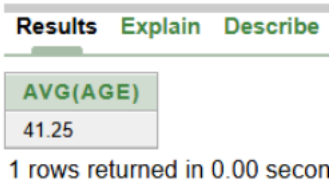
Question: (ii) SELECT MAX(SsurvivalYears) FROM CANCERDATA;

Output:



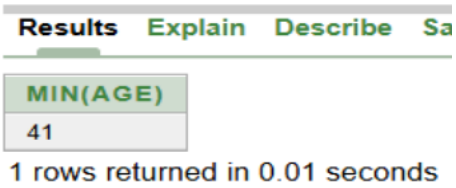
Question: (iii) SELECT AVG(Age) FROM CANCERDATA WHERE CancerType='Breast';

Output:



Question: (iv) SELECT MIN(Age) FROM CANCERDATA WHERE Stage='II';

Output:



19. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables ITEMS and TRADERS

(a) To display the details of all the items in ascending order of item names (i.e. INAME).
(b) To display item name and price of all those items, whose price is in the range of 10000 and 22000 (both values inclusive).

(c) To display the number of items, which are traded by each trader. The expected output of this query should be

T01 2

T02 2

T03 1

(d) To display the price, item name (i.e. INAME) and quantity (i.e. QTY) of those items which have quantity more than 150.

(e) To display the names of those traders, who are either from DELHI or from MUMBAI.

(f) To display the name of the companies and the bane of the items in descending order of company names.

(g) Obtain the outputs of the following SQL queries based on the data given in tables ITEMS and TRADERS.

(g1) SELECT MAX (PRICE), MIN (PRICE) FROM ITEMS;

(g2) SELECT PRICE * QTY AMOUNT FROM ITEMS WHERE CODE = 1004;

(g3) SELE CT DISTINCT TCODE FROM ITEMS;

(g4) SELECT INAME, TNAME FROM ITEMS I, TRASERS T WHERE I, TCODE AND QTY<100;

ITEMS

CODE	INAME	QTY	PRICE	COMPANY	TCODE
1001	DIGITAL PAD12i	120	11000	XENITA	T01
1006	LED SCREEN 40	70	38000	SANTORA	T02
1004	CAR GPS SYSTEM	50	21500	GEOKNOW	T01
1003	DIGITAL CAMERA 12X	160	8000	DIGICLICK	T02
1005	PEN DRIVE 32 GB	600	1200	STOREHOME	T03

TRADERS

TCODE	TNAME	CITY
T01	ELECTRONIC SALES	MUMBAI
T03	BUSY STORE CORP	DELHI
T02	DISP HOUSE INC	CHENNAI

Syntax:

```
create table items (code int, iname varchar(50), qty int, price int, company varchar(50), tcode varchar(10));
insert into items values (1001, 'digital pad12i', 120, 11000, 'xenita', 't01');
insert into items values (1006, 'led screen 40', 70, 38000, 'santora', 't02');
insert into items values (1004, 'car gps system', 50, 21500, 'geoknow', 't01');
insert into items values (1003, 'digital camera 12x', 160, 8000, 'digiclick', 't02');
insert into items values (1005, 'pen drive 32 gb', 600, 1200, 'storehome', 't03');
create table traders (tcode varchar(10), tname varchar(50), city varchar(50));
insert into traders values ('t01', 'electronic sales', 'mumbai');
insert into traders values ('t03', 'busy store corp', 'delhi');
insert into traders values ('t02', 'disp house inc', 'chennai');
```

Question: (a) To display the details of all the items in ascending order of item names (i.e. INAME).
desc;

Input:

select * from items order by iname asc;

Output:

Results Explain Describe Saved SQL History

CODE	INAME	QTY	PRICE	COMPANY	TCODE
1004	car gps system	50	21500	geoknow	t01
1003	digital camera 12x	160	8000	digiclick	t02
1001	digital pad12i	120	11000	xenita	t01
1006	led screen 40	70	38000	santora	t02
1005	pen drive 32 gb	600	1200	storehome	t03

5 rows returned in 0.10 seconds CSV Export

Question: (b) To display item name and price of all those items, whose price is in the range of 10000 and 22000 (both values inclusive).

Input:

select iname, price from items where price between 10000 and 22000;

Output:

Results Explain Describe

INAME	PRICE
digital pad12i	11000
car gps system	21500

2 rows returned in 0.03 second

Question: (c) To display the number of items, which are traded by each trader. The expected output of this query should be
T01 2
T02 2
T03 1

Input:

select tcode, count(*) from items group by tcode;

Output:

Results Explain Describe

TCODE	COUNT(*)
t01	2
t02	2
t03	1

3 rows returned in 0.02 seconc

Question: (d) To display the price, item name (i.e. INAME) and quantity (i.e. QTY) of those items which have quantity more than 150.

Input:

select price, iname, qty from items where qty > 150;

Output:

Results Explain Describe Saved S

PRICE	INAME	QTY
8000	digital camera 12x	160
1200	pen drive 32 gb	600

2 rows returned in 0.00 seconds

Question: (e) To display the names of those traders, who are either from DELHI or from MUMBAI.

Input:

select tname from traders where city in ('delhi', 'mumbai');

Output:

Results Explain D

TNAME
electronic sales
busy store corp

2 rows returned in 0.(

Question: (f) To display the name of the companies and the bane of the items in descending order of company names.

Input:

select company, iname from items order by company;

Output:

Results Explain Describe Sa

COMPANY	INAME
xenita	digital pad12i
storehome	pen drive 32 gb
santora	led screen 40
geoknow	car gps system
digiclick	digital camera 12x

5 rows returned in 0.00 seconds

Question: (g) Obtain the outputs of the following SQL queries based on the data given in tables ITEMS and TRADERS.

(g1) SELECT MAX (PRICE), MIN (PRICE) FROM ITEMS;

Output:

Results Explain Describe Save

MAX(PRICE)	MIN(PRICE)
38000	1200

1 rows returned in 0.00 seconds

Question: (g2) SELECT PRICE * QTY AMOUNT FROM ITEMS WHERE CODE = 1004;

Output:

Results Explain

AMOUNT
1075000

1 rows returned in (

Question: (g3) SELE CT DISTINCT TCODE FROM ITEMS;

Output:

Results Explain Describe

TCODE
t01
t02
t03

3 rows returned in 0.00 secon

Question: (g4) SELECT INAME, TNAME FROM ITEMS I, TRASERS T WHERE I, TCODE AND QTY<100;

Output:

Results Explain Describe Sav

INAME	TNAME
car gps system	electronic sales
led screen 40	disp house inc

2 rows returned in 0.03 seconds

Ex. No: 15 **Mini Project**

DATE:04/10/2026 **Student Academic Performance Analysis**

Aim

To develop a Student Performance Analysis System using SQL and SQLite to store, manage, analyze, and evaluate student academic performance based on marks, study time, attendance, and performance trends.

Software Used

- Github Codespaces
- SQLite
- Oracle Database
- SQL
- PL/SQL

Description

The Student Performance Analysis System is developed to manage and analyze student academic records. The system stores student details, school information, study time, absences, and examination grades.

SQL queries are used to retrieve and analyze student performance data, while PL/SQL functions and procedures are used to classify students, generate reports, and evaluate academic progress. The system helps identify high-performing students and students who require academic support.

Database Tables

1. STUDENT_PERFORMANCE

Column Name	Data Type	Constraint
STUDENT_ID	NUMBER	PRIMARY KEY
STUDENT_NAME	VARCHAR2(30)	NOT NULL
SCHOOL	VARCHAR2(30)	NOT NULL
AGE	NUMBER	NOT NULL
STUDY_TIME	NUMBER	NOT NULL
ABSENCES	NUMBER	NOT NULL
G1	NUMBER	NOT NULL
G2	NUMBER	NOT NULL
G3	NUMBER	NOT NULL

Dataset: UCI Student Performance

Table: STUDENT_PERFORMANCE

Problem statement: Develop a student performance analysis system to evaluate academic marks, studytime, absences, and performance trends.

SQL queries:

1.Display student records with school, age, study time, absences, and grades.

2.Calculate the average final grade for each school.

3.Identify students who scored at least 15 in the final examination.

4.Display students with more than 10 absences.

5.Calculate the difference between first-period and final grades for each student.

PL/SQL tasks:

1.Create a function to classify students based on final examination marks.

2.Create a procedure to display the academic performance of a student using student ID.

3.Create a procedure to generate a school-wise performance report and identify students requiring academic support.

TABLE: STUDENT_PERFORMANCE

INPUT: create table student_performance(
student_id number primary key,
student_name varchar2(50),
school varchar2(50),
age number,
study_time number,
absences number,
g1 number,
g2 number,
g3 number
);

OUTPUT:

Table STUDENT_PERFORMANCE created.

INSERT VALUES 25

```
insert into student_performance values(101,'Arun','ABC School',15,2,4,12,14,15);
insert into student_performance values(102,'Priya','ABC School',16,3,2,15,16,18);
insert into student_performance values(103,'Rahul','XYZ School',15,1,12,8,10,11);
insert into student_performance values(104,'Divya','XYZ School',17,4,1,17,18,19);
insert into student_performance values(105,'Kiran','ABC School',16,2,15,10,12,13);
insert into student_performance values(106,'Meena','XYZ School',15,3,5,14,15,16);
insert into student_performance values(107,'Vijay','ABC School',17,4,0,18,19,20);
insert into student_performance values(108,'Anu','XYZ School',16,2,8,13,14,15);
insert into student_performance values(109,'Ravi','ABC School',15,1,20,7,8,9);
insert into student_performance values(110,'Sita','XYZ School',17,3,3,16,17,18);
```

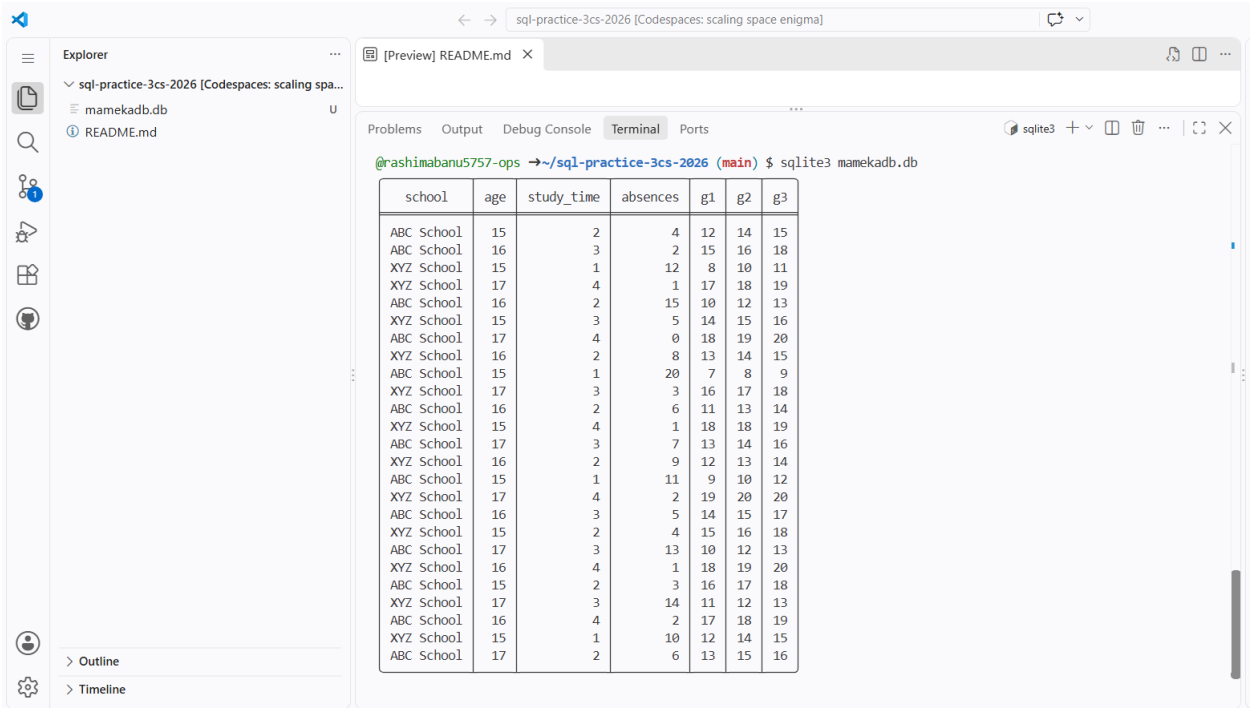
```
insert into student_performance values(111,'Kumar','ABC School',16,2,6,11,13,14);
insert into student_performance values(112,'Deepa','XYZ School',15,4,1,18,18,19);
insert into student_performance values(113,'Hari','ABC School',17,3,7,13,14,16);
insert into student_performance values(114,'Latha','XYZ School',16,2,9,12,13,14);
insert into student_performance values(115,'Manoj','ABC School',15,1,11,9,10,12);
insert into student_performance values(116,'Nisha','XYZ School',17,4,2,19,20,20);
insert into student_performance values(117,'Prakash','ABC School',16,3,5,14,15,17);
insert into student_performance values(118,'Rekha','XYZ School',15,2,4,15,16,18);
insert into student_performance values(119,'Saravanan','ABC School',17,3,13,10,12,13);
insert into student_performance values(120,'Uma','XYZ School',16,4,1,18,19,20);
insert into student_performance values(121,'Vani','ABC School',15,2,3,16,17,18);
insert into student_performance values(122,'Yogesh','XYZ School',17,3,14,11,12,13);
insert into student_performance values(123,'Ajith','ABC School',16,4,2,17,18,19);
insert into student_performance values(124,'Bhavya','XYZ School',15,1,10,12,14,15);
insert into student_performance values(125,'Charan','ABC School',17,2,6,13,15,16);
```

SQL QUERIES

QUESTION: 1. Display student records with school, age, study time, absences, and grades :

SQL> select school,age,study_time,absences,g1,g2,g3 from student_performance;

OUTPUT:



school	age	study_time	absences	g1	g2	g3
ABC School	15	2	4	12	14	15
ABC School	16	3	2	15	16	18
XYZ School	15	1	12	8	10	11
XYZ School	17	4	1	17	18	19
ABC School	16	2	15	10	12	13
XYZ School	15	3	5	14	15	16
ABC School	17	4	0	18	19	20
XYZ School	16	2	8	13	14	15
ABC School	15	1	20	7	8	9
XYZ School	17	3	3	16	17	18
ABC School	16	2	6	11	13	14
XYZ School	15	4	1	18	18	19
ABC School	17	3	7	13	14	16
XYZ School	16	2	9	12	13	14
ABC School	15	1	11	9	10	12
XYZ School	17	4	2	19	20	20
ABC School	16	3	5	14	15	17
XYZ School	15	2	4	15	16	18
ABC School	17	3	13	10	12	13
XYZ School	16	4	1	18	19	20
ABC School	15	2	3	16	17	18
XYZ School	17	3	14	11	12	13
ABC School	16	4	2	17	18	19
XYZ School	15	1	10	12	14	15
ABC School	17	2	6	13	15	16

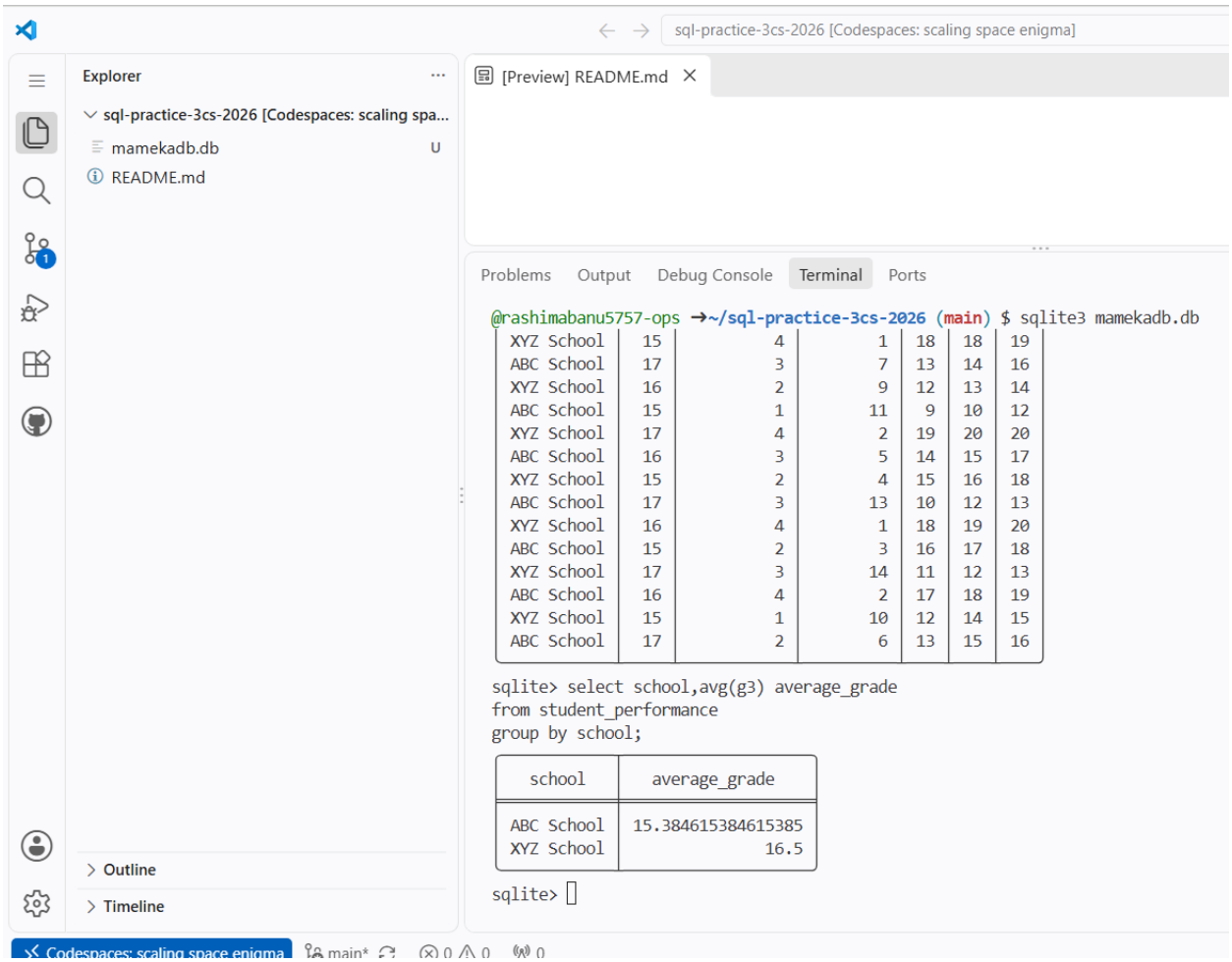
QUESTION: 2. Calculate the average final grade for each school

SQL> select school,avg(g3) average_grade

from student_performance

group by school;

OUTPUT:



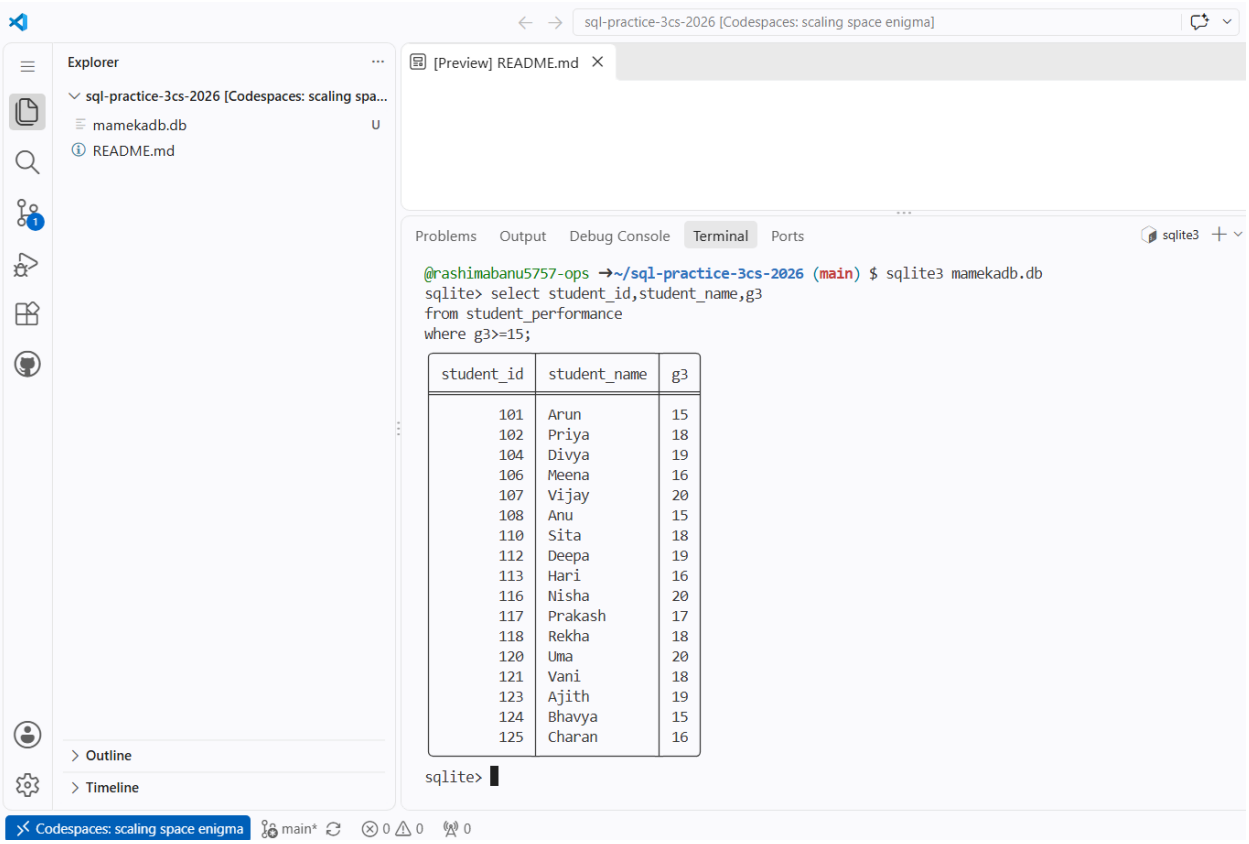
QUESTION: 3. Identify students who scored at least 15 in the final examination

SQL> select student_id,student_name,g3

from student_performance

where g3>=15;

OUTPUT:



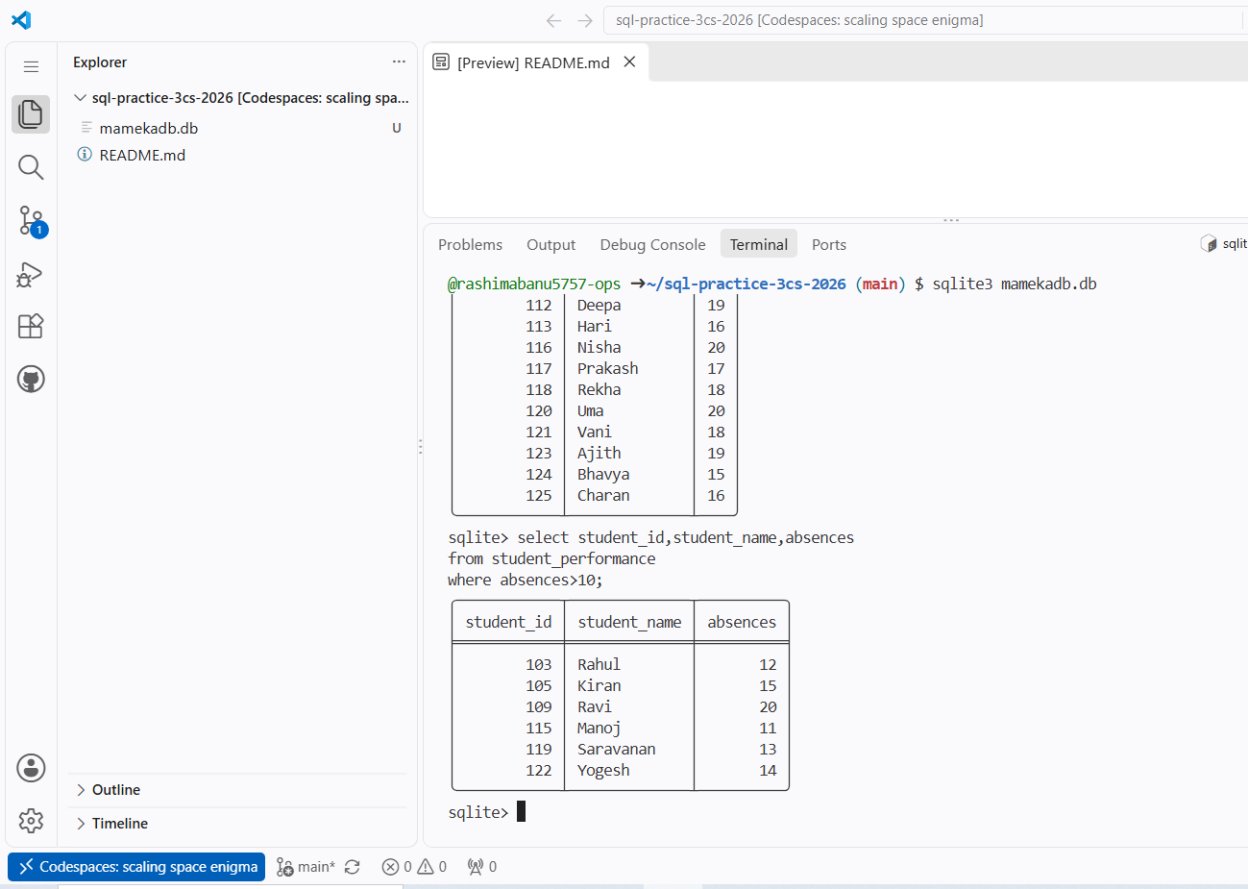
QUESTION: 4. Display students with more than 10 absences

SQL> select student_id,student_name,absences

from student_performance

where absences>10;

OUTPUT:

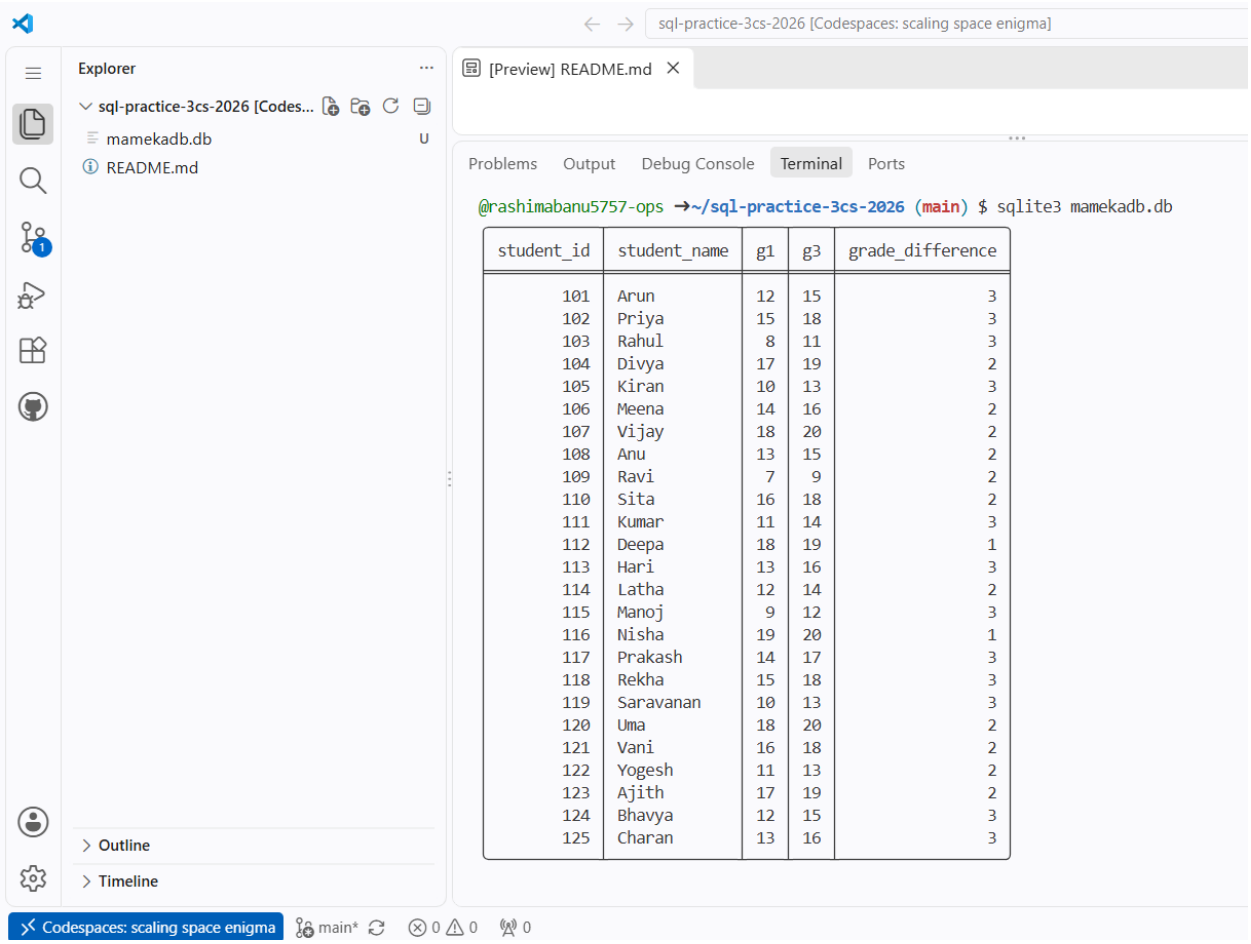


QUESTION: 5. Calculate the difference between first-period and final grades for each student

SQL> select student_id,student_name,g1,g3,(g3-g1) grade_difference

from student_performance;

OUTPUT:



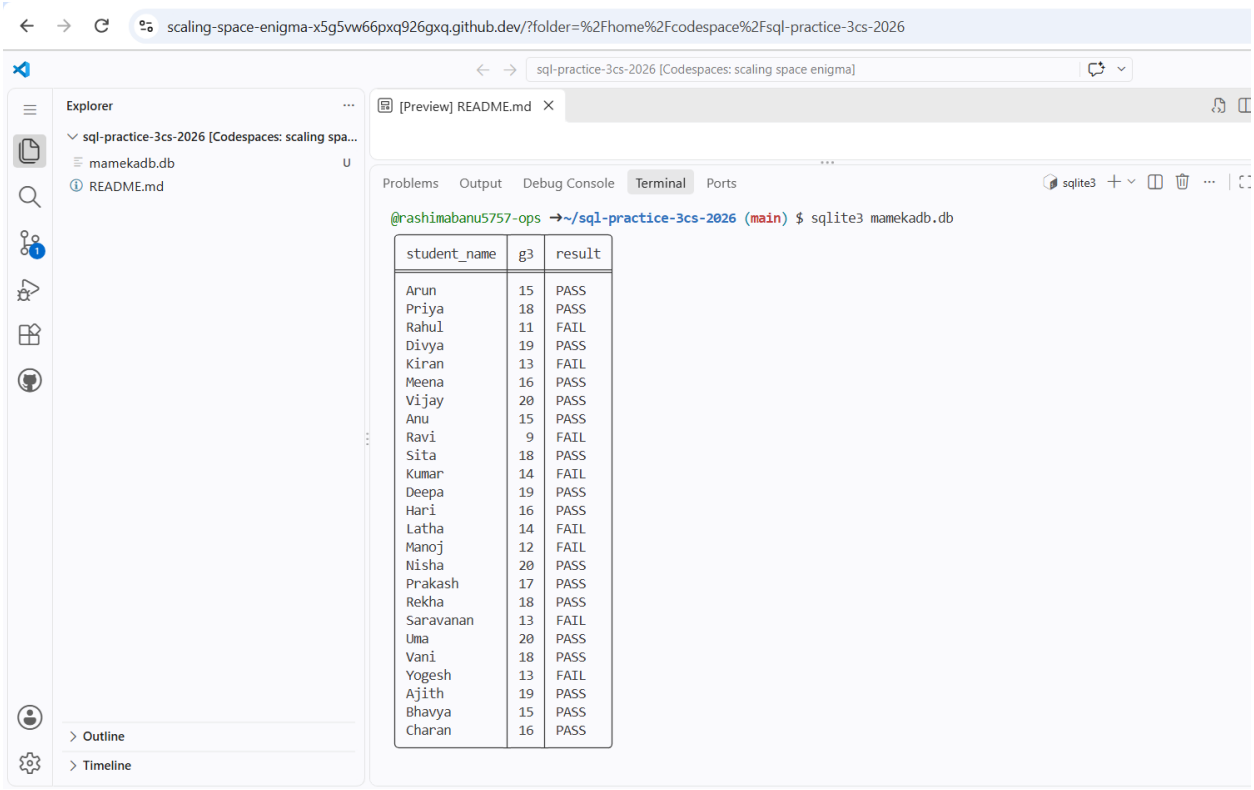
student_id	student_name	g1	g3	grade_difference
101	Arun	12	15	3
102	Priya	15	18	3
103	Rahul	8	11	3
104	Divya	17	19	2
105	Kiran	10	13	3
106	Meena	14	16	2
107	Vijay	18	20	2
108	Anu	13	15	2
109	Ravi	7	9	2
110	Sita	16	18	2
111	Kumar	11	14	3
112	Deepa	18	19	1
113	Hari	13	16	3
114	Latha	12	14	2
115	Manoj	9	12	3
116	Nisha	19	20	1
117	Prakash	14	17	3
118	Rekha	15	18	3
119	Saravanan	10	13	3
120	Uma	18	20	2
121	Vani	16	18	2
122	Yogesh	11	13	2
123	Ajith	17	19	2
124	Bhavya	12	15	3
125	Charan	13	16	3

PL/SQL TASKS

QUESTION: 1. Function to classify students based on final examination marks

```
SQL> select student_name,
       g3,
       case
          when g3 >= 15 then 'PASS'
          else 'FAIL'
       end as result
from student_performance;
```

OUTPUT:



student_name	g3	result
Arun	15	PASS
Priya	18	PASS
Rahul	11	FAIL
Divya	19	PASS
Kiran	13	FAIL
Meena	16	PASS
Vijay	20	PASS
Anu	15	PASS
Ravi	9	FAIL
Sita	18	PASS
Kumar	14	FAIL
Deepa	19	PASS
Hari	16	PASS
Latha	14	FAIL
Manoj	12	FAIL
Nisha	20	PASS
Prakash	17	PASS
Rekha	18	PASS
Saravanan	13	FAIL
Uma	20	PASS
Vani	18	PASS
Yogesh	13	FAIL
Ajith	19	PASS
Bhavya	15	PASS
Charan	16	PASS

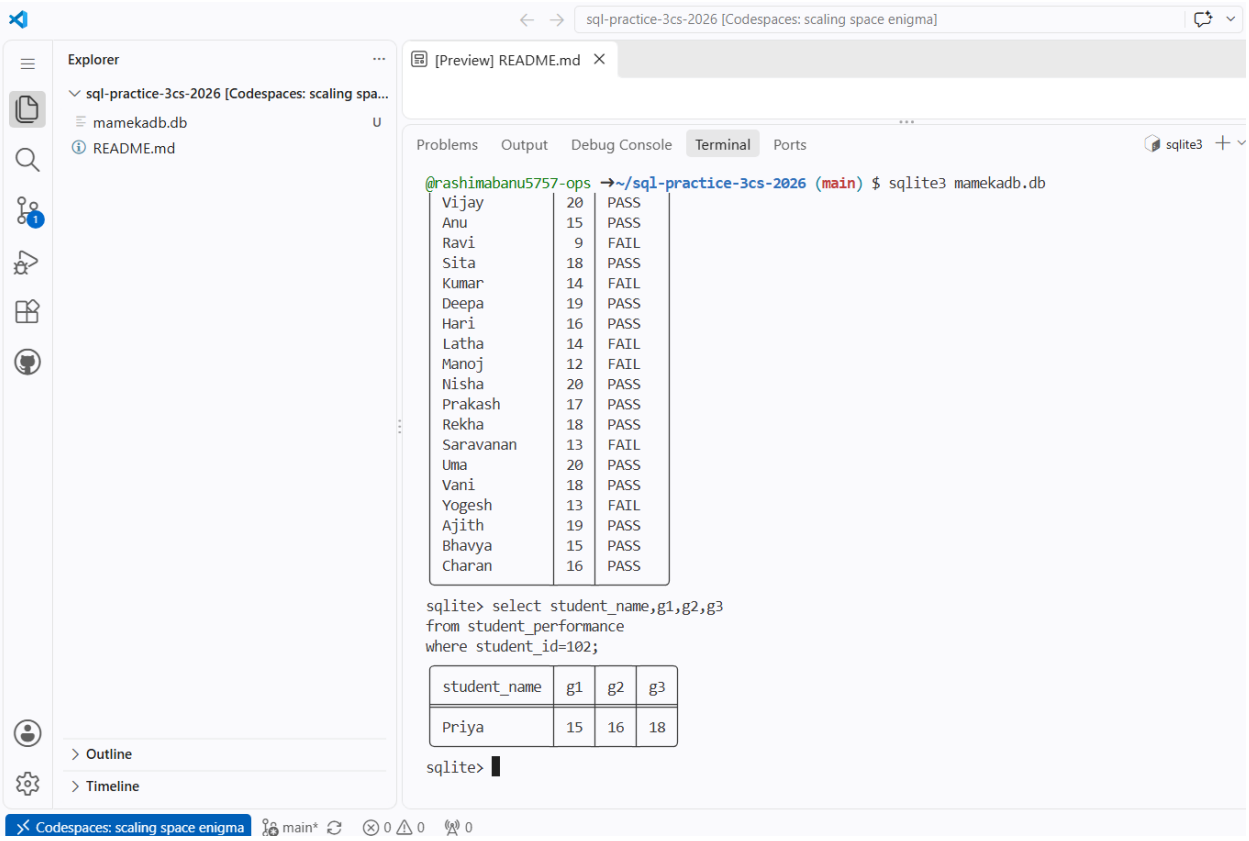
QUESTION: 2. Procedure to display academic performance using student ID

SQL> select student_name,g1,g2,g3

from student_performance

where student_id=102;

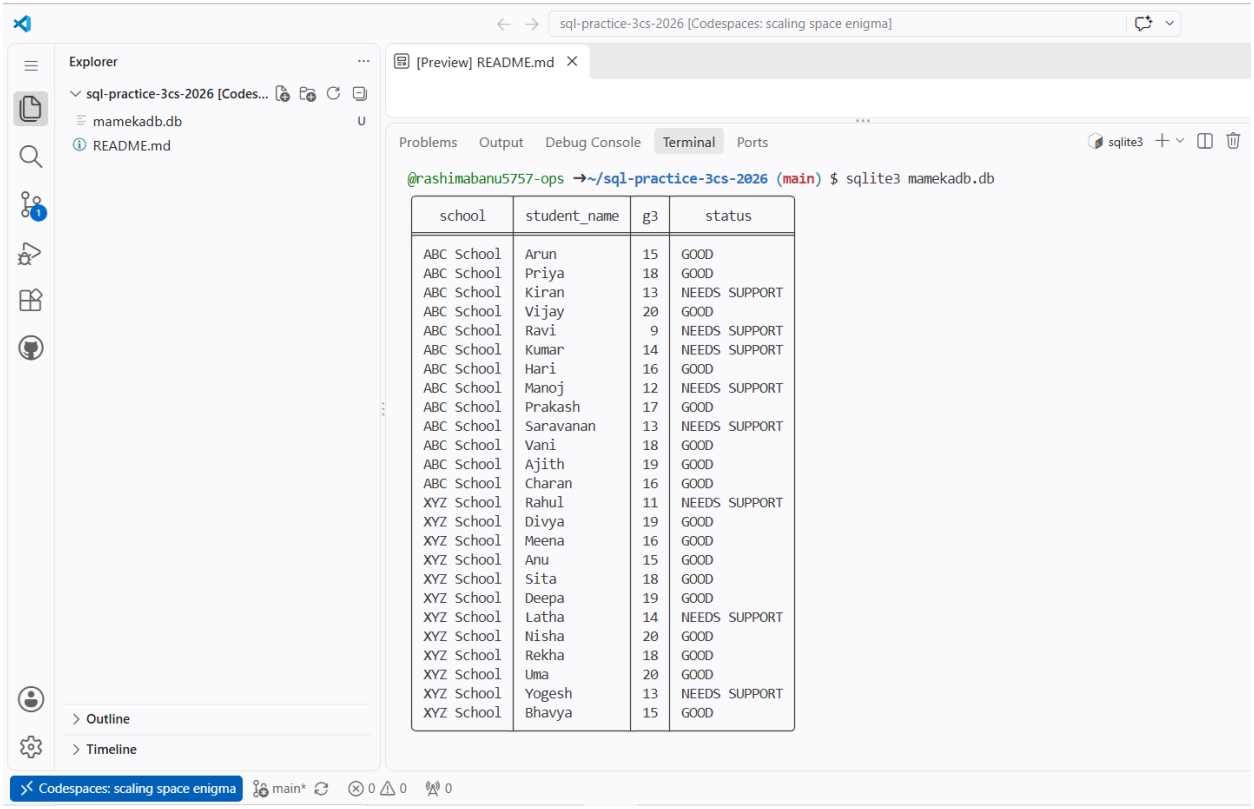
OUTPUT:



QUESTION: 3. Procedure to generate a school-wise performance report and identify students requiring academic support

```
SQL>  select school,
      student_name,
      g3,
      case
         when g3 < 15 then 'NEEDS SUPPORT'
         else 'GOOD'
      end as status
from student_performance
order by school;
```

OUTPUT:



school	student_name	g3	status
ABC School	Arun	15	GOOD
ABC School	Priya	18	GOOD
ABC School	Kiran	13	NEEDS SUPPORT
ABC School	Vijay	20	GOOD
ABC School	Ravi	9	NEEDS SUPPORT
ABC School	Kumar	14	NEEDS SUPPORT
ABC School	Hari	16	GOOD
ABC School	Manoj	12	NEEDS SUPPORT
ABC School	Prakash	17	GOOD
ABC School	Saravanan	13	NEEDS SUPPORT
ABC School	Vani	18	GOOD
ABC School	Ajith	19	GOOD
ABC School	Charan	16	GOOD
XYZ School	Rahul	11	NEEDS SUPPORT
XYZ School	Divya	19	GOOD
XYZ School	Meena	16	GOOD
XYZ School	Anu	15	GOOD
XYZ School	Sita	18	GOOD
XYZ School	Deepa	19	GOOD
XYZ School	Latha	14	NEEDS SUPPORT
XYZ School	Nisha	20	GOOD
XYZ School	Rekha	18	GOOD
XYZ School	Uma	20	GOOD
XYZ School	Yogesh	13	NEEDS SUPPORT
XYZ School	Bhavya	15	GOOD

Exception Handling

Test Case 1 – Student Not Found

SQL> begin
student_report(999);
exception
when no_data_found then
dbms_output.put_line('Student ID not found');
when others then
dbms_output.put_line('Unexpected error occurred');
end;
/

OUTPUT:

status
STUDENT ID NOT FOUND

Test Case 2 – Invalid Final Grade

SQL> select student_name,
g3,
case
when g3 is null then 'ERROR'
when g3>=15 then 'PASS'
else 'FAIL'
end as result
from student_performance;

OUTPUT:

```
sqlite> select student_name,
       g3,
       case
           when g3 is null then 'ERROR'
           when g3>=15 then 'PASS'
           else 'FAIL'
       end as result
from student_performance
where student_id=126;
```

student_name	g3	result
Test Student	NULL	ERROR

```
sqlite>
```

Test Case 3 – School-wise Report with Academic Support

SQL> select case
when exists (select 1 from student_performance)
then 'School report generated successfully'
else 'Error while generating school report'
end as message;

OUTPUT:

```
sqlite> select case
when exists (select 1 from student_performance)
then 'School report generated successfully'
else 'Error while generating school report'
end as message;
```

message
School report generated successfully

```
sqlite>
```

Dataset Insights

- The dataset contains **25 student performance records**.
- The records contain student name, school, age, study time, absences, and examination grades.
- Final examination performance is represented by **G3**.
- Students scoring **15 or above** are considered to have good performance.
- Students scoring **below 15** are identified as requiring **academic support**.
- Invalid student IDs are handled using the `NO_DATA_FOUND` exception.
- Other unexpected errors are handled using the `WHEN OTHERS` exception.
- School-wise analysis helps identify students who require additional academic support.

Result

Thus, the **Student Performance Analysis System** was successfully implemented using **SQL and PL/SQL**. The required student records were stored and analyzed using SQL queries. PL/SQL functions and procedures were used to classify student performance, display academic details, and generate school-wise reports. **Exception handling was implemented successfully to handle invalid student IDs, missing data, and unexpected errors.**

Ex. No: 16 **Khan Academy**

DATE:05/09/2026

Challenge: Book list database

Step 1

What are your favorite books? You can make a database table to store them in! In this first step, create a table to store your list of books. It should have columns for `id`, `name`, and `rating`.

Solution:

CREATE TABLE books (

id INTEGER PRIMARY KEY,

name TEXT,

rating INTEGER



Step 2

Now, add three of your favorite books into the table.

Solution:

INSERT INTO books VALUES (1, "The Hobbit", 5);

INSERT INTO books VALUES (2, "1984", 5);

INSERT INTO books VALUES (3, "To Kill a Mockingbird", 4);



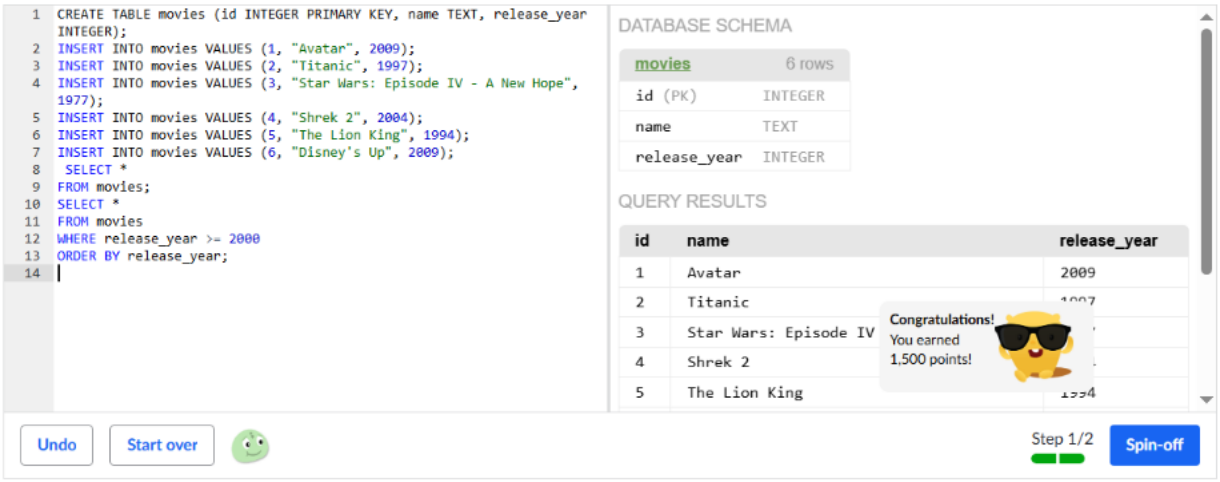
Challenge: Box office hits database

Step 1

This database contains an incomplete list of box office hits and their release year. In this challenge, you're going to get the results back out of the database in different ways! In this first step, just select all the movies.

Solution:

```
CREATE TABLE movies (id INTEGER PRIMARY KEY, name TEXT, release_year INTEGER);
INSERT INTO movies VALUES (1, "Avatar", 2009);
INSERT INTO movies VALUES (2, "Titanic", 1997);
INSERT INTO movies VALUES (3, "Star Wars: Episode IV - A New Hope", 1977);
INSERT INTO movies VALUES (4, "Shrek 2", 2004);
INSERT INTO movies VALUES (5, "The Lion King", 1994);
INSERT INTO movies VALUES (6, "Disney's Up", 2009);
SELECT *
FROM movies;
```



Step 2

Now, add a second query after the first, that retrieves only the movies that were released in the year 2000 or later, not before. Sort the results so that the earlier movies are listed first. You should have 2 `SELECT` statements after this step.

Solution:

```
SELECT *

FROM movies;

SELECT *

FROM movies

WHERE release_year >= 2000

ORDER BY release_year;
```



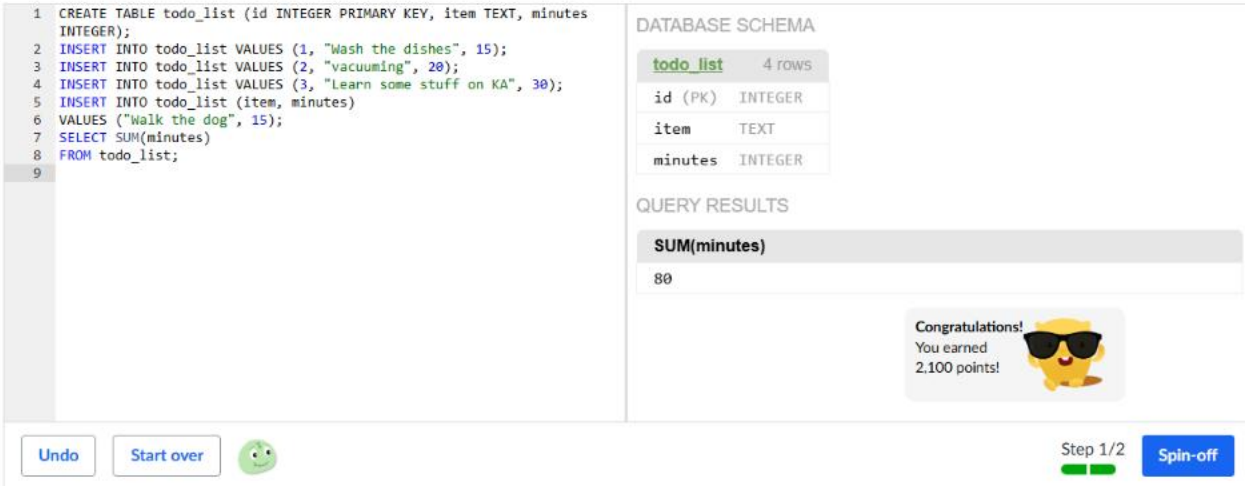
Challenge: TODO list database stats

Step 1

Here's a table containing a TODO list with the number of minutes it will take to complete each item. Insert another item to your todo list with the estimated minutes it will take.

Solution:

```
CREATE TABLE todo_list (id INTEGER PRIMARY KEY, item TEXT, minutes INTEGER);
INSERT INTO todo_list VALUES (1, "Wash the dishes", 15);
INSERT INTO todo_list VALUES (2, "vacuuming", 20);
INSERT INTO todo_list VALUES (3, "Learn some stuff on KA", 30);
INSERT INTO todo_list (item, minutes)
VALUES ("Walk the dog", 15);
```



Step 2

Select the `SUM` of minutes it will take to do all of the items on your TODO list. You should only have one `SELECT` statement.

Solution:

SELECT SUM(minutes)

FROM todo_list;



Project: Design a store database

Question:

Create your own store! Your store should sell one type of things, like clothing or bikes, whatever you want your store to specialize in. You should have a table for all the items in your store, and at least 5 columns for the kind of data you think you'd need to store. You should sell at least 15 items, and use select statements to order your items by price and show at least one statistic about the items.

Solution:

```
CREATE TABLE store (
    id INTEGER PRIMARY KEY,
    name TEXT,
    price INTEGER,
    category TEXT,
    quantity INTEGER
);

INSERT INTO store VALUES (1, "Pen", 2, "Stationery", 50);
INSERT INTO store VALUES (2, "Pencil", 1, "Stationery", 100);
INSERT INTO store VALUES (3, "Notebook", 5, "Stationery", 30);
INSERT INTO store VALUES (4, "Eraser", 1, "Stationery", 40);
INSERT INTO store VALUES (5, "Marker", 3, "Stationery", 25);
INSERT INTO store VALUES (6, "Ruler", 2, "Stationery", 20);
INSERT INTO store VALUES (7, "Glue", 4, "Stationery", 15);
INSERT INTO store VALUES (8, "Scissors", 6, "Stationery", 10);
INSERT INTO store VALUES (9, "Folder", 3, "Stationery", 18);
INSERT INTO store VALUES (10, "Highlighter", 2, "Stationery", 22);
INSERT INTO store VALUES (11, "Crayons", 7, "Stationery", 12);
INSERT INTO store VALUES (12, "Tape", 3, "Stationery", 16);
INSERT INTO store VALUES (13, "Stapler", 8, "Stationery", 8);
```

INSERT INTO store VALUES (14, "Paper Pack", 10, "Stationery", 14);

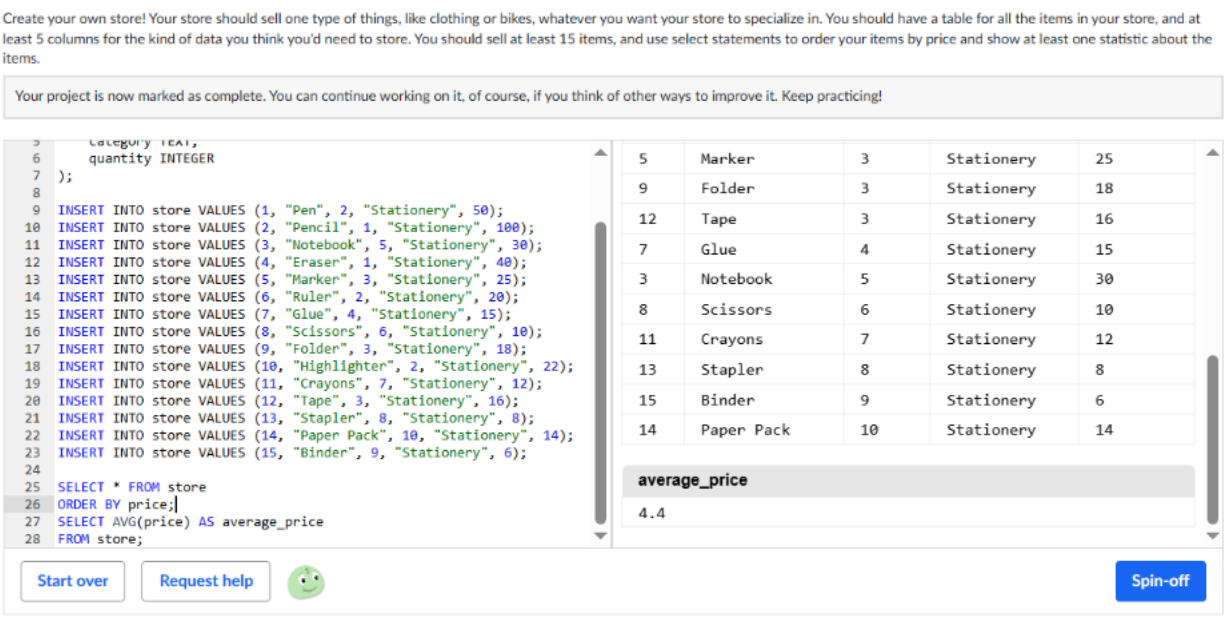
INSERT INTO store VALUES (15, "Binder", 9, "Stationery", 6);

SELECT * FROM store

ORDER BY price;

SELECT AVG(price) AS average_price

FROM store;



Challenge: Karaoke song selector

Step 1

Ever sung karaoke? It's a place where you sing songs with your friends, and it's a lot of fun. We've created a table with songs, and in this challenge, you'll use queries to decide what songs to sing. For the first step, select all the song titles.

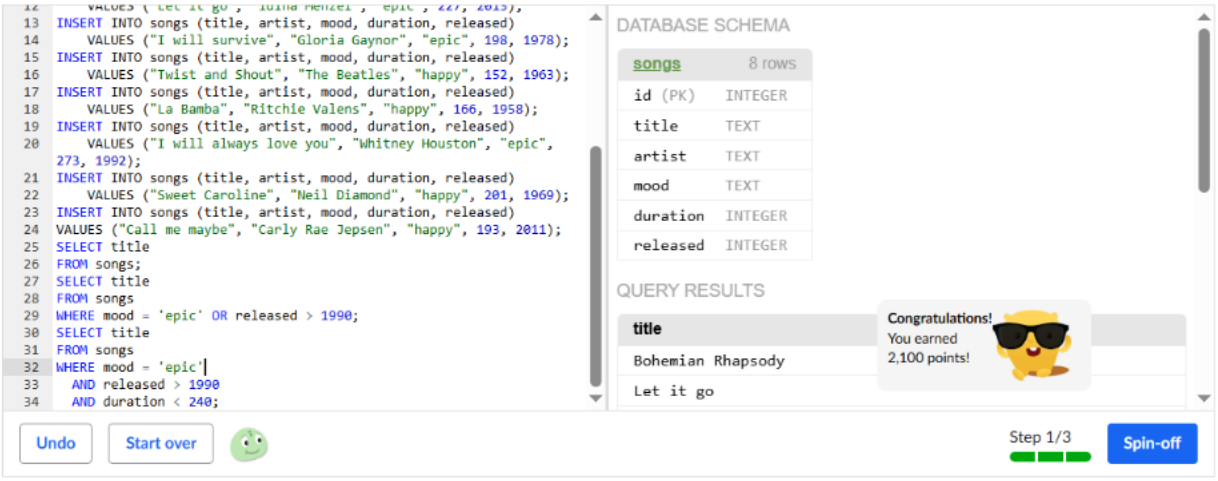
CREATE TABLE songs (

id INTEGER PRIMARY KEY,

title TEXT,

artist TEXT,

```
    mood TEXT,
    duration INTEGER,
    released INTEGER);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Bohemian Rhapsody", "Queen", "epic", 60, 1975);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Let it go", "Idina Menzel", "epic", 227, 2013);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will survive", "Gloria Gaynor", "epic", 198, 1978);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Twist and Shout", "The Beatles", "happy", 152, 1963);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("La Bamba", "Ritchie Valens", "happy", 166, 1958);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will always love you", "Whitney Houston", "epic", 273, 1992);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Sweet Caroline", "Neil Diamond", "happy", 201, 1969);
INSERT INTO songs (title, artist, mood, duration, released)
VALUES ("Call me maybe", "Carly Rae Jepsen", "happy", 193, 2011);
SELECT title
FROM songs;
```



Step 2

Maybe your friends only like singing either recent songs or truly epic songs. Add another `SELECT` that uses `OR` to show the titles of the songs that have an 'epic' mood or a release date after 1990.

(Tip: If you're not sure how to select rows where a column equals a value, remember that you can check the documentation below.)

Solution:

```
SELECT title

FROM songs

WHERE mood = 'epic' OR released > 1990;

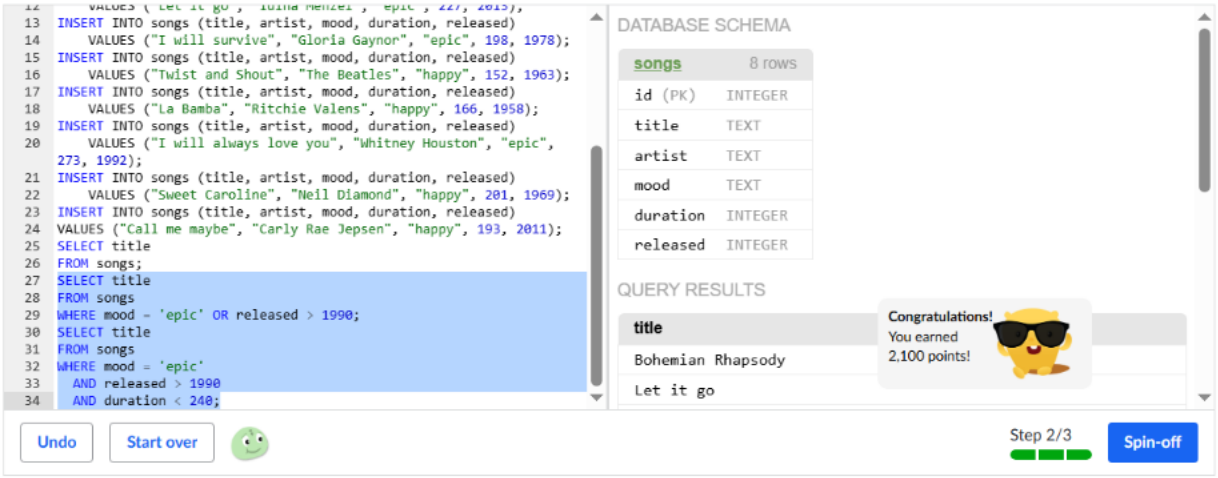
SELECT title

FROM songs

WHERE mood = 'epic'

  AND released > 1990

  AND duration < 240;
```



Step 3

People get picky at the end of the night. Add another `SELECT` that uses `AND` to show the titles of songs that are 'epic', **and** released after 1990, **and** less than 4 minutes long.
Note that the `duration` column is measured in seconds.

Solution:

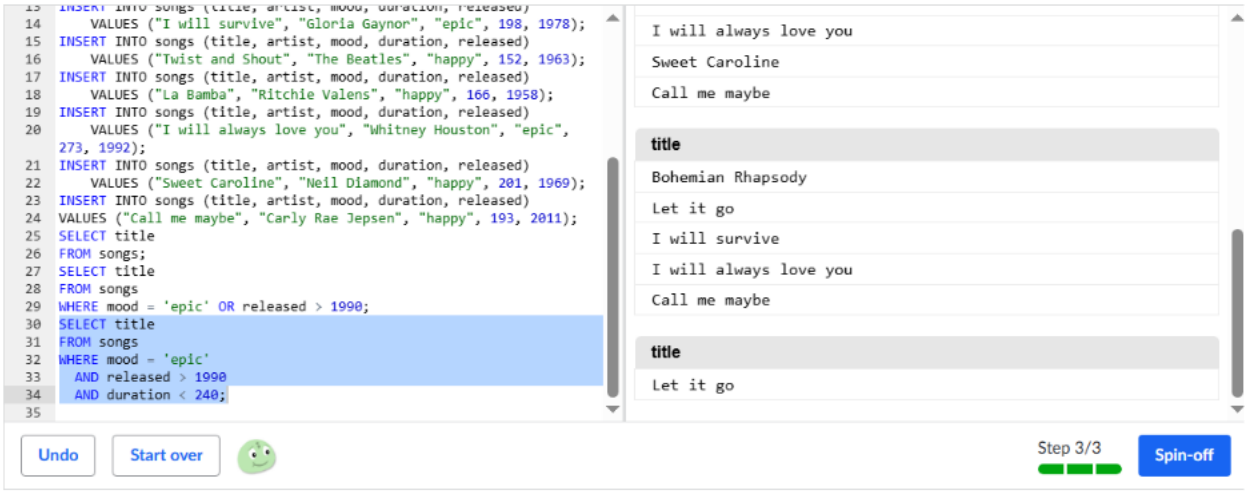
```
SELECT title

FROM songs

WHERE mood = 'epic'

  AND released > 1990

  AND duration < 240;
```



Challenge: Playlist maker

Step 1

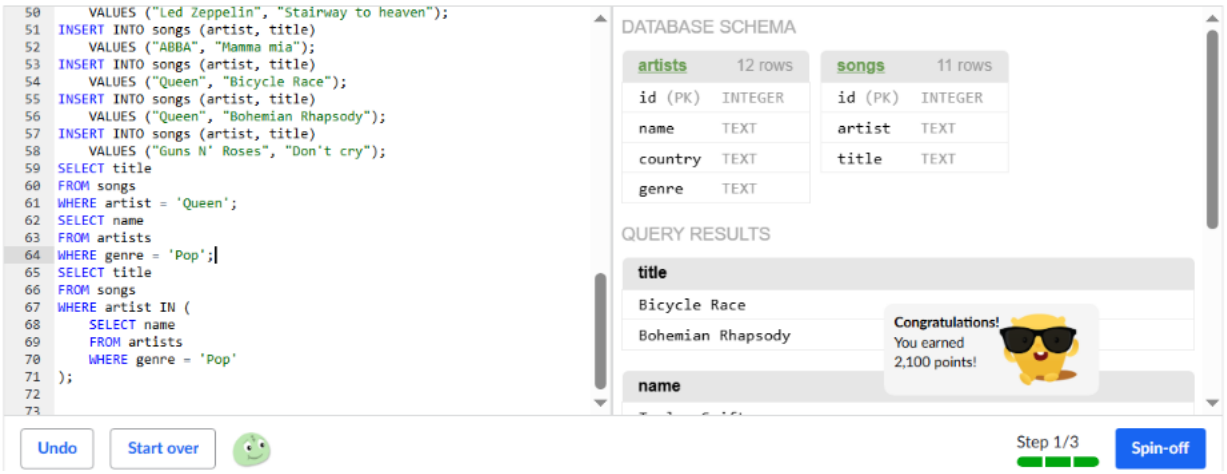
We've created a database of songs and artists, and you'll make playlists from them in this challenge. In this first step, select the `title` of all the songs by the `artist` named 'Queen'.

Solution:

SELECT title

FROM songs

WHERE artist = 'Queen';



Step 2

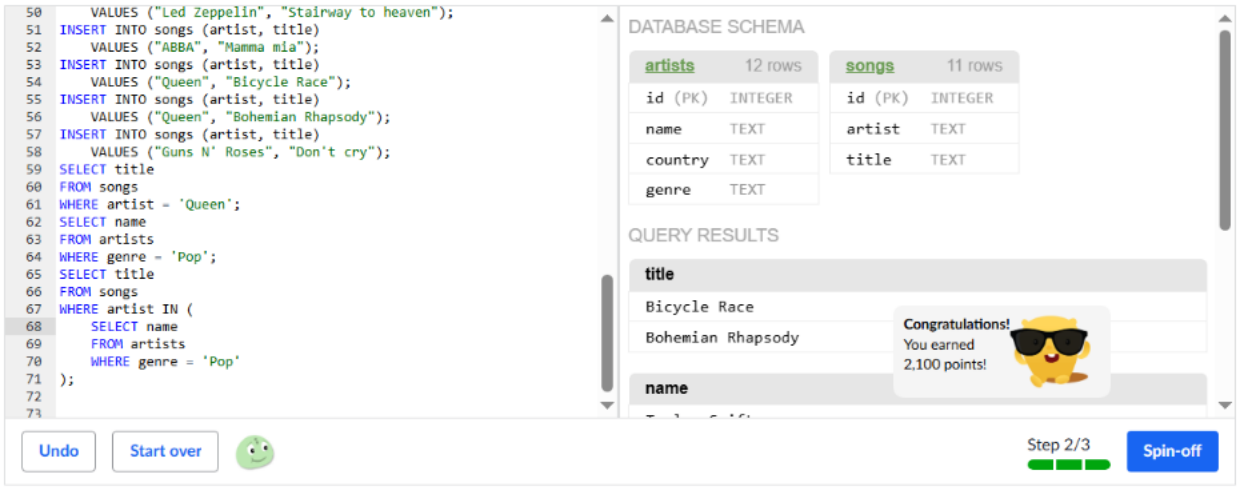
Now you'll make a 'Pop' playlist. In preparation, select the `name` of all of the artists from the 'Pop' genre.
(Tip: Make sure you type it 'Pop', SQL considers that different from 'pop'.)

Solution:

SELECT name

FROM artists

WHERE genre = 'Pop';



Step 3

To finish creating the 'Pop' playlist, add another query that will select the `title` of all the songs from the 'Pop' artists. It should use `IN` on a nested subquery that's based on your previous query.

Solution:

SELECT title

FROM songs

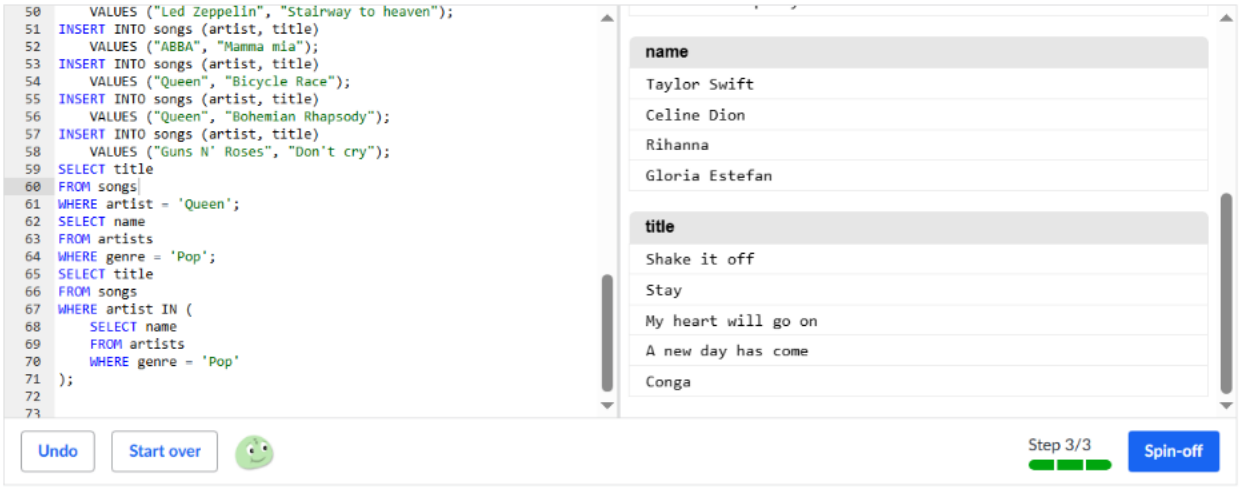
WHERE artist IN (

  SELECT name

  FROM artists

  WHERE genre = 'Pop'

);



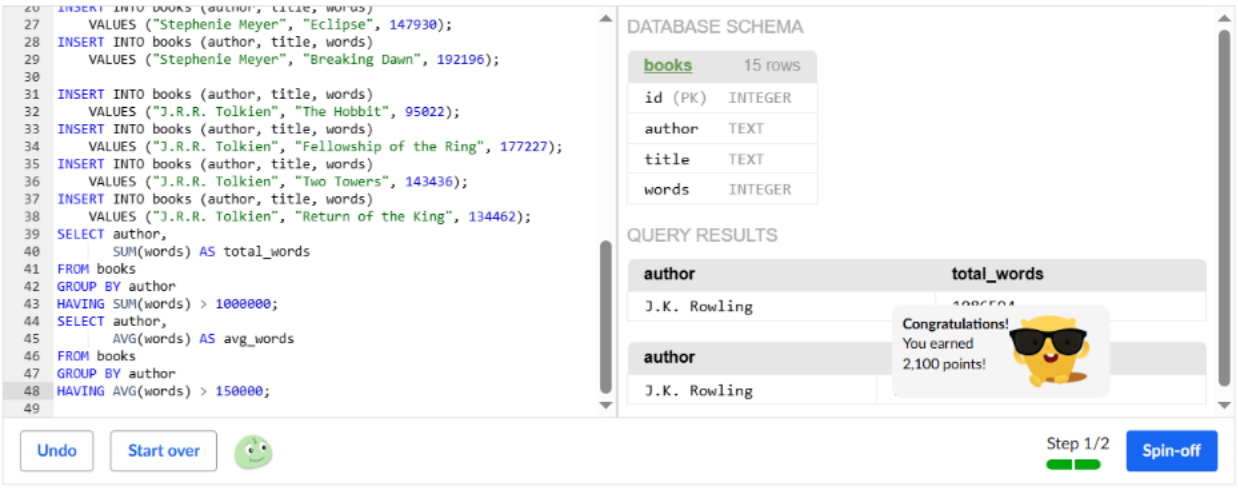
Challenge: The wordiest author

Step 1

We've created a database of a few popular authors and their books, with word counts for each book. In this first step, select all the authors who have written more than 1 million words, using `GROUP BY` and `HAVING`. Your results table should include the 'author' and their total word count as a 'total_words' column.

Solution:

```
SELECT author,
    SUM(words) AS total_words
FROM books
GROUP BY author
HAVING SUM(words) > 1000000;
```



Step 2

Now select all the authors that write more than an average of 150,000 words per book. Your results table should include the 'author' and average words as an 'avg_words' column.

Solution:

```
FROM books

GROUP BY author

HAVING SUM(words) > 1000000;

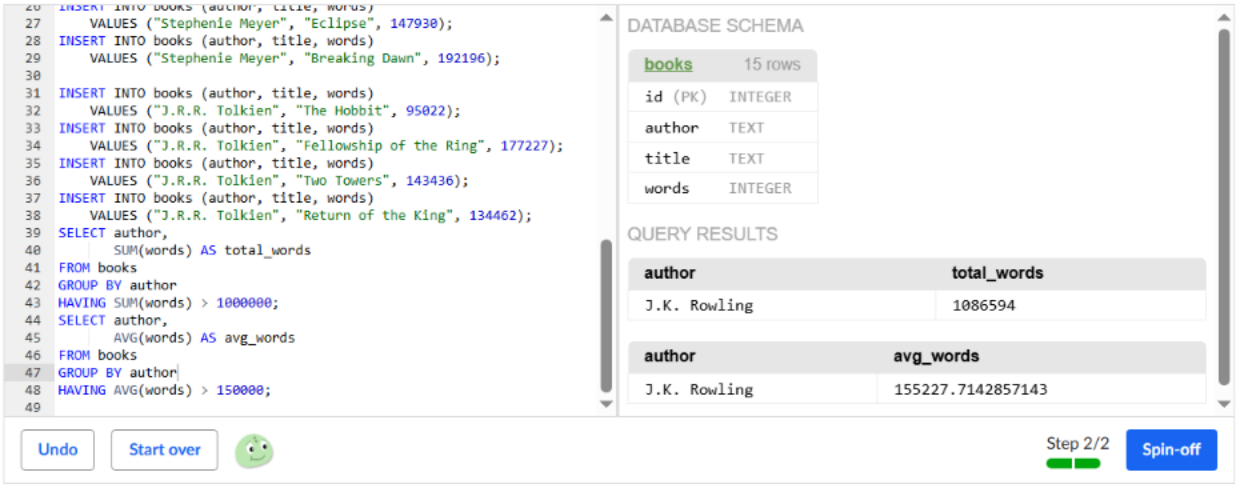
SELECT author,

    AVG(words) AS avg_words

FROM books

GROUP BY author

HAVING AVG(words) > 150000;
```



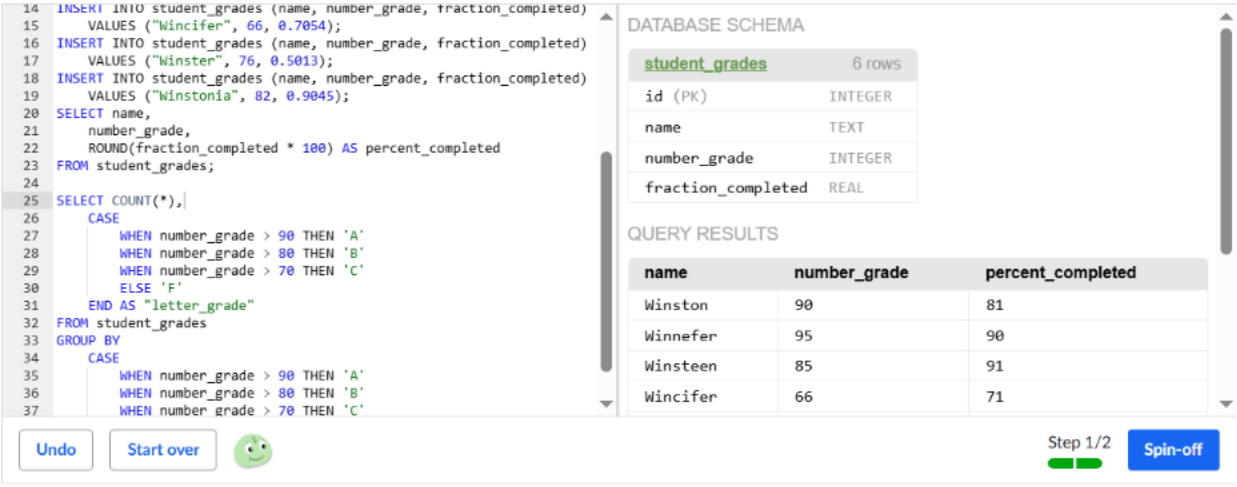
Challenge: Gradebook

Step 1

We've created a database to track student grades, with their name, number grade, and what percent of activities they've completed. In this first step, select all of the rows, and display the `name`, `number_grade`, and `percent_completed`, which you can compute by multiplying and rounding the `fraction_completed` column.

Solution:

```
SELECT name,
    number_grade,
    ROUND(fraction_completed * 100) AS percent_completed
FROM student_grades;
```



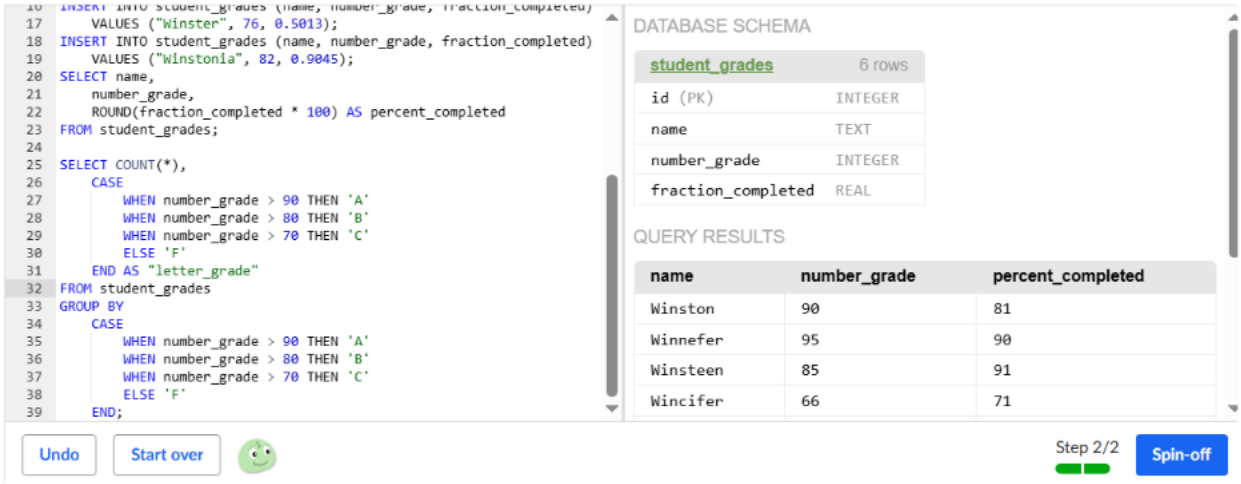
Step 2

Now, this step is a little tricky. The goal is a table that shows how many students have earned which `letter_grade`. You can output the `letter_grade` by using `CASE` with the `number_grade` column, outputting 'A' for grades > 90, 'B' for grades > 80, 'C' for grades > 70, and 'F' otherwise. Then you can use `COUNT` with `GROUP BY` to show the number of students with each of those grades.

Solution:

```
SELECT COUNT(*),
  CASE
    WHEN number_grade > 90 THEN 'A'
    WHEN number_grade > 80 THEN 'B'
    WHEN number_grade > 70 THEN 'C'
    ELSE 'F'
  END AS "letter_grade"
FROM student_grades
GROUP BY
  CASE
    WHEN number_grade > 90 THEN 'A'
```

```
    WHEN number_grade > 80 THEN 'B'
    WHEN number_grade > 70 THEN 'C'
    ELSE 'F'
END;
```



Project: Data dig

Question:

We've curated a set of interesting data sets for you: NASA astronauts, Superbowl results, Pokemon stats, NBA players, Top movies, Top countries by population, Solar system objects by size, Marvel characters, Furniture store sales, Earned KA badges, Winston's donut logs, Card game results, and NFL draft picks.

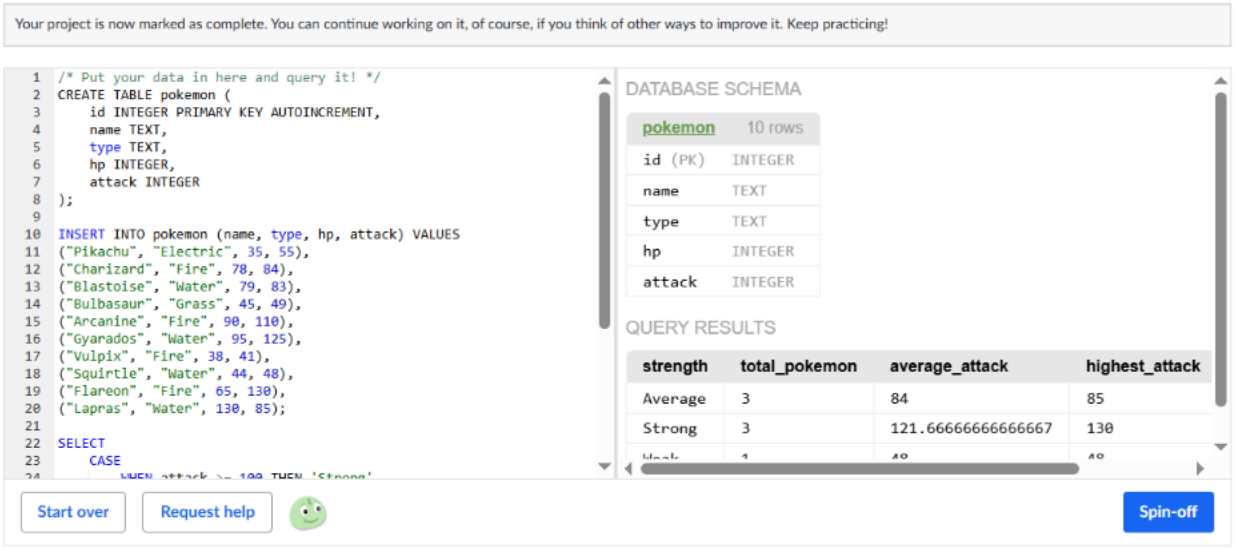
Pick one of those data sets or create a data set like that, and use advanced `SELECT` queries to discover things about the data. What sort of questions might one have about that data, like if they were using it for an app or a business idea? Here are some ideas:

What are average, max, and min values in the data?
What about those numbers per category in the data (using HAVING)?
What ways are there to group the data values that don't exist yet (using CASE)?
What interesting ways are there to filter the data (using AND/OR)?

Solution:

```
CREATE TABLE pokemon (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    type TEXT,
    hp INTEGER,
    attack INTEGER
);
 INSERT INTO pokemon (name, type, hp, attack) VALUES
("Pikachu", "Electric", 35, 55),
("Charizard", "Fire", 78, 84),
("Blastoise", "Water", 79, 83),
("Bulbasaur", "Grass", 45, 49),
("Arcanine", "Fire", 90, 110),
("Gyarados", "Water", 95, 125),
("Vulpix", "Fire", 38, 41),
("Squirtle", "Water", 44, 48),
("Flareon", "Fire", 65, 130),
("Lapras", "Water", 130, 85);
SELECT
    CASE
        WHEN attack >= 100 THEN 'Strong'
        WHEN attack >= 70 THEN 'Average'
        ELSE 'Weak'
```

```
    END AS strength,
    COUNT(*) AS total_pokemon,
    AVG(attack) AS average_attack,
    MAX(attack) AS highest_attack,
    MIN(attack) AS lowest_attack
FROM pokemon
WHERE hp >= 40
AND (type = 'Fire' OR type = 'Water')
GROUP BY strength
HAVING COUNT(*) >= 1;
```



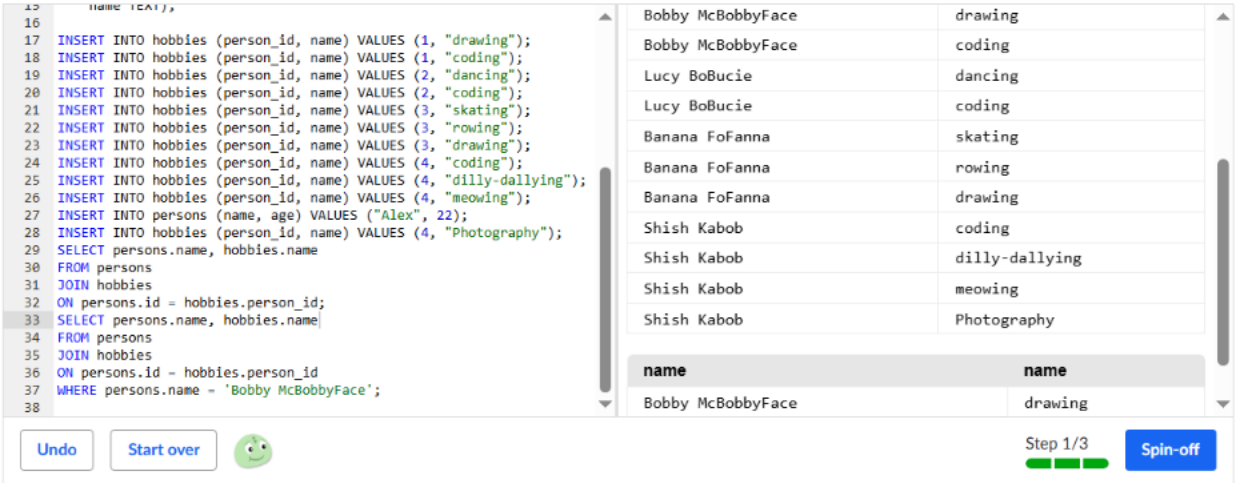
Challenge: Bobby's Hobbies

Step 1

We've created a database of people and hobbies, and each row in `hobbies` is related to a row in `persons` via the `person_id` column. In this first step, insert one more row in `persons` and then one more row in `hobbies` that is related to the newly inserted person.

Solution:

SELECT persons.name, hobbies.name



Step 2

Now, select the 2 tables with a join so that you can see each person's name next to their hobby.

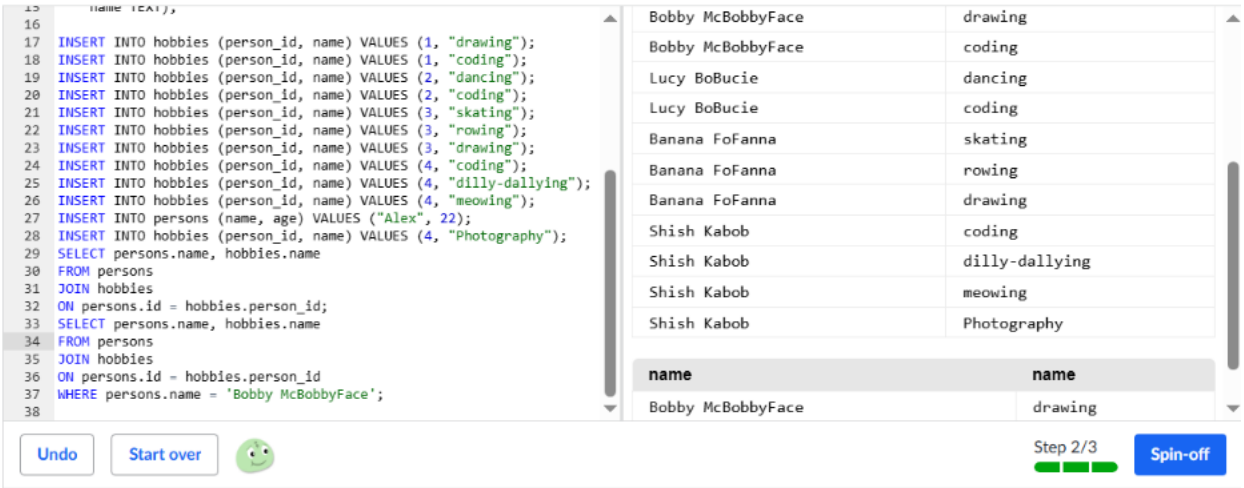
Solution:

FROM persons

JOIN hobbies

ON persons.id = hobbies.person_id;

SELECT persons.name, hobbies.name



Step 3

Now, add an additional query that shows **only** the name and hobbies of 'Bobby McBobbyFace', using `JOIN` combined with `WHERE`.

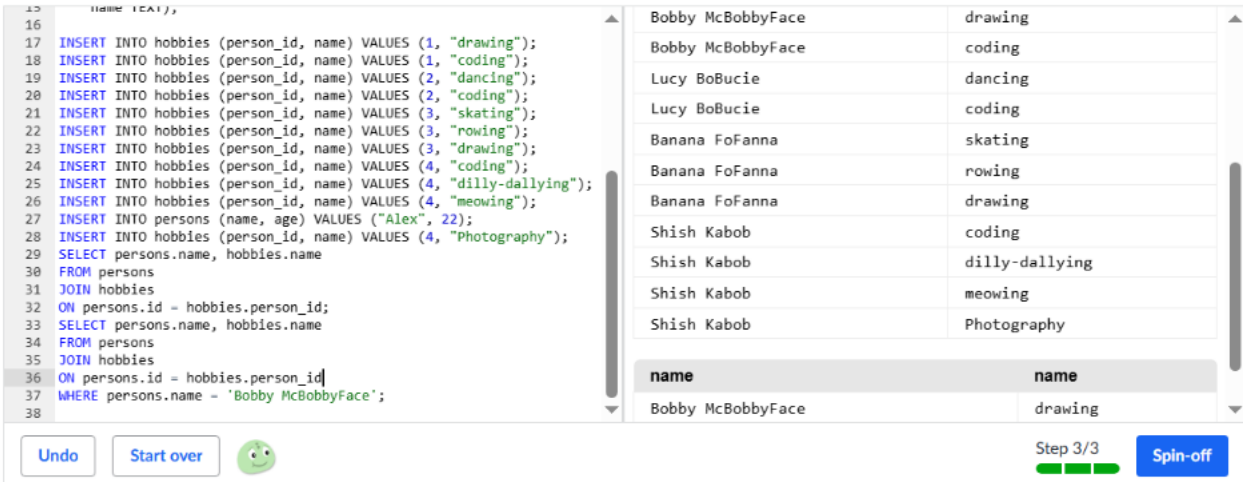
Solution:

FROM persons

JOIN hobbies

ON persons.id = hobbies.person_id

WHERE persons.name = 'Bobby McBobbyFace';



Challenge: Customer's orders

Step 1

We've created a database for customers and their orders. Not all of the customers have made orders, however. Come up with a query that lists the `name` and `email` of every customer followed by the `item` and `price` of orders they've made. Use a `LEFT OUTER JOIN` so that a customer is listed even if they've made no orders, and don't add any `ORDER BY`.

Solution:

VALUES (1, "TARDIS", 1000000.00);

SELECT customers.name,

customers.email,

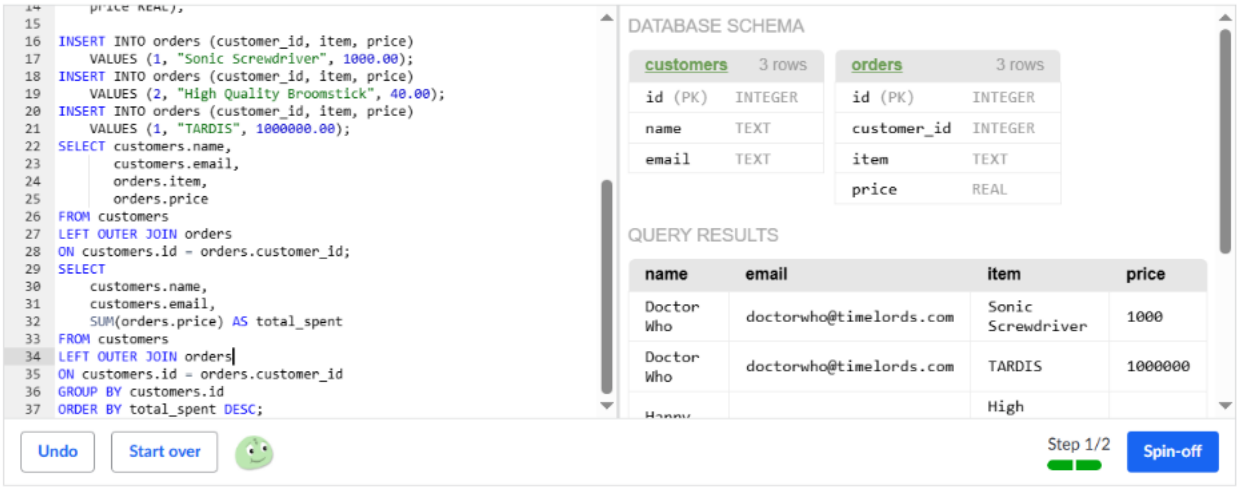
orders.item,

orders.price

FROM customers

LEFT OUTER JOIN orders

ON customers.id = orders.customer_id;



Step 2

Now, create another query that will result in one row per each customer, with their `name`, `email`, and total amount of money they've spent on orders. Sort the rows according to the total money spent, from the most spent to the least spent.
(Tip: You should always `GROUP BY` on the column that is most likely to be unique in a row.)

Solution:

SELECT

customers.name,

customers.email,

SUM(orders.price) AS total_spent

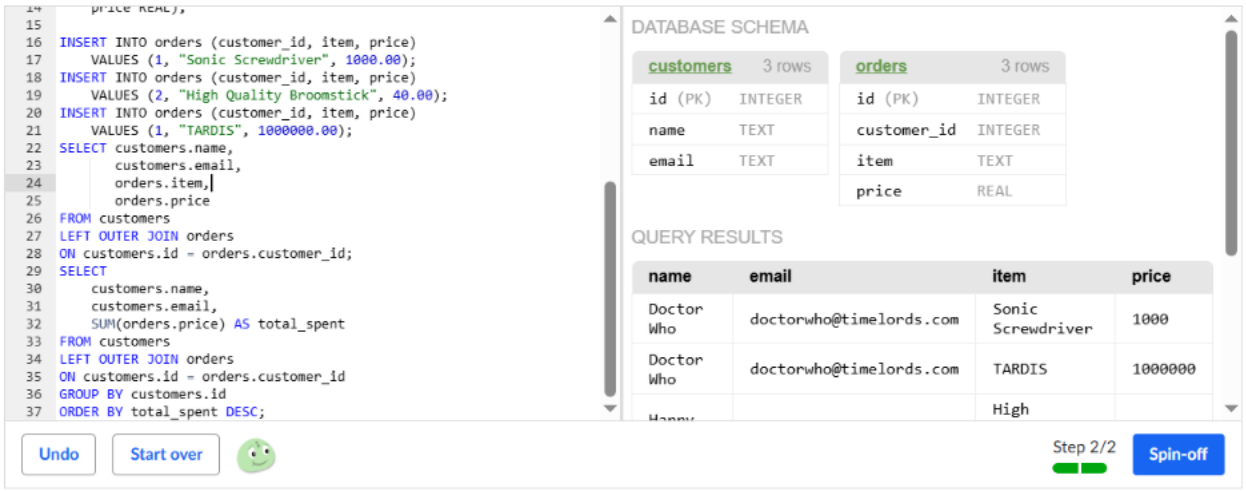
FROM customers

LEFT OUTER JOIN orders

ON customers.id = orders.customer_id

GROUP BY customers.id

ORDER BY total_spent DESC;



Challenge: Sequels in SQL

Step 1

We've created a table with all the 'Harry Potter' movies, with a `sequel_id` column that matches the `id` of the sequel for each movie. Issue a `SELECT` that will show the title of each movie next to its sequel's title (or `NULL` if it doesn't have a sequel).

Solution:

2011, NULL);

SELECT

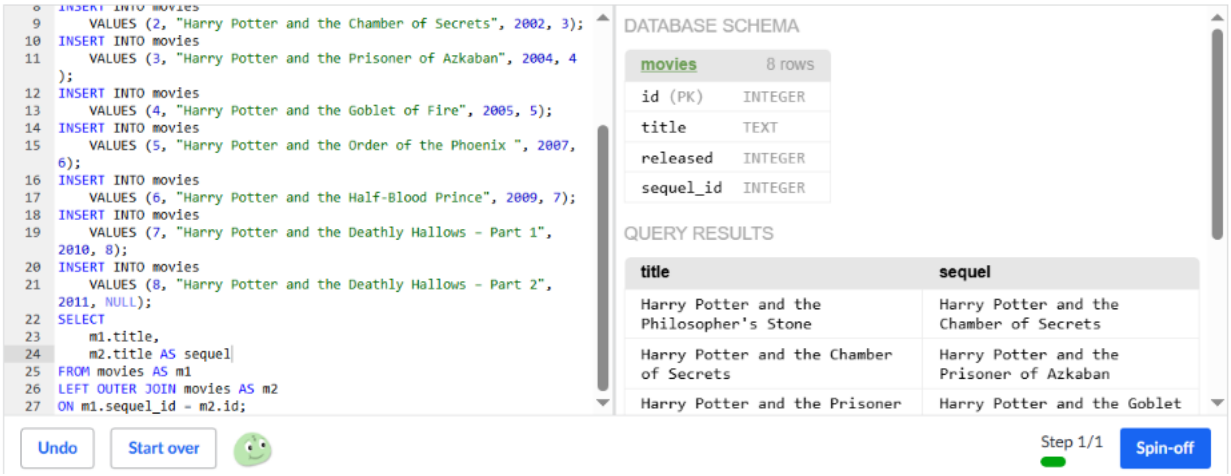
m1.title,

m2.title AS sequel

FROM movies AS m1

LEFT OUTER JOIN movies AS m2

ON m1.sequel_id = m2.id;



Challenge: Friend Book

Step 1

We've created a database for a friend networking site, with a table storing data on each person, a table on each person's hobbies, and a table of friend connections between the people. In this first step, use a `JOIN` to display a table showing people's names with their hobbies.

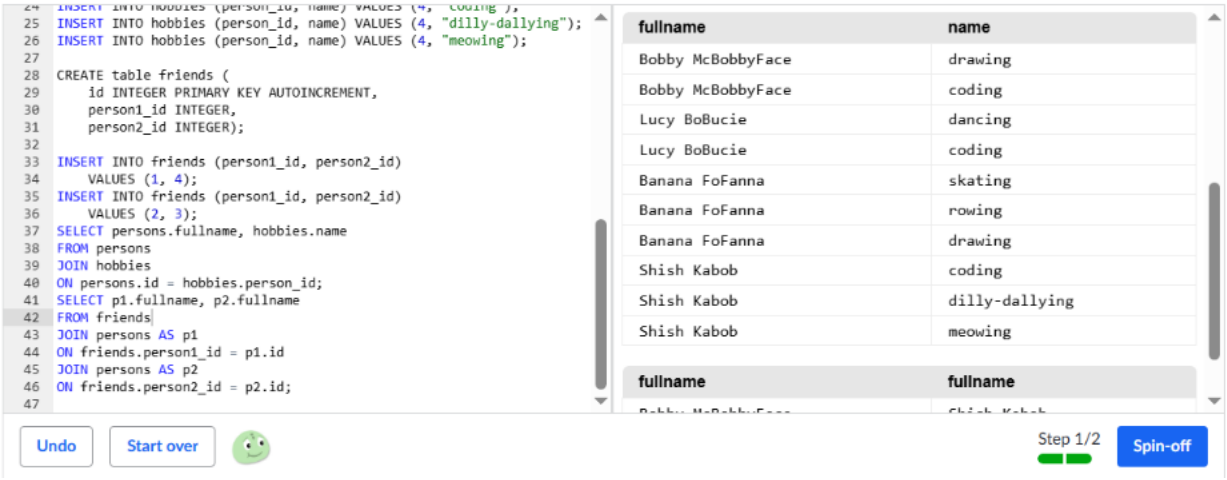
Solution:

SELECT persons.fullname, hobbies.name

FROM persons

JOIN hobbies

ON persons.id = hobbies.person_id;



Step 2

Now, use another `SELECT` with a `JOIN` to show the names of each pair of friends, based on the data in the `friends` table.

Solution:

SELECT p1.fullname, p2.fullname

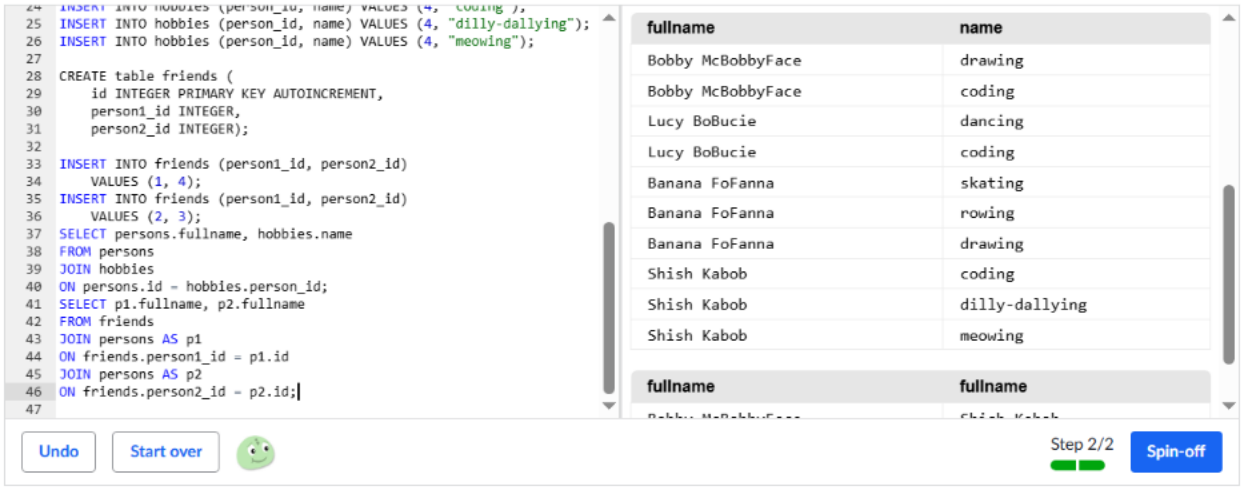
FROM friends

JOIN persons AS p1

ON friends.person1_id = p1.id

JOIN persons AS p2

ON friends.person2_id = p2.id;



Project: Famous people

Question:

In this project, you're going to make your own table with some small set of "famous people", then make more tables about things they do and join those to create nice human readable lists.

For example, here are types of famous people and the questions your data could answer:

Movie stars: What movies are they in? Are they married to each other?
Singers: What songs did they write? Where are they from?
Authors: What books did they write?
Fictional characters: How are they related to other characters? What books do they show up in?

Solution:

```
CREATE TABLE authors (

    id INTEGER PRIMARY KEY AUTOINCREMENT,

    name TEXT,

    country TEXT

);

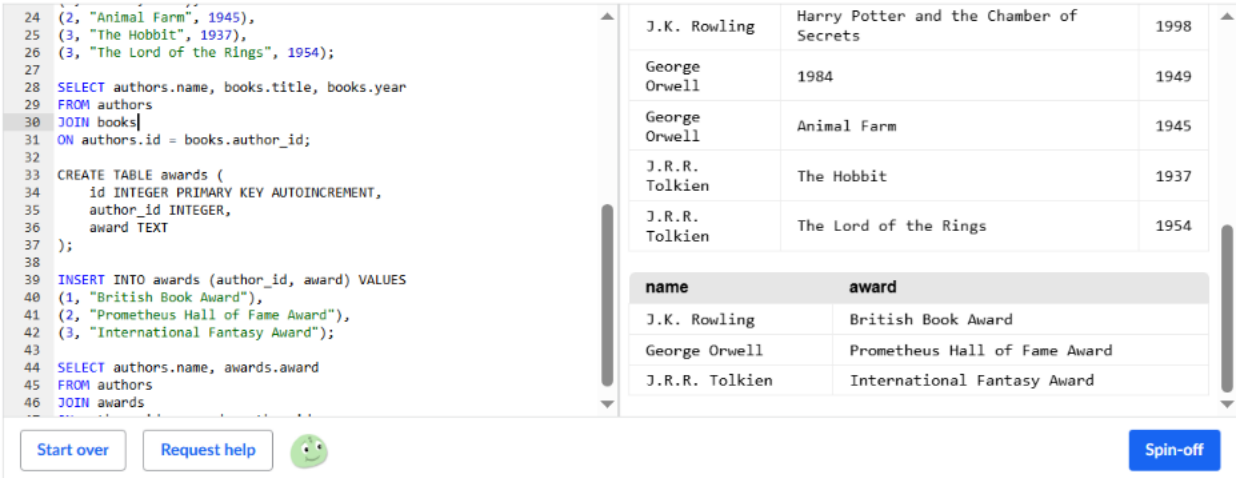
 INSERT INTO authors (name, country) VALUES
```

```
("J.K. Rowling", "United Kingdom"),
("George Orwell", "United Kingdom"),
("J.R.R. Tolkien", "United Kingdom");
CREATE TABLE books (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    author_id INTEGER,
    title TEXT,
    year INTEGER
);
INSERT INTO books (author_id, title, year) VALUES
(1, "Harry Potter and the Philosopher's Stone", 1997),
(1, "Harry Potter and the Chamber of Secrets", 1998),
(2, "1984", 1949),
(2, "Animal Farm", 1945),
(3, "The Hobbit", 1937),
(3, "The Lord of the Rings", 1954);
SELECT authors.name, books.title, books.year
FROM authors
JOIN books
ON authors.id = books.author_id;
CREATE TABLE awards (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    author_id INTEGER,
```

```
    award TEXT
);

INSERT INTO awards (author_id, award) VALUES
(1, "British Book Award"),
(2, "Prometheus Hall of Fame Award"),
(3, "International Fantasy Award");

SELECT authors.name, awards.award
FROM authors
JOIN awards
ON authors.id = awards.author_id;
```



Challenge: Dynamic Documents

Step 1

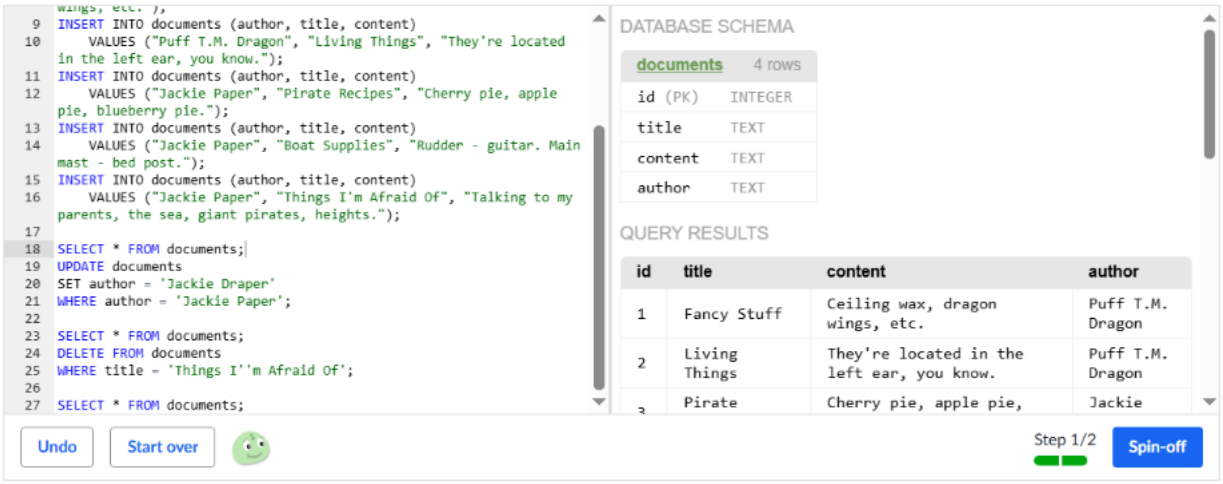
We've created a database for a documents app, with rows for each document with it's title, content, and author. In this first step, use `UPDATE` to change the author to 'Jackie Draper' for all rows where it's currently 'Jackie Paper'. Then re-select all the rows to make sure the table changed like you expected.

Solution:

SELECT * FROM documents;

UPDATE documents SET author = 'Jackie Draper'

WHERE author = 'Jackie Paper';



Step 2

Now you'll delete a row, being very careful not to delete all the rows. Only delete the row where the title is 'Things I'm Afraid Of'. Then re-select all the rows to make sure the table changed like you expected.

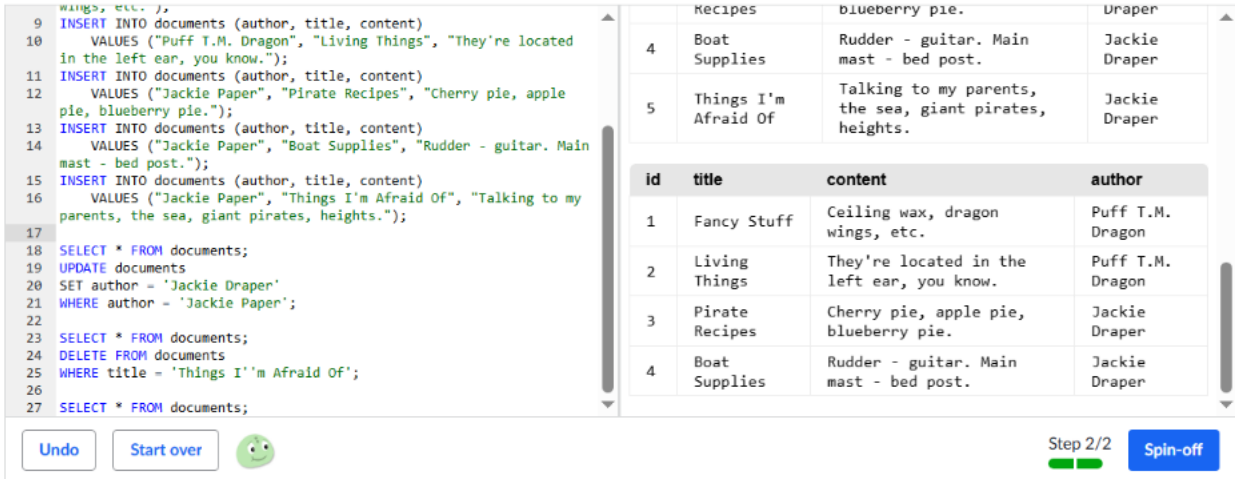
Solution:

SELECT * FROM documents;

DELETE FROM documents

WHERE title = 'Things I''m Afraid Of';

SELECT * FROM documents;



Challenge: Clothing alterations

Step 1

We've created a database of clothes, and decided we need a price column. Use `ALTER` to add a 'price' column to the table. Then select all the columns in each row to see what your table looks like now.

Solution:

```
CREATE TABLE clothes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    type TEXT,
    design TEXT);


INSERT INTO clothes (type, design)
    VALUES ("dress", "pink polka dots");
INSERT INTO clothes (type, design)
    VALUES ("pants", "rainbow tie-dye");
```

INSERT INTO clothes (type, design)

VALUES ("blazer", "black sequin");ALTER TABLE clothes

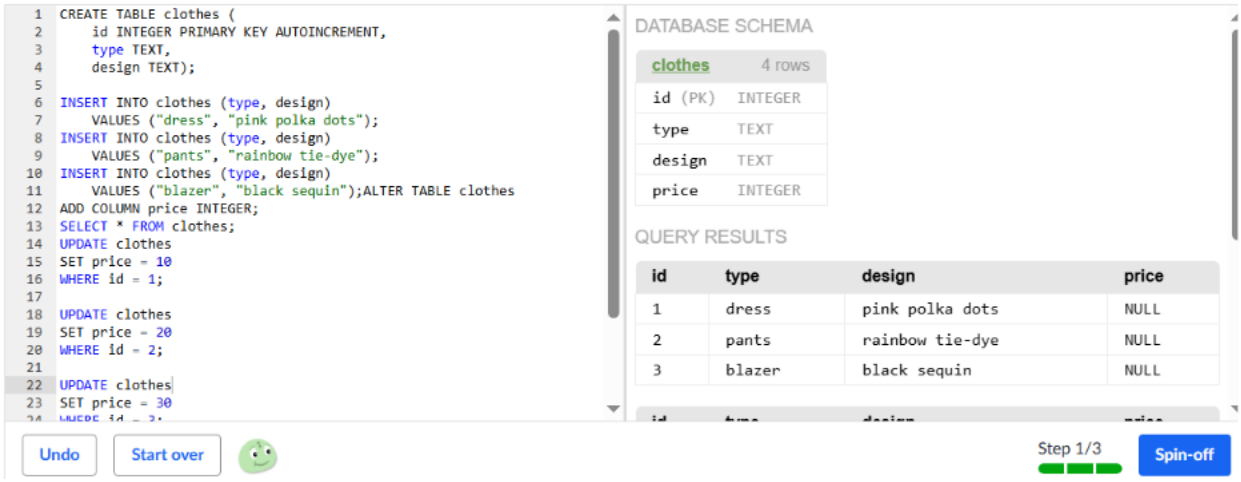
ADD COLUMN price INTEGER;

SELECT * FROM clothes;

UPDATE clothes

SET price = 10

WHERE id = 1;



Step 2

Now assign each item a price, using `UPDATE` - item 1 should be 10 dollars, item 2 should be 20 dollars, item 3 should be 30 dollars. When you're done, do another `SELECT` of all the rows to check that it worked as expected.

Solution:

PDATE clothes

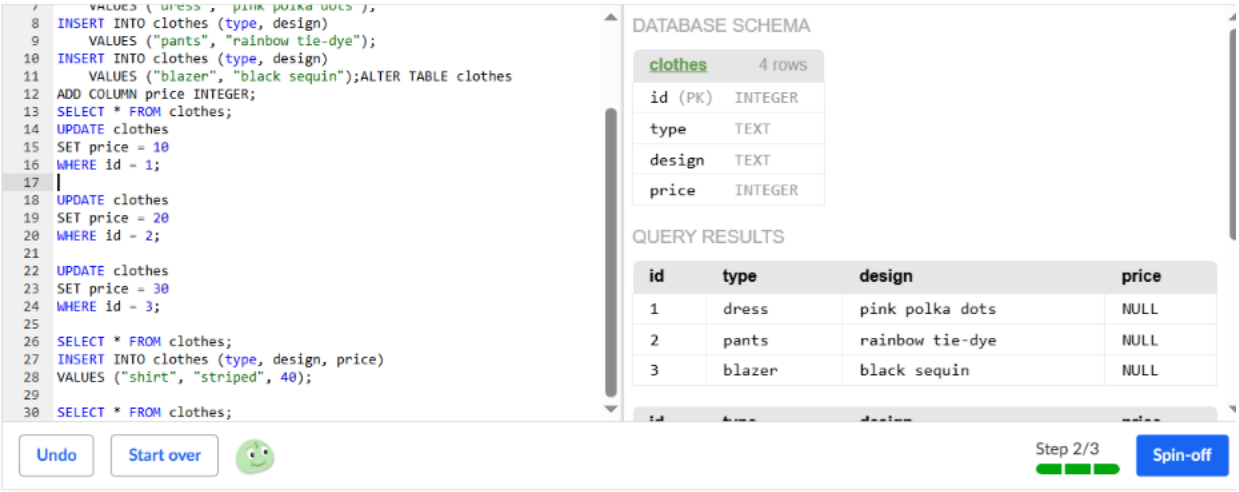
SET price = 20

WHERE id = 2

UPDATE clothes

SET price = 30

WHERE id = 3;



Step 3

Now insert a new item into the table that has all three attributes filled in, including 'price'. Do one final `SELECT` of all the rows to check it worked.

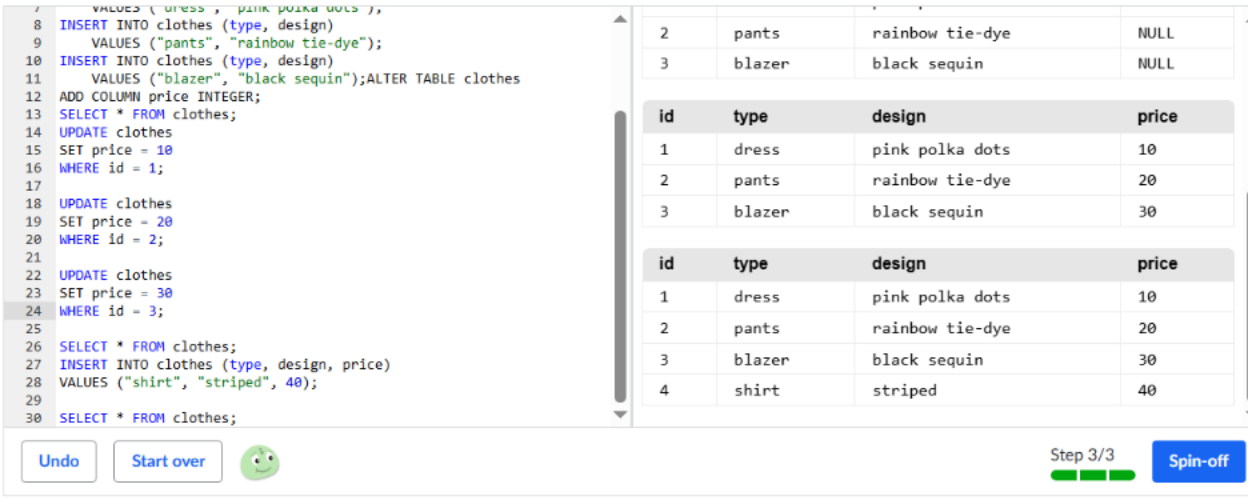
Solution:

SELECT * FROM clothes;

INSERT INTO clothes (type, design, price)

VALUES ("shirt", "striped", 40);

SELECT * FROM clothes;



Project: App impersonator

Question:

Think about your favorite apps, and pick one that stores your data- like a game that stores scores, an app that lets you post updates, etc. Now in this project, you're going to imagine that the app stores your data in a SQL database (which is pretty likely!), and write SQL statements that might look like their own SQL.

`CREATE` a table to store the data.
`INSERT` a few example rows in the table.
Use an `UPDATE` to emulate what happens when you edit data in the app.
Use a `DELETE` to emulate what happens when you delete data in the app.

Solution:

```
CREATE TABLE posts (

    id INTEGER PRIMARY KEY AUTOINCREMENT,

    username TEXT,

    content TEXT,

    likes INTEGER

);
```

INSERT INTO posts (username, content, likes)

VALUES ("Alice", "Hello everyone!", 15);

INSERT INTO posts (username, content, likes)

VALUES ("Bob", "Learning SQL is fun!", 20);

INSERT INTO posts (username, content, likes)

VALUES ("Charlie", "Enjoying my vacation.", 30);

SELECT * FROM posts;

UPDATE posts

SET content = "Learning SQL is really fun!"

WHERE username = "Bob";

SELECT * FROM posts;

DELETE FROM posts

WHERE username = "Alice"

SELECT * FROM posts;



Hackerrank

1.Revising the Select Query I

Query all columns for all American cities in the **CITY** table with populations larger than `100000`. The **CountryCode** for America is `USA`.

The **CITY** table is described as follows:

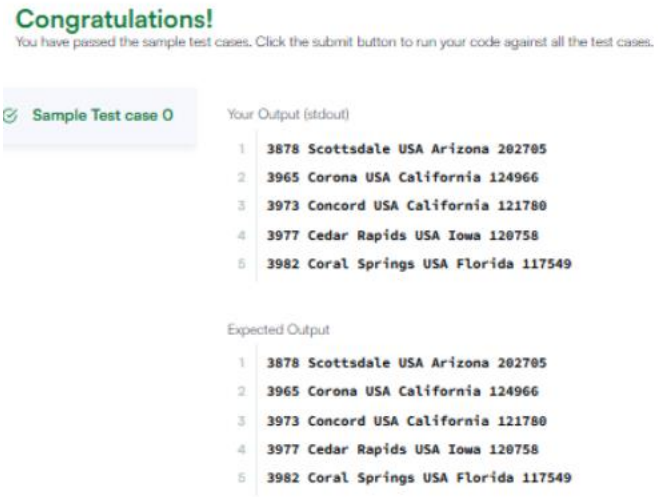
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT * FROM CITY WHERE COUNTRYCODE = 'USA' AND POPULATION > 100000;
```

Output:



2. Revising the Select Query II

Query the **NAME** field for all American cities in the **CITY** table with populations larger than `120000`. The *CountryCode* for America is `USA`.

The **CITY** table is described as follows:

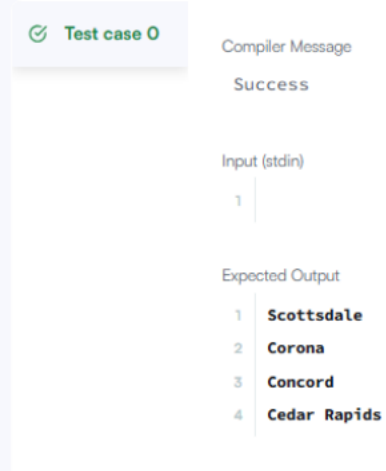
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT NAMEFROM CITY WHERE COUNTRYCODE = 'USA' AND POPULATION > 120000;
```

Output:



3. Select All

Query all columns (attributes) for every row in the **CITY** table.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT * FROM CITY;
```

Output:



4. Select By ID

Query all columns for a city in **CITY** with the *ID* `1661`.

The **CITY** table is described as follows:



Input:

```
SELECT * FROM CITY WHERE ID = 1661;
```

Output:



5. Japanese Cities' Attributes

Query all attributes of every Japanese city in the **CITY** table. The **COUNTRYCODE** for Japan is `JPN`.

The **CITY** table is described as follows:



Input:

```
SELECT * FROM CITY WHERE COUNTRYCODE = 'JPN';
```

Output:



6. Japanese Cities' Names

Query the names of all the Japanese cities in the **CITY** table. The **COUNTRYCODE** for Japan is `JPN`.

The **CITY** table is described as follows:



Input:

```
SELECT NAME FROM CITY WHERE COUNTRYCODE = 'JPN';
```

Output:



7. Weather Observation Station 1

Query a list of **CITY** and **STATE** from the **STATION** table.

The **STATION** table is described as follows:

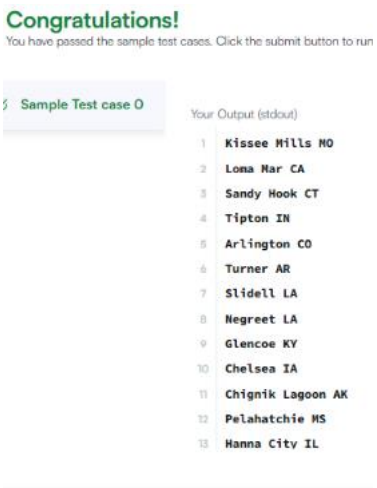


where **LAT_N** is the northern latitude and **LONG_W** is the western longitude.

Input:

```
SELECT CITY, STATE FROM STATION;
```

Output:



8. Weather Observation Station 3

Query a list of **CITY** names from **STATION** for cities that have an even **ID** number. Print the results in any order, but exclude duplicates from the answer.
The **STATION** table is described as follows:

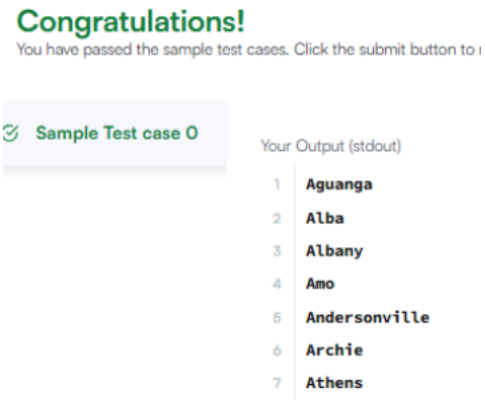


where **LAT_N** is the northern latitude and **LONG_W** is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE MOD(ID, 2) = 0 ORDER BY CITY;
```

Output:



9. Weather Observation Station 4

Find the difference between the total number of **CITY** entries in the table and the number of distinct **CITY** entries in the table.
The **STATION** table is described as follows:



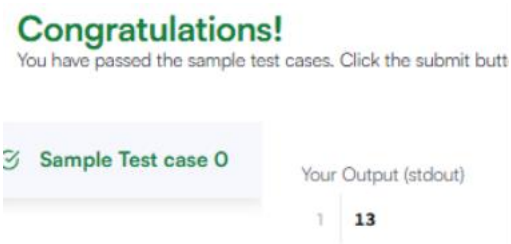
where **LAT_N** is the northern latitude and **LONG_W** is the western longitude.

For example, if there are three records in the table with **CITY** values 'New York', 'New York', 'Bengalaru', there are 2 different city names: 'New York' and 'Bengalaru'.

Input:

```
SELECT COUNT(CITY) - COUNT (DISTINCT City) FROM STATION ;
```

Output:



10. Weather Observation Station 5

Query the two cities in **STATION** with the shortest and longest *CITY* names, as well as their respective lengths (i.e.: number of characters in the name). If there is more than one smallest or largest city, choose the one that comes first when ordered alphabetically.
The **STATION** table is described as follows:



where **LAT_N** is the northern latitude and **LONG_W** is the western longitude.

Sample Input

For example, **CITY** has four entries: **DEF, ABC, PQRS** and **WXY**.

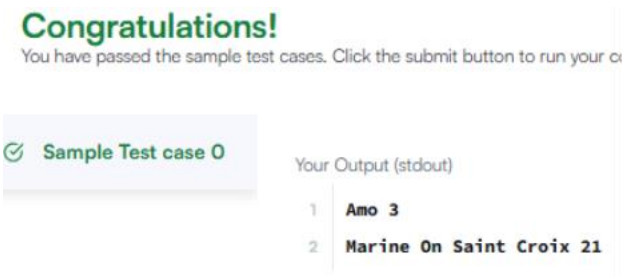
Sample Output

```
ABC 3
PQRS 4
```

Input:

```
SELECT CITY, LENGTH(CITY) FROM STATION ORDER BY LENGTH(CITY), CITY LIMIT 1;
SELECT CITY, LENGTH(CITY) FROM STATION ORDER BY LENGTH(CITY) DESC, CITY LIMIT
1;
```

Output:



11. Weather Observation Station 6

Query the list of *CITY* names starting with vowels (i.e., `a`, `e`, `i`, `o`, or `u`) from **STATION**. Your result *cannot* contain duplicates.

Input Format

The **STATION** table is described as follows:

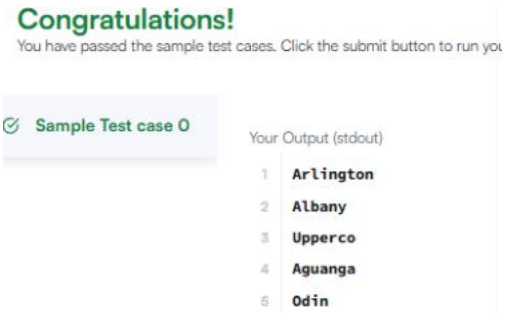


where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP '^[aeiouAEIOU]';
```

Output:



12. Weather Observation Station 7

Query the list of *CITY* names ending with vowels (a, e, i, o, u) from **STATION**. Your result *cannot* contain duplicates.

Input Format

The **STATION** table is described as follows:



where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP '[aeiouAEIOU]$';
```

Output:



13. Weather Observation Station 8

Query the list of *CITY* names from **STATION** which have vowels (i.e., *a*, *e*, *i*, *o*, and *u*) as both their first *and* last characters. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:



where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP
'^[aeiouAEIOU].*[aeiouAEIOU]$';
```

Output:



14. Weather Observation Station 9

Query the list of *CITY* names from **STATION** that *do not start* with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:



where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP '^[^aeiouAEIOU]';
```

Output:



15. Weather Observation Station 10

Query the list of *CITY* names from **STATION** that *do not end* with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

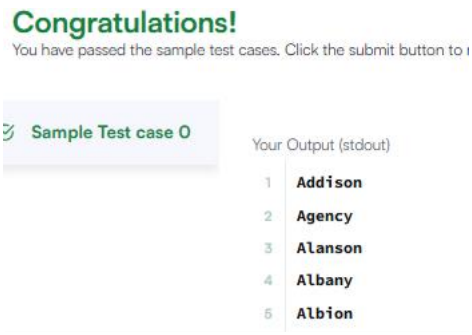


where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE RIGHT(CITY, 1) NOT IN ('A', 'E', 'I',
'O', 'U') ORDER BY CITY;
```

Output:



16. Weather Observation Station 11

Query the list of *CITY* names from **STATION** that either do not start with vowels or do not end with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:



where *LAT_N* is the northern latitude and *LONG_W* is the western longitude

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY NOT REGEXP '^[aeiouAEIOU]' OR
CITY NOT REGEXP '[aeiouAEIOU]$';
```

Output:



17. Weather Observation Station 12

Query the list of *CITY* names from **STATION** that *do not start* with vowels and *do not end* with vowels. Your result cannot contain duplicates



where *LAT_N* is the northern latitude and *LONG_W* is the western longitud

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP '^[^aeiouAEIOU]' AND CITY REGEXP '[^aeiouAEIOU]$';
```

Output:



18. Higher Than 75 Marks

Query the *Name* of any student in **STUDENTS** who scored higher than *Marks*. Order your output by the *last three characters* of each name. If two or more students both have names ending in the same last three characters (i.e.: Bobby, Robby, etc.), secondary sort them by ascending *ID*.

Input Format

The **STUDENTS** table is described as follows:

Column	Type
ID	Integer
Name	String
Marks	Integer

The *Name* column only contains uppercase (`A-Z`) and lowercase (`a-z`) letters.

Input:

```
SELECT NAME FROM STUDENTS WHERE MARKS > 75 ORDER BY RIGHT(NAME, 3), ID ASC;
```

Output:



19. Employee Names

Write a query that prints a list of employee names (i.e.: the *name* attribute) from the **Employee** table in alphabetical order.

Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is their monthly salary.

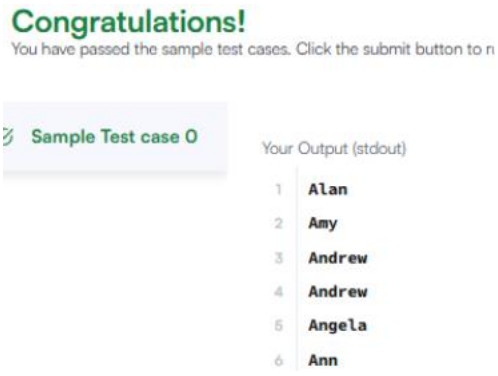
Sample Input



Input:

```
SELECT name FROM Employee ORDER BY name;
```

Output:



20. Employee Salaries

Write a query that prints a list of employee names (i.e.: the *name* attribute) for employees in **Employee** having a salary greater than per month who have been employees for less than months. Sort your result by ascending *employee_id*.

Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is the their monthly salary.



Input:

```
SELECT name FROM Employee WHERE salary > 2000   AND months < 10 ORDER BY
employee_id ASC;
```

Output:



21. Type of Triangle

Write a query identifying the *type* of each record in the **TRIANGLES** table using its three side lengths. Output one of the following statements for each record in the table:

- **Equilateral**: It's a triangle with sides of equal length.
- **Isosceles**: It's a triangle with sides of equal length.
- **Scalene**: It's a triangle with sides of differing lengths.
- **Not A Triangle**: The given values of *A*, *B*, and *C* don't form a triangle.

Input Format

The **TRIANGLES** table is described as follows:

Column	*Type*
A	*Integer*
B	*Integer*
C	*Integer*

Each row in the table denotes the lengths of each of a triangle's three sides.

Input:

```
SELECT CASE  WHEN A + B <= C OR A + C <= B OR B + C <= A THEN 'Not A Triangle
WHEN A = B AND B = C THEN 'Equilateral' WHEN A = B OR B = C OR A = C THEN
'Isosceles 'ELSE 'Scalene'END FROM TRIANGLES;
```

Output:



23. The PADS

Generate the following two result sets:

1. Query an *alphabetically ordered* list of all names in **OCCUPATIONS**, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For example: `AnActorName(A)`, `ADoctorName(D)`, `AProfessorName(P)`, and `ASingerName(S)`.
2. Query the number of ocurrences of each occupation in **OCCUPATIONS**. Sort the occurrences in *ascending order*, and output them in the following format:

   ```
   There are a total of [occupation_count] [occupation]s.
   ```

   where `[occupation_count]` is the number of occurrences of an occupation in **OCCUPATIONS** and `[occupation]` is the *lowercase* occupation name. If more than one *Occupation* has the same `[occupation_count]`, they should be ordered alphabetically.

Note: There will be at least two entries in the table for each type of occupation.

Input Format

The **OCCUPATIONS** table is described as follows:  *Occupation* will only contain one of the following values: **Doctor**, **Professor**, **Singer** or **Actor**.

Input:

```
SELECT CONCAT(Name, '(', LEFT(Occupation, 1), ')') FROM OCCUPATIONS ORDER BY
Name ASC;
SELECT CONCAT('There are a total of ', COUNT(Occupation), ' ',
LOWER(Occupation), 's.') FROM OCCUPATIONS
GROUP BY Occupation
ORDER BY COUNT(Occupation) ASC, Occupation ASC;
```

Output:



24. Occupations

Pivot the *Occupation* column in **OCCUPATIONS** so that each *Name* is sorted alphabetically and displayed underneath its corresponding *Occupation*. The output should consist of four columns (*Doctor*, *Professor*, *Singer*, and *Actor*) in that specific order, with their respective names listed alphabetically under each column.

Note: Print **NULL** when there are no more names corresponding to an occupation.

Input Format

The **OCCUPATIONS** table is described as follows:

Column	Type
Name	String
Occupation	String

Occupation will only contain one of the following values: **Doctor**, **Professor**, **Singer** or **Actor**.

Input:

```
SELECT
    MAX(CASE WHEN Occupation = 'Doctor' THEN Name END) AS Doctor,
    MAX(CASE WHEN Occupation = 'Professor' THEN Name END) AS Professor,
    MAX(CASE WHEN Occupation = 'Singer' THEN Name END) AS Singer,
    MAX(CASE WHEN Occupation = 'Actor' THEN Name END) AS Actor
FROM (
    SELECT
        Name,
        Occupation,
        ROW_NUMBER() OVER (PARTITION BY Occupation ORDER BY Name) AS rn
    FROM OCCUPATIONS
) AS sub
GROUP BY rn;
```

Output:



25. Binary Tree Nodes

You are given a table, *BST*, containing two columns: *N* and *P,* where *N* represents the value of a node in *Binary Tree*, and *P* is the parent of *N*.

Column	Type
N	Integer
P	Integer

Write a query to find the node type of *Binary Tree* ordered by the value of the node. Output one of the following for each node:

- *Root*: If node is root node.
- *Leaf*: If node is leaf node.
- *Inner*: If node is neither root nor leaf node.

Input:

```
SELECT N, CASE WHEN P IS NULL THEN 'Root'
WHEN N IN (SELECT P FROM BST WHERE P IS NOT NULL) THEN 'Inner'
 ELSE 'Leaf' END FROM BST ORDER BY N;
```

Output:



26. New Companies

Amber's conglomerate corporation just acquired some new companies. Each of the companies follows this hierarchy: 

Given the table schemas below, write a query to print the *company_code*, *founder* name, total number of *lead* managers, total number of *senior* managers, total number of *managers*, and total number of *employees*. Order your output by ascending *company_code*.

Note:

- The tables may contain duplicate records.
- The *company_code* is string, so the sorting should not be **numeric**. For example, if the *company_codes* are *C_1*, *C_2*, and *C_10*, then the ascending *company_codes* will be *C_1*, *C_10*, and *C_2*.

Input:

```
SELECT
    c.company_code,
    c.founder,
```

```
    COUNT(DISTINCT lm.lead_manager_code),
    COUNT(DISTINCT sm.senior_manager_code),
    COUNT(DISTINCT m.manager_code),
    COUNT(DISTINCT e.employee_code)
FROM Company c
LEFT JOIN Lead_Manager lm ON c.company_code = lm.company_code
LEFT JOIN Senior_Manager sm ON lm.lead_manager_code = sm.lead_manager_code
LEFT JOIN Manager m ON sm.senior_manager_code = m.senior_manager_code
LEFT JOIN Employee e ON m.manager_code = e.manager_code
GROUP BY c.company_code, c.founder
ORDER BY c.company_code ASC;
```

Output:



27. Revising Aggregations - The Count Function

Query a *count* of the number of cities in **CITY** having a *Population* larger than 100,000.

Input Format

The **CITY** table is described as follows: 

Input:

```
SELECT COUNT(*) FROM CITY WHERE POPULATION > 100000;
```

Output:



28. Revising Aggregations - Averages

Query the average population of all cities in **CITY** where *District* is **California**.

Input Format

The **CITY** table is described as follows: 

Input:

```
SELECT AVG(POPULATION) FROM CITY WHERE DISTRICT = 'California';
```

Output:



29. Average Population

Query the average population for all cities in **CITY**, rounded *down* to the nearest integer.

Input Format

The **CITY** table is described as follows: 

Input:

```
SELECT FLOOR(AVG(POPULATION)) FROM CITY;
```

Output:



30. Japan Population

Query the sum of the populations for all Japanese cities in **CITY**. The *COUNTRYCODE* for Japan is **JPN**.

Input Format

The **CITY** table is described as follows: 

Input:

```
SELECT SUM(POPULATION) FROM CITY WHERE COUNTRYCODE = 'JPN';
```

Output:



31. Population Density Difference

Input Format

The **CITY** table is described as follows: 

Input:

```
SELECT MAX(POPULATION) - MIN(POPULATION) FROM CITY;
```

Output:



32. The Blunder

Samantha was tasked with calculating the average monthly salaries for all employees in the **EMPLOYEES** table, but did not realize her keyboard's key was broken until after completing the calculation. She wants your help finding the difference between her miscalculation (using salaries with any zeros removed), and the actual average salary.

Write a query calculating the amount of error (i.e.: average monthly salaries), and round it up to the next integer.

Input Format

The **EMPLOYEES** table is described as follows:



Note: *Salary* is per month.

Input:

```
SELECT CEIL(AVG(SALARY) - AVG(REPLACE(SALARY, '0', ''))) FROM EMPLOYEES;
```

Output:



33. Top Earners

We define an employee's *total earnings* to be their monthly worked, and the *maximum total earnings* to be the maximum total earnings for any employee in the **Employee** table. Write a query to find the *maximum total earnings* for all employees as well as the total number of employees who have maximum total earnings. Then print these values as space-separated integers.

Input Format

The **Employee** table containing employee data for a company is described as follows:



where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is the their monthly salary.

Input:

```
SELECT (salary * months) AS earnings, COUNT(*) FROM Employee GROUP BY
earnings ORDER BY earnings DESC LIMIT 1;
```

Output:



34. Weather Observation Station 2

Query the following two values from the **STATION** table:

1. The sum of all values in *LAT_N* rounded to a scale of decimal places.
2. The sum of all values in *LONG_W* rounded to a scale of decimal places.

Input Format

The **STATION** table is described as follows:



where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(SUM(LAT_N), 2), ROUND(SUM(LONG_W), 2) FROM STATION;
```

Output:



35. Weather Observation Station 13

Query the sum of *Northern Latitudes* (*LAT_N*) from **STATION** having values greater than and less than . Truncate your answer to decimal places.

Input Format

The **STATION** table is described as follows:



where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(SUM(LAT_N), 4) FROM STATION WHERE LAT_N > 38.7880 AND LAT_N < 137.2345;
```

Output:



35. Weather Observation Station 14

Query the greatest value of the *Northern Latitudes* (*LAT_N*) from **STATION** that is less than . Truncate your answer to decimal places.

Input Format

The **STATION** table is described as follows:



where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(MAX(LAT_N), 4) FROM STATION WHERE LAT_N < 137.2345;
```

Output:



36. Weather Observation Station 15

Query the *Western Longitude* (*LONG_W*) for the largest *Northern Latitude* (*LAT_N*) in **STATION** that is less than . Round your answer to decimal places.

Input Format

The **STATION** table is described as follows:



where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(LONG_W, 4) FROM STATION WHERE LAT_N = (
SELECT MAX(LAT_N) FROM STATION WHERE LAT_N < 137.2345 );
```

Output:



37. Weather Observation Station 16

Query the smallest *Northern Latitude* (*LAT_N*) from **STATION** that is greater than . Round your answer to decimal places.

Input Format

The **STATION** table is described as follows:



where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```sql
SELECT ROUND(MIN(LAT_N), 4) FROM STATION WHERE LAT_N > 38.7780;
```

Output:



38. Weather Observation Station 17

Query the *Western Longitude* (*LONG_W*)where the smallest *Northern Latitude* (*LAT_N*) in **STATION** is greater than . Round your answer to decimal places.

Input Format

The **STATION** table is described as follows:



where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```sql
SELECT ROUND(LONG_W, 4) FROM STATION WHERE LAT_N = (
SELECT MIN(LAT_N) FROM STATION WHERE LAT_N > 38.7780
);
```

Output:



39. Weather Observation Station 18



Input:

```
SELECT ROUND(ABS(MAX(LAT_N) - MIN(LAT_N)) + ABS(MAX(LONG_W) - MIN(LONG_W)),4)
FROM STATION;
```

Output:



40. Weather Observation Station 19

Consider and to be two points on a 2D plane where are the respective minimum and maximum values of *Northern Latitude* (*LAT_N*) and are the respective minimum and maximum values of *Western Longitude* (*LONG_W*) in **STATION**.

Query the Euclidean Distance between points and and *format your answer* to display decimal digits.

Input Format

The **STATION** table is described as follows:



where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(SQRT(POWER(MAX(LAT_N) - MIN(LAT_N), 2) +
POWER(MAX(LONG_W)MIN(LONG_W), 2)), 4) FROM STATION;
```

Output:



41. Weather Observation Station 20

A *median* is defined as a number separating the higher half of a data set from the lower half. Query the *median* of the *Northern Latitudes* (*LAT_N*) from **STATION** and round your answer to decimal places.

Input Format

The **STATION** table is described as follows:



where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(S.LAT_N, 4)
FROM STATION AS S
WHERE (SELECT COUNT(LAT_N) FROM STATION WHERE LAT_N < S.LAT_N) = (SELECT
COUNT(LAT_N) FROM STATION WHERE LAT_N > S.LAT_N);
```

Output:



42. Population Census

Given the **CITY** and **COUNTRY** tables, query the sum of the populations of all cities where the *CONTINENT* is *'Asia'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows: 



Input:

```
SELECT SUM(CITY.POPULATION)
 FROM CITY JOIN COUNTRY
     ON CITY.COUNTRYCODE = COUNTRY.CODE
WHERE COUNTRY.CONTINENT = 'Asia';
```

Output:



43. African Cities

Given the **CITY** and **COUNTRY** tables, query the names of all cities where the *CONTINENT* is *'Africa'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows: 



Input:

```
SELECT CITY.NAME FROM CITYINNER JOIN COUNTRYON CITY.CountryCode =
COUNTRY.Code WHERE COUNTRY.Continent = 'Africa';
```

Output:



44. Average Population of Each Continent

Given the **CITY** and **COUNTRY** tables, query the names of all the continents (*COUNTRY.Continent*) and their respective average city populations (*CITY.Population*) rounded *down* to the nearest integer.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows: 



Input:

```
SELECT COUNTRY.Continent, FLOOR(AVG(CITY.Population))FROM CITY
INNER JOIN COUNTRY ON CITY.CountryCode = COUNTRY.Code
GROUP BY COUNTRY.Continent;
```

Output:



45. The Report

You are given two tables: *Students* and *Grades*. *Students* contains three columns *ID*, *Name* and *Marks*.



Grades contains the following data:



Ketty gives *Eve* a task to generate a report containing three columns: *Name*, *Grade* and *Mark*. *Ketty* doesn't want the NAMES of those students who received a grade lower than *8*. The report must be in descending order by grade -- i.e. higher grades are entered first. If there is more than one student with the same grade (8-10) assigned to them, order those particular students by their name alphabetically. Finally, if the grade is lower than 8, use "NULL" as their name and list them by their grades in descending order. If there is more than one student with the same grade (1-7) assigned to them, order those particular students by their marks in ascending order.

Write a query to help Eve.

Input:

```
SELECT
    CASE
        WHEN g.Grade < 8 THEN NULL
        ELSE s.Name
    END AS Name,
    g.Grade,
    s.Marks
FROM Students s
JOIN Grades g
  ON s.Marks BETWEEN g.Min_Mark AND g.Max_Mark
ORDER BY
    g.Grade DESC,
    CASE WHEN g.Grade >= 8 THEN s.Name END ASC,
    CASE WHEN g.Grade < 8 THEN s.Marks END ASC;
```

Output:

46. Top Competitors

Julia just finished conducting a coding contest, and she needs your help assembling the leaderboard! Write a query to print the respective *hacker_id* and *name* of hackers who achieved full scores for *more than one* challenge. Order your output in descending order by the total number of challenges in which the hacker earned a full score. If more than one hacker received full scores in same number of challenges, then sort them by ascending *hacker_id*.

Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.
- *Difficulty:* The *difficult_level* is the level of difficulty of the challenge, and *score* is the maximum score that can be achieved for a challenge at that difficulty level.
- *Challenges:* The *challenge_id* is the id of the challenge, the *hacker_id* is the id of the hacker who created the challenge, and *difficulty_level* is the level of difficulty of the challenge.

Input:

```
SELECT
    h.hacker_id,
    h.name
FROM submissions s
INNER JOIN hackers h
    ON s.hacker_id = h.hacker_id
INNER JOIN challenges c
    ON s.challenge_id = c.challenge_id
INNER JOIN difficulty d
    ON c.difficulty_level = d.difficulty_level
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(s.challenge_id) > 1
ORDER BY COUNT(s.challenge_id) DESC, h.hacker_id ASC;
```

Output:



47. Ollivander's Inventory

Harry Potter and his friends are at Ollivander's with Ron, finally replacing Charlie's old broken wand.

Hermione decides the best way to choose is by determining the minimum number of gold galleons needed to buy each *non-evil* wand of high power and age. Write a query to print the *id*, *age*, *coins_needed*, and *power* of the wands that Ron's interested in, sorted in order of descending *power*. If more than one wand has same power, sort the result in order of descending *age*.

Input Format

The following tables contain data on the wands in Ollivander's inventory:

- *Wands:* The *id* is the id of the wand, *code* is the code of the wand, *coins_needed* is the total number of gold galleons needed to buy the wand, and *power* denotes the quality of the wand (the higher the power, the better the wand is).

Column	Type
id	Integer
code	Integer
coins_needed	Integer
power	Integer

- *Wands_Property:* The *code* is the code of the wand, *age* is the age of the wand, and *is_evil* denotes whether the wand is good for the dark arts. If the value of *is_evil* is *0*, it means that the wand is not evil. The mapping between *code* and *age* is one-one, meaning that if there are two pairs, and , then and .

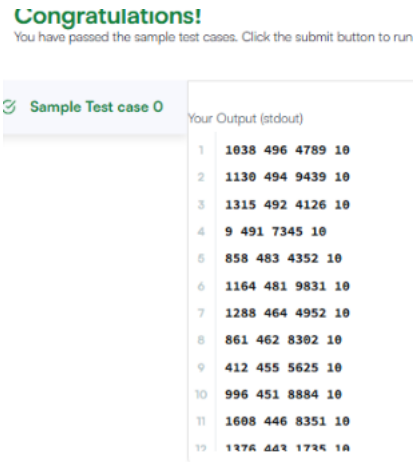
Column	Type
code	Integer
age	Integer
is_evil	Integer

Input:

```
SELECT
    w.id,
    p.age,
    w.coins_needed,
    w.power
FROM
    wands AS w
JOIN
    wands_property AS p ON w.code = p.code
WHERE
```

```
    p.is_evil = 0
    AND w.coins_needed = (
        SELECT
            MIN(w1.coins_needed)
        FROM
            wands AS w1
        JOIN
            wands_property AS p1 ON w1.code = p1.code
        WHERE
            p1.is_evil = 0
            AND w1.power = w.power
            AND p1.age = p.age
    )
ORDER BY
    w.power DESC,
    p.age DESC;
```

Output:



48. Challenges

Julia asked her students to create some coding challenges. Write a query to print the *hacker_id*, *name*, and the total number of challenges created by each student. Sort your results by the total number of challenges in descending order. If more than one student created the same number of challenges, then sort the result by *hacker_id*. If more than one student created the same number of challenges and the count is less than the maximum number of challenges created, then exclude those students from the result.

Input Format

The following tables contain challenge data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Challenges:* The *challenge_id* is the id of the challenge, and *hacker_id* is the id of the student who created the challenge.

Column	Type
challenge_id	Integer
hacker_id	Integer

Input:

```
SELECT H.HACKER_ID,
       H.NAME,
       COUNT(C.CHALLENGE_ID) AS C_COUNT
FROM HACKERS H
JOIN CHALLENGES C ON C.HACKER_ID = H.HACKER_ID
GROUP BY H.HACKER_ID, H.NAME
HAVING C_COUNT =
    (SELECT COUNT(C2.CHALLENGE_ID) AS C_MAX
     FROM CHALLENGES AS C2
     GROUP BY C2.HACKER_ID
     ORDER BY C_MAX DESC LIMIT 1)
OR C_COUNT IN
    (SELECT DISTINCT C_COMPARE AS C_UNIQUE
     FROM (SELECT H2.HACKER_ID,
                  H2.NAME,
                  COUNT(CHALLENGE_ID) AS C_COMPARE
           FROM HACKERS H2
           JOIN CHALLENGES C ON C.HACKER_ID = H2.HACKER_ID
           GROUP BY H2.HACKER_ID, H2.NAME) COUNTS
     GROUP BY C_COMPARE
     HAVING COUNT(C_COMPARE) = 1)
ORDER BY C_COUNT DESC, H.HACKER_ID;
```

Output:



49. Contest Leaderboard

You did such a great job helping Julia with her last oding contest challenge that she wants you to work on this one, too.The total score of a hacker is the sum of their maximum scores for all of the challenges. Write a query to print the *hacker_id*, *name*, and total score of the hackers ordered by the descending score. If more than one hacker achieved the same total score, then sort the result by ascending *hacker_id*. Exclude all hackers with a total score of from your result.

Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Submissions:* The *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, *challenge_id* is the id of the challenge for which the submission belongs to, and *score* is the score of the submission.

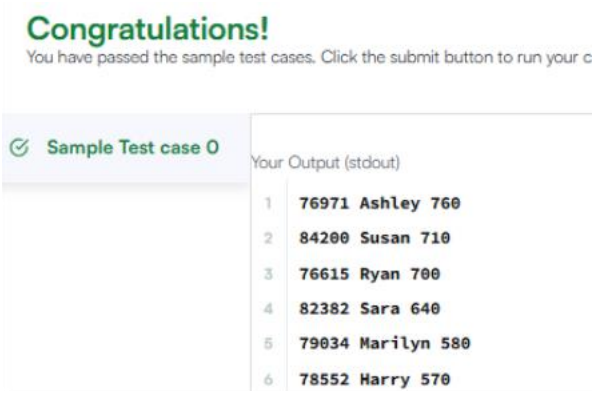
Column	Type
submission_id	Integer
hacker_id	Integer
challenge_id	Integer
score	Integer

Input:

```
SELECT X.hacker_id,
(SELECT H.NAME FROM HACKERS H
                    WHERE H.HACKER_ID = X.HACKER_ID) NAME,
SUM(X.SCORE) TOTAL_SCORE FROM
  (SELECT HACKER_ID, MAX(SCORE) SCORE FROM SUBMISSIONS S
```

```
   GROUP BY 1, S.CHALLENGE_ID) X
GROUP BY 1
HAVING TOTAL_SCORE > 0
ORDER BY TOTAL_SCORE DESC, HACKER_ID;
```

Output:



50. SQL Project Planning

You are given a table, *Projects*, containing three columns: *Task_ID*, *Start_Date* and *End_Date*. It is guaranteed that the difference between the *End_Date* and the *Start_Date* is equal to *1* day for each row in the table.

Column	*Type*
Task_ID	*Integer*
Start_Date	*Date*
End_Date	*Date*

If the *End_Date* of the tasks are consecutive, then they are part of the same project. Samantha is interested in finding the total number of different projects completed.

Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project.

Sample Input

Task_ID	*Start_Date*	*End_Date*
1	*2015-10-01*	*2015-10-02*
2	*2015-10-02*	*2015-10-03*
3	*2015-10-03*	*2015-10-04*
4	*2015-10-13*	*2015-10-14*
5	*2015-10-14*	*2015-10-15*
6	*2015-10-28*	*2015-10-29*
7	*2015-10-30*	*2015-10-31*

Sample Output

```
2015-10-28 2015-10-29
2015-10-30 2015-10-31
2015-10-13 2015-10-15
2015-10-01 2015-10-04
```

Input:

```
SELECT Start_Date, MIN(End_Date) AS Min_End_Date
FROM (
    SELECT Start_Date
    FROM Projects
    WHERE Start_Date NOT IN (SELECT End_Date FROM Projects)
) AS a
JOIN (
    SELECT End_Date
    FROM Projects
    WHERE End_Date NOT IN (SELECT Start_Date FROM Projects)
) AS b ON a.Start_Date < b.End_Date
GROUP BY Start_Date
ORDER BY DATEDIFF(MIN(End_Date), Start_Date) ASC, Start_Date ASC;
```

Output:



51. Placements

You are given three tables: *Students*, *Friends* and *Packages. Students* contains two columns: *ID* and *Name*. *Friends* contains two columns: *ID* and *Friend_ID* (*ID* of the ONLY best friend). *Packages* contains two columns: *ID* and *Salary* (offered salary in $ thousands per month).

Column	Type
ID	Integer
Name	String

Students

Column	Type
ID	Integer
Friend_ID	Integer

Friends

Column	Type
ID	Integer
Salary	Float

Packages

Write a query to output the names of those students whose best friends got offered a higher salary than them. Names must be ordered by the salary amount offered to the best friends. It is guaranteed that no two students got same salary offer.

Sample Input

ID	Name
1	Ashley
2	Samantha
3	Julia
4	Scarlet

Students

ID	Friend_ID
1	2
2	3
3	4
4	1

Friends

ID	Salary
1	15.20
2	10.06
3	11.55
4	12.12

Packages

Sample Output

```
Samantha
Julia
Scarlet
```

Explanation

See the following table:

ID	1	2	3	4
Name	Ashley	Samantha	Julia	Scarlet
Salary	15.20	10.06	11.55	12.12
Friend ID	2	3	4	1
Friend Salary	10.06	11.55	12.12	15.20

Now,

- *Samantha's* best friend got offered a higher salary than her at 11.55
- *Julia's* best friend got offered a higher salary than her at 12.12
- *Scarlet's* best friend got offered a higher salary than her at 15.2
- *Ashley's* best friend did NOT get offered a higher salary than her

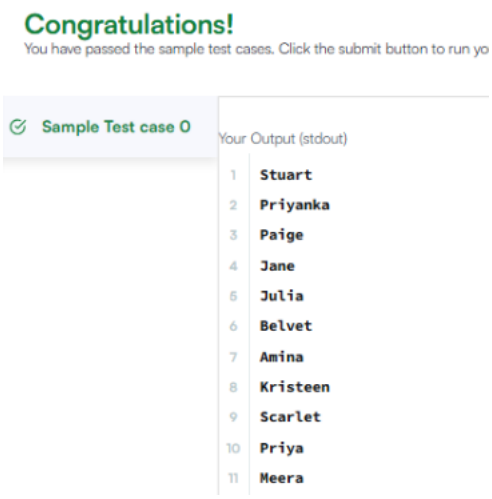
The name output, when ordered by the salary offered to their friends, will be:

- *Samantha*
- *Julia*
- *Scarlet*

Input:

```
SELECT S.NAME
FROM STUDENTS S
JOIN FRIENDS F ON S.ID = F.ID
JOIN PACKAGES P1 ON S.ID = P1.ID
JOIN PACKAGES P2 ON F.FRIEND_ID = P2.ID
WHERE P2.SALARY > P1.SALARY
ORDER BY P2.SALARY;
```

Output:



52. Symmetric Pairs

You are given a table, *Functions*, containing two columns: *X* and *Y*.

Column	*Type*
X	*Integer*
Y	*Integer*

Two pairs (X_1, Y_1) and (X_2, Y_2) are said to be *symmetric pairs* if $X_1 = Y_2$ and $X_2 = Y_1$.

Write a query to output all such *symmetric pairs* in ascending order by the value of *X*. List the rows such that $X_1 \leq Y_1$.

Sample Input

X	*Y*
20	*20*
20	*20*
20	*21*
23	*22*
22	*23*
21	*20*

Input:

```
SELECT f1.X, f1.Y FROM Functions f1
INNER JOIN Functions f2 ON f1.X = f2.Y AND f1.Y = f2.X
GROUP BY f1.X, f1.Y
```

```
HAVING f1.X < f1.Y OR COUNT(f1.X) > 1
ORDER BY f1.X;
```

Output:



53. 15 Days of Learning SQL

Julia conducted a days of learning SQL contest. The start date of the contest was *March 01, 2016* and the end date was *March 15, 2016*.

Write a query to print total number of unique hackers who made at least submission each day (starting on the first day of the contest), and find the *hacker_id* and *name* of the hacker who made maximum number of submissions each day. If more than one such hacker has a maximum number of submissions, print the lowest *hacker_id*. The query should print this information for each day of the contest, sorted by the date.

Input Format

The following tables hold contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Submissions:* The *submission_date* is the date of the submission, *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, and *score* is the score of the submission.

Column	Type
submission_date	Date
submission_id	Integer
hacker_id	Integer
score	Integer

Sample Input

For the following sample input, assume that the end date of the contest was *March 06, 2016*.

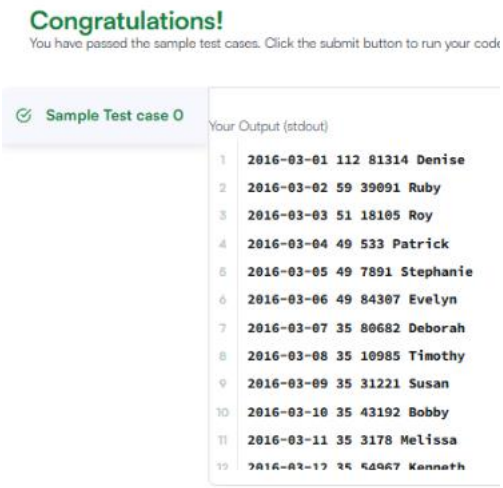
submission_date	submission_id	hacker_id	score
2016-03-01	8494	20703	0
2016-03-01	22403	53473	15
2016-03-01	23965	79722	60
2016-03-01	30173	36396	70
2016-03-02	34928	20703	0
2016-03-02	38740	15758	60
2016-03-02	42769	79722	25
2016-03-02	44364	79722	60
2016-03-03	45440	20703	0
2016-03-03	49050	36396	70
2016-03-03	50273	79722	5
2016-03-04	50344	20703	0
2016-03-04	51360	44065	90
2016-03-04	54404	53473	65
2016-03-04	61533	79722	45
2016-03-05	72852	20703	0
2016-03-05	74546	38289	0
2016-03-05	76487	62529	0
2016-03-05	82439	36396	10
2016-03-05	90006	36396	40
2016-03-06	90404	20703	0

Input:

```
SELECT SUBMISSION_DATE,
(SELECT COUNT(DISTINCT HACKER_ID)
 FROM SUBMISSIONS S2
```

```
 WHERE S2.SUBMISSION_DATE = S1.SUBMISSION_DATE AND
(SELECT COUNT(DISTINCT S3.SUBMISSION_DATE)
 FROM SUBMISSIONS S3 WHERE S3.HACKER_ID = S2.HACKER_ID AND S3.SUBMISSION_DATE
< S1.SUBMISSION_DATE) = DATEDIFF(S1.SUBMISSION_DATE , '2016-03-01')),
(SELECT HACKER_ID FROM SUBMISSIONS S2 WHERE S2.SUBMISSION_DATE =
S1.SUBMISSION_DATE
GROUP BY HACKER_ID ORDER BY COUNT(SUBMISSION_ID) DESC, HACKER_ID LIMIT 1) AS
TMP,
(SELECT NAME FROM HACKERS WHERE HACKER_ID = TMP)
FROM
(SELECT DISTINCT SUBMISSION_DATE FROM SUBMISSIONS) S1
GROUP BY SUBMISSION_DATE;
```

Output:



54. Draw The Triangle 1

P(R) represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:

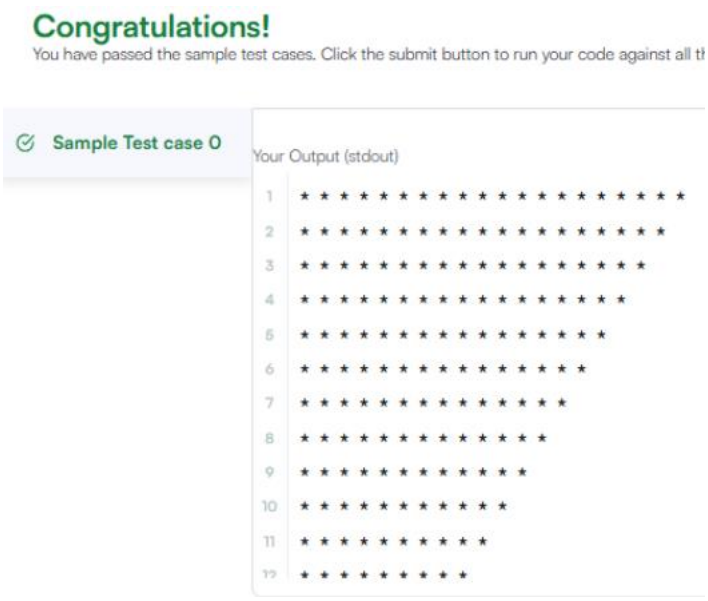
```
* * * * *
* * * *
* * *
* *
*
```

Write a query to print the pattern *P(20)*.

Input:

```
SET @TEMP := 21;
SELECT REPEAT('* ', @TEMP := @TEMP - 1)
FROM INFORMATION_SCHEMA.TABLES;
```

Output:



55. Draw The Triangle 2

P(R) represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:

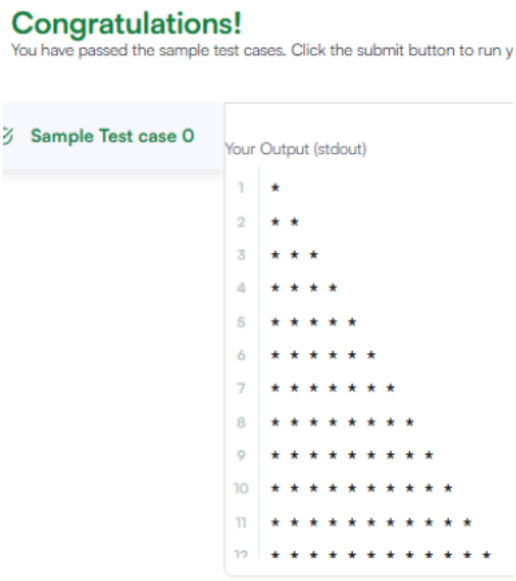
```
*
* *
* * *
* * * *
* * * * *
```

Write a query to print the pattern *P(20)*.

Input:

```
SET @number = 0;
SELECT REPEAT('* ', @number := @number + 1)
FROM information_schema.tables
WHERE @number < 20;
```

Output:



56. Print Prime Numbers

Write a query to print all *prime numbers* less than or equal to . Print your result on a single line, and use the ampersand () character as your separator (instead of a space).

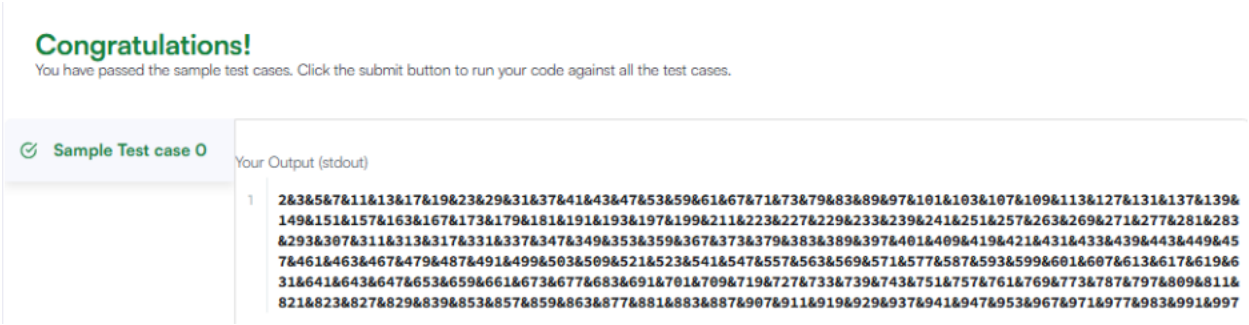
For example, the output for all prime numbers would be:

```
2&3&5&7
```

Input:

```
WITH RECURSIVE nums AS (
    SELECT 2 AS n
    UNION ALL
    SELECT n + 1 FROM nums WHERE n < 1000
)
SELECT GROUP_CONCAT(n SEPARATOR '&')
FROM nums p
WHERE NOT EXISTS (
    SELECT 1 FROM nums d WHERE d.n <= SQRT(p.n) AND d.n > 1 AND p.n % d.n = 0
);
```

Output:



57. Interviews

Samantha interviews many candidates from different colleges using coding challenges and contests. Write a query to print the *contest_id*, *hacker_id*, *name*, and the sums of *total_submissions*, *total_accepted_submissions*, *total_views*, and *total_unique_views* for each contest sorted by *contest_id*. Exclude the contest from the result if all four sums are .

Note: A specific contest can be used to screen candidates at more than one college, but each college only holds screening contest.

Input Format

The following tables hold interview data:

- *Contests:* The *contest_id* is the id of the contest, *hacker_id* is the id of the hacker who created the contest, and *name* is the name of the hacker.

Column	Type
contest_id	Integer
hacker_id	Integer
name	String

- *Colleges:* The *college_id* is the id of the college, and *contest_id* is the id of the contest that Samantha used to screen the candidates.

Column	Type
college_id	Integer
contest_id	Integer

- *Challenges:* The *challenge_id* is the id of the challenge that belongs to one of the contests whose contest_id Samantha forgot, and *college_id* is the id of the college where the challenge was given to candidates.

Column	Type
challenge_id	Integer
college_id	Integer

- *View_Stats:* The *challenge_id* is the id of the challenge, *total_views* is the number of times the challenge was viewed by candidates, and *total_unique_views* is the number of times the challenge was viewed by unique candidates.

Column	Type
challenge_id	Integer
total_views	Integer
total_unique_views	Integer

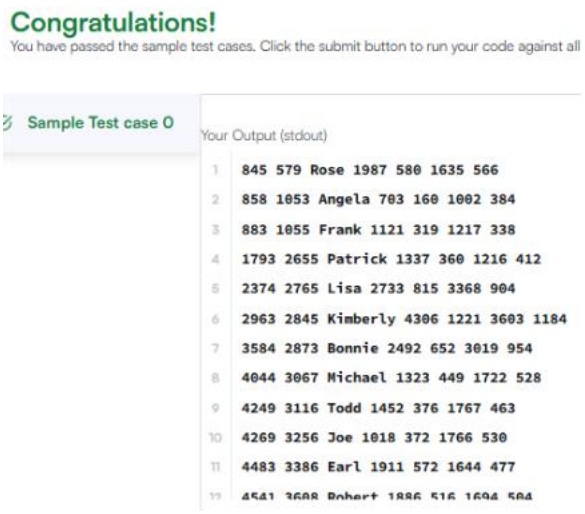
- *Submission_Stats:* The *challenge_id* is the id of the challenge, *total_submissions* is the number of submissions for the challenge, and *total_accepted_submission* is the number of submissions that achieved full scores.

Column	Type
challenge_id	Integer
total_submissions	Integer
total_accepted_submissions	Integer

Input:

```
SELECT t1.contest_id, t1.hacker_id, t1.name,
       SUM(ts) as ts, SUM(tas) as tas, SUM(tv) as tv, SUM(tuv) as tuv
FROM CONTESTS AS t1
LEFT JOIN COLLEGES AS t2
    ON t1.contest_id = t2.contest_id
LEFT JOIN CHALLENGES AS t3
    ON t2.college_id = t3.college_id
LEFT JOIN (SELECT challenge_id,
           SUM(total_views) AS tv, SUM(total_unique_views) as tuv
           FROM View_Stats
           GROUP BY challenge_id) AS t4
    ON t3.challenge_id = t4.challenge_id
LEFT JOIN (SELECT challenge_id,
           SUM(total_submissions) AS ts, SUM(total_accepted_submissions) as
tas
           FROM Submission_Stats
           GROUP BY challenge_id) as t5
    ON t3.challenge_id = t5.challenge_id
GROUP BY t1.contest_id, t1.hacker_id, t1.name
HAVING (ts + tas + tv + tuv > 0)
ORDER BY t1.contest_id;
```

Output:



CODE CHEF

1. ALL PRODUCTS

Task:

Write a query to output `Products` table.

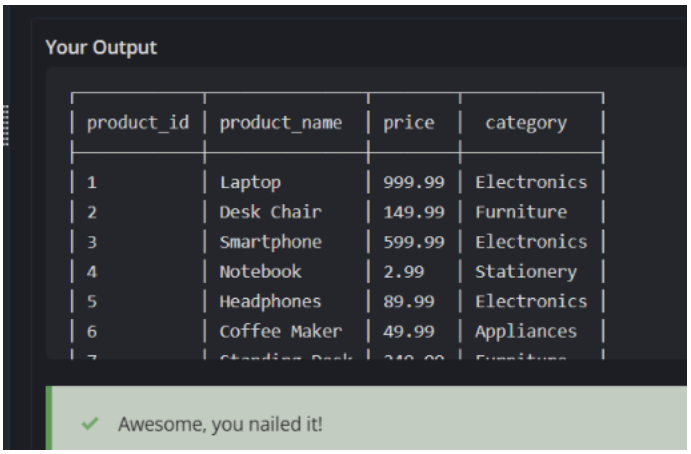
Table Name: `Products`

Product_ID	Product_Name	Price	Category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

Input:

```
SELECT * FROM Products;
```

Output:



2. HIGH PRICE OF PRODUCTS

Task:

Write a query to find all `product_name` and `category` that have a price greater than 100.00 from the **Products** table.

Table Name: `Products`

Product_ID	Product_Name	Price	Category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

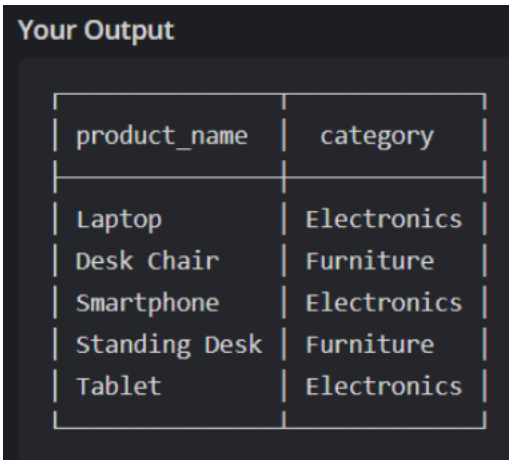
Expected Columns:

- product_name
- category

Input:

```sql
SELECT product_name, category FROM Products WHERE price > 100.00;
```

Output:



3. AVERAGE SALARY

Task:

Write a query to calculate the average salary across all companies combined. Rename the column as `avg_salary`.

Table Name: `Works`

Employee_ID	Company_Name	Salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

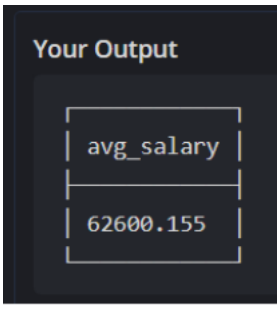
Expected Column:

- avg_salary

Input:

```sql
SELECT AVG(salary) AS avg_salary FROM Works;
```

Output:



4. LOCATE PEOPLE

Task:

Write a query to retrieve the `department_name` and `location` of departments located in a city that starts with 'S'.

Table Name: `departments`

Department_ID	Department_Name	Location
1	Human Resources	New York
2	Research and Development	San Francisco
3	Sales	Los Angeles
4	Marketing	New York
5	Customer Support	Boston
6	Finance	Austin
7	IT Support	Seattle
8	Product Management	San Francisco
9	Quality Assurance	Los Angeles
10	Legal	Bostn

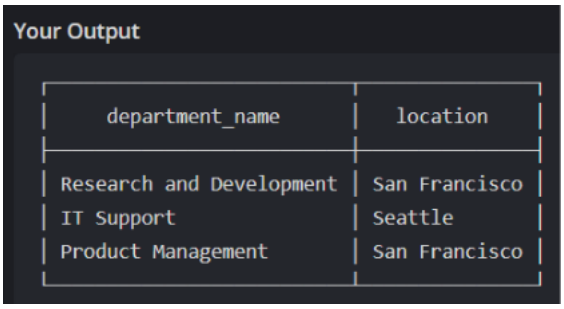
Expected Columns:

- department_name
- location

Input:

```sql
SELECT department_name, location FROM departments WHERE location LIKE 'S%';
```

Output:



5. DISTINCT COMPANIES

Task:

Write a query to select all the distinct companies (`company_name`) in the `Works` table.

Table Name: `Works`

Employee_ID	Company_Name	Salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

Expected Column:

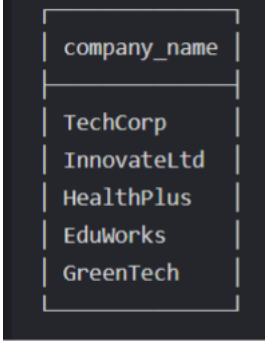
- company name

Input:

```
SELECT DISTINCT company_name FROM Works;
```

Output:

Your Output



6. FICTION COLLECTION SIZE

Task:

Write a query to find the total count of books whose genre is **Fiction**.

Note: Output column name should be **fiction_count**.

Table Name: Books

ID	Title	Author	Genre	Price	Published_Year
1	The Great Gatsby	F. Scott Fitzgerald	Fiction	10.99	1925
2	1984	George Orwell	Dystopian	15.99	1949
3	To Kill a Mockingbird	Harper Lee	Fiction	12.99	1960
4	The Catcher in the Rye	J.D. Salinger	Fiction	14.99	1951
5	Brave New World	Aldous Huxley	Dystopian	13.99	1932
6	The Hobbit	J.R.R. Tolkien	Fantasy	9.99	1937
7	Moby Dick	Herman Melville	Fiction	18.50	1851
8	War and Peace	Leo Tolstoy	Historical	20.00	1869
9	The Picture of Dorian Gray	Oscar Wilde	Fiction	11.50	1890

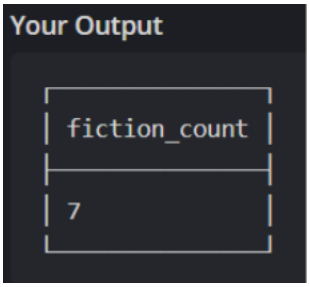
10	The Alchemist	Paulo Coelho	Fiction	16.00	1988
11	Fahrenheit 451	Ray Bradbury	Dystopian	12.50	1953
12	The Chronicles of Narnia	C.S. Lewis	Fantasy	14.00	1950
13	The Handmaid's Tale	Margaret Atwood	Dystopian	15.50	1985
14	A Tale of Two Cities	Charles Dickens	Historical	13.00	1859
15	Little Women	Louisa May Alcott	Fiction	9.50	1868

Expected Column:

- fiction_count
- **Input:**

```
SELECT COUNT(*) AS fiction_count FROM Books WHERE genre = 'Fiction';
```

Output:



7. LIST OF MOVIES WITH RATINGS

Task:

Write a query to select only the movie names where the ratings are greater than 7 but less than 9.

Table Name: `Cinema`

Movie_ID	Movie_Name	Description	Rating
1	War	great 3D	8.9

2	Science	fiction	8.5
3	Irish	boring	6.2
4	Ice Song	Fantasy	8.6
5	House Card	Interesting	9.1
6	The Escape	Thriller	7.8
7	Solar Flare	Sci-Fi	8.3
8	The Joker	Drama	9.0
9	Lost Dreams	Mystery	7.5
10	Galaxy War	Action	8.7

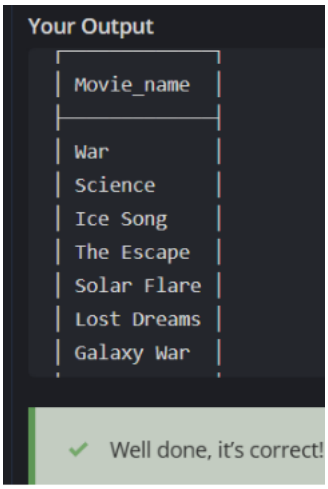
Expected Column:

- Movie_name

Input:

```
SELECT Movie_name FROM Cinema WHERE Rating > 7.0 AND Rating < 9.0;
```

Output:



8. HANDLING NULL VALUES

Task:

Write a query to retrieve `book_id`, `title`, `author` and `published_year` of the books which have **NULL** rating for their books.

Table Name: `Library`

Book_ID	Title	Author	Published_Year	Rating
1	The Great Gatsby	F. Scott Fitzgerald	1925	4.2
2	To Kill a Mockingbird	Harper Lee	1960	NULL
3	1984	George Orwell	1949	4.8
4	The Catcher in the Rye	J.D. Salinger	1951	NULL
5	Brave New World	Aldous Huxley	1932	4.3

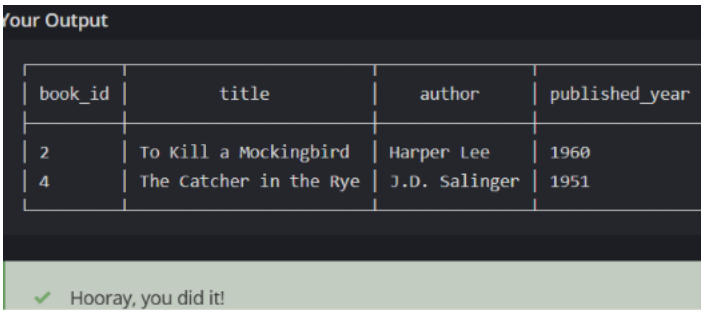
Expected Columns:

- book_id
- title
- author
- published_year

Input:

```sql
SELECT book_id, title, author, published_year FROM Library WHERE rating IS NULL;
```

Output:



9. SALARY OF EMPLOYEES

Task:

Create a query to retrieve the `employee_name`, `company`, and `salary` for employees in the full-time category, ordered by salary in **descending** order.

Table Name: `Employees`

Employee_ID	Employee_Name	Company	Category	Department	Salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000

5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	IT	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

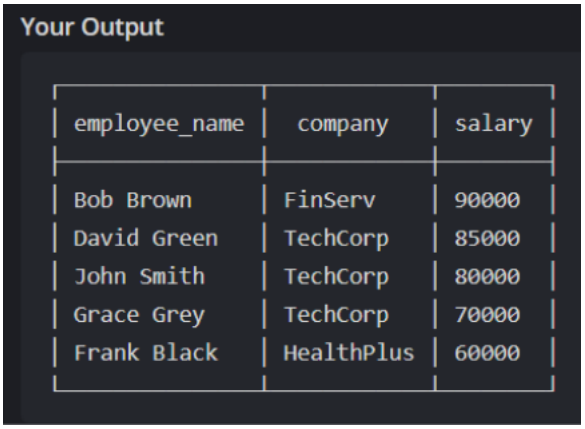
Expected Columns:

- employee_name
- company
- salary

Input:

```
SELECT employee_name, company, salary FROM Employees WHERE category = 'Full-Time' ORDER BY salary DESC;
```

Output:



10. DEPARTMENT OF EACH EMPLOYEE

Task:

Write a query to group the employees by their department and display the total number of employees (as `total_employees`) in each department.

Table Name: `Employees`

Employee_ID	Employee_Name	Company	Category	Department	Salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000

3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	Sales	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

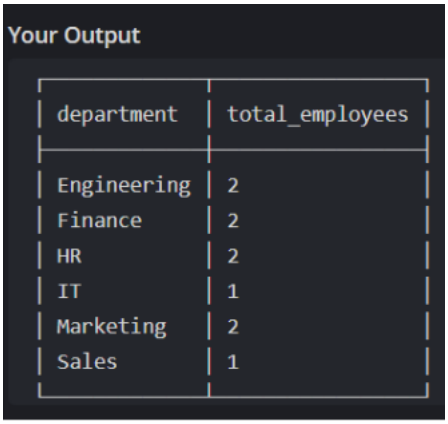
Expected Column:

- department
- total_employees

Input:

```sql
SELECT department, COUNT(*) AS total_employees FROM Employees GROUP BY department;
```

Output:



11. ARTICLE VIEWS

Task:

Write a query to retrieve the `author_id`, `author_name`, and `publication_name` for authors whose articles got zero views. The result should be sorted by `author_id` in ascending order.

Return the result table sorted by id in **ascending order**.

Table Name: `Views`

Article_ID	Author_ID	Author_Name	Viewer_ID	View_Count	Publication_Name
101	1	John Doe	1	3	The Daily Times
102	2	Jane Smith	2	1	Global News
103	3	Emily Clark	4	0	Tech Monthly
104	4	Mark Lewis	4	2	Health Digest
105	5	Sara White	3	0	The Business Journal
106	1	John Doe	2	0	The Daily Times
107	2	Jane Smith	2	2	Global News

Expected Columns:

- author_id
- author_name
- publication_name

Input:

```
SELECT DISTINCT author_id, author_name, publication_name FROM Views WHERE view_count = 0
ORDER BY author_id ASC;
```

Output:

Your Output

author_id	author_name	publication_name
1	John Doe	The Daily Times
3	Emily Clark	Tech Monthly
5	Sara White	The Business Journal

12. PLAYER PERFORMANCE INSIGHTS

Task:

Write a query to find the names of the **top 3 distinct players** by highest score who have **won** matches, including their **scores**.

Table 1: `Players`

Player_ID	Player_Name	Score	Rank
1	Alice	1200	5

2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6

Table 2: `Matches`

Match_ID	Player1	Player2	Winner	Match_Date
101	Alice	Bob	Bob	2024-01-15
102	Charlie	David	David	2024-01-16
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-18
105	Charlie	Eve	Charlie	2024-01-19

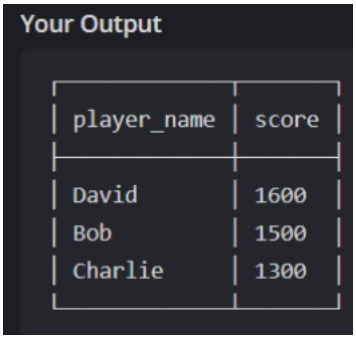
Expected Columns:

- player_name
- score

Input:

```
SELECT DISTINCT player_name, score
FROM Players JOIN Matches WHERE winner=player_name
ORDER BY score DESC
LIMIT 3;
```

Output:



13. PLAYER DETAILS

Task:

Write a query to retrieve the details of the **last five matches** played, including the match ID, the names of the players who participated, the name of the winning player, match date, and the final score of the winner.

There are two tables named **`Players`** and **`Matches`**.

Table 1: `Players`

Player_ID	Player_Name	Score	Rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6
6	Frank	1450	3
7	Grace	1350	7
8	Hank	1250	9
9	Ivy	1550	8
10	Jack	1400	10

Table 2: `Matches`

Match_ID	Player_1	Player_2	Winner	Match_Date
101	Alice	Bob	Bob	2024-01-25
102	Charlie	David	David	2024-01-06
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-01
105	Charlie	Eve	Charlie	2024-01-15
106	Frank	Hank	Frank	2024-01-29
107	Grace	Ivy	Ivy	2024-01-10
108	Jack	Alice	Jack	2024-01-19
109	Bob	Charlie	Bob	2024-01-13
110	David	Eve	David	2024-01-24

Expected Output Columns:

- match_id
- player_1
- player_2
- winner
- match_date
- score

Input:

```
SELECT m.match_id, m.player_1, m.player_2, m.winner, m.match_date, p.score
FROM Matches m
JOIN Players p ON m.winner = p.player_name
ORDER BY m.match_date DESC
LIMIT 5;
```

Output:

